

# Open-FDD Documentation

Generated by scripts/build\_docs\_pdf.py

June 9, 2026

- 1 Home
- 2 Open-FDD
  - 2.1 Who it is for
  - 2.2 Start here
  - 2.3 v3 edge stack
  - 2.4 Distribution
  - 2.5 License
- 3 Run with Docker images
- 4 Run with Docker images
  - 4.1 Images
  - 4.2 1. Install and validate Docker
  - 4.3 2. Bootstrap the site (one script)
    - 4.3.1 Manual start (optional)
  - 4.4 Long-term operation
    - 4.4.1 Survive power cycles
    - 4.4.2 Start / stop / restart
    - 4.4.3 LAN access
  - 4.5 Deployment choices
  - 4.6 Next steps
- 5 First login and health check
- 6 First login and health check
  - 6.1 Public health
  - 6.2 Login
  - 6.3 Smoke checklist
  - 6.4 Stack detail (authenticated)
  - 6.5 Troubleshooting
- 7 Quick Start
- 8 Quick Start
- 9 Updating the stack
- 10 Updating the stack
  - 10.1 Before you start
  - 10.2 Fetch helper scripts (no git clone)
  - 10.3 One-command upgrade (recommended)
  - 10.4 Step by step
    - 10.4.1 1. Backup site state
    - 10.4.2 2. Verify images (optional)
    - 10.4.3 3. Pull and recreate

- 10.4.4 4. Verify
- 10.5 Dashboard UI updates
- 10.6 Rollback
- 10.7 Safe cleanup
- 10.8 Next steps
- 11 Local development
- 12 Local development
  - 12.1 Clone and auth
  - 12.2 Start stack
  - 12.3 BACnet bench overlay (optional)
  - 12.4 Production-like local stack (Caddy, LAN scans)
  - 12.5 Docs preview
- 13 Build Docker images
- 14 Build Docker images
  - 14.1 Local build
  - 14.2 Publish to GHCR (maintainers)
- 15 Developer Guide
- 16 Developer Guide
- 17 Tests and CI
- 18 Tests and CI
  - 18.1 Local tests
  - 18.2 CI (GitHub Actions)
  - 18.3 Docs build (same as CI)
  - 18.4 Security-related CI
- 19 Contributing
- 20 Contributing
  - 20.1 Expectations
  - 20.2 Pull request checklist
  - 20.3 Repository layout (short)
  - 20.4 Security issues
  - 20.5 Agent-assisted development
- 21 Health checks (developer)
- 22 Health checks (developer)
  - 22.1 Clone and local stack
  - 22.2 Stack health script
  - 22.3 Compose overlays
  - 22.4 BACnet poll pipeline
  - 22.5 pytest (bridge security / BACnet)
  - 22.6 Pentest production stack (LAN ZAP)
  - 22.7 Post-deploy insurance (inventory hosts)
- 23 Security testing cycle
- 24 Security testing cycle
  - 24.1 Who owns what
  - 24.2 Per-revision workflow
    - 24.2.1 1. Local functional tests
    - 24.2.2 2. Pentest production stack (on Open-FDD host)
    - 24.2.3 3. Host + exposure checks (on edge VM)
    - 24.2.4 4. LAN security scan (from laptop)
    - 24.2.5 5. Pull request + automated review
    - 24.2.6 6. Release
    - 24.2.7 7. Lab edge validation
    - 24.2.8 8. Next cycle

- 24.3 What each scan layer covers
- 24.4 Open-source contributors
- 25 Expression cookbook (Arrow-native)
- 26 Expression cookbook (Arrow-native)
  - 26.1 Legacy → modern translation
  - 26.2 How Arrow rules are structured
    - 26.2.1 Command scaling (0-1 vs 0-100 %)
    - 26.2.2 Occupied hours (schedule gating)
  - 26.3 Sensor validation (bounds, flatline, rate of change)
    - 26.3.1 Bounds (out of range) — oob\_rolling
    - 26.3.2 Flatline (stuck sensor) — flatline\_1h
    - 26.3.3 Rate of change (spike) — rate\_of\_change
    - 26.3.4 Mixing envelope (MAT vs OAT/RAT) — mixing\_envelope
  - 26.4 GL36-inspired AHU rules (legacy expression → fault code)
    - 26.4.1 Rule A — duct static low at full speed (Arrow sketch)
  - 26.5 Starter pack (VAV / AHU baseline)
    - 26.5.1 Economizer starters
  - 26.6 VAV, central plant, heat pump
  - 26.7 Opportunistic / ventilation
  - 26.8 Data quality vs equipment faults
  - 26.9 Metric (°C) equivalents
  - 26.10 Binding rules to the fault catalog
  - 26.11 Pandas YAML → Arrow checklist (commissioning)
- 27 System overview
- 28 System overview
  - 28.1 Docker edge stack (production)
  - 28.2 Components
  - 28.3 Deployment
  - 28.4 Optional PyPI-only path
- 29 Arrow recipes
- 30 Arrow recipes (default)
  - 30.1 Simple threshold
  - 30.2 Supply air temp high
  - 30.3 Fan commanded but no airflow
  - 30.4 Flatline (1 h rolling window)
  - 30.5 Sensor out of range (catalog defaults)
  - 30.6 Sensor flatline + spike (catalog)
- 31 Deployment modes
- 32 Deployment modes
  - 32.1 Mode summary
  - 32.2 v3 container stack (all edge modes)
  - 32.3 Caddy HTTP (typical edge)
  - 32.4 Caddy TLS (OT LAN)
  - 32.5 BACnet data path
  - 32.6 What not to do on production edges
  - 32.7 Related
- 33 Python recipes (Arrow)
- 34 Python recipes (Arrow)
- 35 Containers
- 36 Containers
  - 36.1 GHCR images
  - 36.2 Stack diagram

- 36.3 Responsibilities
- 36.4 Network and ports (typical edge)
- 36.5 Long-lived deployment
- 36.6 Persistent data
- 36.7 Optional host services
- 37 Lookback window helper
- 38 Lookback window helper
  - 38.1 Recommended pattern (copy into rule.py)
  - 38.2 Why not hide this behind a library call yet?
  - 38.3 Lookback vs rolling samples
  - 38.4 Batch intervals
- 39 Data flow
- 40 Data flow
  - 40.1 Telemetry ingest
    - 40.1.1 BACnet
    - 40.1.2 Modbus TCP
    - 40.1.3 JSON API (HTTP/HTTPS)
  - 40.2 Rule evaluation
  - 40.3 Model sync
  - 40.4 Retention
- 41 Architecture
- 42 Architecture
- 43 Rule Cookbook
- 44 Rule Cookbook
- 45 Windowing & debugging
- 46 Windowing & debugging
  - 46.1 Lookback vs rolling window
  - 46.2 Rolling windows (Arrow)
  - 46.3 Rule Lab console
- 47 GL36 algorithm stubs
- 48 GL36 algorithm stubs (doc-only)
  - 48.1 Shared lookback helper
  - 48.2 VAV zone cooling requests (0-3)
  - 48.3 AHU duct static trim & respond (advisory fault)
  - 48.4 Chiller plant enable (valve request counting)
  - 48.5 Hot water plant HWST trim & respond (advisory)
  - 48.6 Chilled water plant (DP + CHWST reset advisory)
  - 48.7 Testing later
- 49 Dashboard overview
- 50 Dashboard overview
  - 50.1 Primary areas
  - 50.2 Live updates
  - 50.3 Roles
- 51 Auth and roles
- 52 Auth and roles
  - 52.1 Production (LAN / edge)
  - 52.2 Localhost dev
- 53 Rule Lab
- 54 Rule Lab
  - 54.1 Workflow
  - 54.2 Dev kit zip (download)
  - 54.3 Lookback windows

- 54.4 Algorithms (supervisory — coming soon)
- 54.5 Related
- 55 JSON API driver
- 56 JSON API driver
  - 56.1 Supported features
  - 56.2 Secrets: `json_api.env.local`
  - 56.3 OpenWeatherMap showcase (recommended demo)
    - 56.3.1 1. Configure env
    - 56.3.2 2. Register from the UI
    - 56.3.3 3. Headless / script
    - 56.3.4 4. FDD: local OA-T vs web weather
  - 56.4 Commissioning (UI)
    - 56.4.1 API for agents
    - 56.4.2 Bench preset
  - 56.5 Authentication examples
    - 56.5.1 Query-string API key via env (OpenWeather)
    - 56.5.2 Bearer token (API key)
    - 56.5.3 HTTP Basic
    - 56.5.4 POST with JSON body
  - 56.6 REST API
  - 56.7 Typical OT URLs
  - 56.8 Limitations
  - 56.9 Disable polling (save API quota)
  - 56.10 Smoke test
- 57 Algorithms
- 58 Algorithms (coming soon)
  - 58.1 GL36 Trim & Respond reference
  - 58.2 Planned workflow (same kit shape as Rule Lab)
  - 58.3 Lookback windows
  - 58.4 AHU supervisory checks (doc-only stubs)
    - 58.4.1 Duct static pressure too high
    - 58.4.2 Supply air temperature too cold (cooling coil over-delivering)
  - 58.5 Plant supervisory checks (doc-only)
  - 58.6 Related
- 59 Model workflow
- 60 Model workflow
  - 60.1 Brick model
  - 60.2 Typical integrator flow
- 61 Driver framework
- 62 Driver framework
  - 62.1 Drivers
  - 62.2 Data layout
  - 62.3 Historian and FDD
  - 62.4 Rule Lab (Arrow upload/download)
  - 62.5 Smoke validation
  - 62.6 Related docs
- 63 Operator Bridge
- 64 Operator Bridge
- 65 Ollama and analytics
- 66 Ollama and analytics
  - 66.1 What runs without login

- 66.2 Ollama setup
- 66.3 Automatic analytics (deterministic, always on)
- 66.4 When Ollama is used
- 66.5 AI Agent (authenticated)
- 66.6 Code map
- 67 FDD and assignments
- 68 FDD and assignments
  - 68.1 What the product actually does
    - 68.1.1 BRICK modeling (integrator + AI-assisted)
    - 68.1.2 Rule assignment (three binding kinds)
    - 68.1.3 CLASS vs device
  - 68.2 Commissioning JSON shape
    - 68.2.1 Rule names vs ids (Rule Lab alignment)
  - 68.3 Related
- 69 Network setup
- 70 Network setup
  - 70.1 Bind address
  - 70.2 Discovery range
  - 70.3 Verify
- 71 Discover and read
- 72 Discover and read
  - 72.1 Operator workflow
- 73 Polling
- 74 Polling
  - 74.1 Commission poll loop (default)
  - 74.2 Bridge ingest worker
  - 74.3 Health
- 75 Write safety
- 76 Write safety
  - 76.1 Defaults
  - 76.2 Enable writes (commission role)
  - 76.3 Operator rules
- 77 BACnet
- 78 BACnet
- 79 Convention
- 80 Fault code convention
  - 80.1 Format
  - 80.2 Categories
  - 80.3 Severity
  - 80.4 Linking rules
  - 80.5 Cookbook mapping
- 81 Live site update
- 82 Live site update
  - 82.1 Layout on the edge host
  - 82.2 Concepts
  - 82.3 1. Verify new images on GHCR
  - 82.4 2. Update dashboard UI (when release changed React)
  - 82.5 3. Pull and recreate (edge host)
  - 82.6 4. Ansible image-only (control machine)
  - 82.7 5. Validate
  - 82.8 Rollback
  - 82.9 Related

- 83 AHU faults
- 84 AHU faults
- 85 Backup and restore
- 86 Backup and restore
  - 86.1 Quick backup (edge host)
  - 86.2 Manual backup (SSH)
  - 86.3 What to protect
  - 86.4 Restore after a bad upgrade
  - 86.5 Safe cleanup
  - 86.6 Related
- 87 VAV faults
- 88 VAV faults
- 89 RTU faults
- 90 RTU faults
- 91 Deployment validation
- 92 Deployment validation
  - 92.1 Edge host smoke (5 minutes)
  - 92.2 Browser checklist
  - 92.3 Control-machine insurance check
  - 92.4 LAN security smoke (optional)
  - 92.5 Arrow / FDD rules (3.0+)
  - 92.6 Related
- 93 Sensor and data quality
- 94 Sensor and data quality
  - 94.1 Catalog codes
  - 94.2 Default bounds (imperial, occupied)
  - 94.3 Communication quality
  - 94.4 Operator workflow
- 95 Logging and audit
- 96 Logging and audit
  - 96.1 Log types
  - 96.2 Audit record schema
    - 96.2.1 Auth events (industry-standard login trail)
    - 96.2.2 OT / commissioning events
    - 96.2.3 Secret redaction
  - 96.3 Integrator API (read logs in the UI stack)
  - 96.4 Retention and rotation (defaults)
  - 96.5 Docker edge: json-file caps (not journald for app audit)
  - 96.6 Local dev (run\_local.sh)
  - 96.7 Quick checks
  - 96.8 Comparison to typical web apps
  - 96.9 Related
- 97 Fault Codes
- 98 Fault Codes
- 99 Operations
- 100 Operations
- 101 LAN hardening
- 102 LAN hardening
  - 102.1 Auth and bind
  - 102.2 Network exposure
  - 102.3 Rule Lab
  - 102.4 TLS and LAN security scans

- 102.5 Ollama on the edge
- 103 ZAP baseline (local bench)
- 104 ZAP baseline (local bench)
  - 104.1 Run
  - 104.2 Expected results (HTTP bench mode)
  - 104.3 Header ownership
  - 104.4 Authenticated scan (later)
  - 104.5 After fixes
- 105 TLS and certificates
- 106 TLS and certificates
  - 106.1 Ownership
  - 106.2 Generate bench certs (local)
  - 106.3 ZAP / Nmap with TLS bench
  - 106.4 Renewal
  - 106.5 Enterprise CA on Caddy (edge)
  - 106.6 Caddy internal CA mode (lab)
  - 106.7 Secrets
- 107 Authenticated scanning (roadmap)
- 108 Authenticated scanning (roadmap)
  - 108.1 Current state (3.0.x)
  - 108.2 Why baseline is not enough
  - 108.3 Planned approach (contributors welcome)
  - 108.4 Manual authenticated ZAP today
  - 108.5 Related
- 109 Secrets and auth
- 110 Secrets and auth
  - 110.1 Files (gitignored)
  - 110.2 Required variables (production)
  - 110.3 GHCR pulls
  - 110.4 Backup
- 111 BACnet write allowlists
- 112 BACnet write allowlists
- 113 Linux host hardening
- 114 Linux host hardening
  - 114.1 Scope
  - 114.2 Quick operator workflow
  - 114.3 Ubuntu version and EOL
  - 114.4 Kernel and package updates
  - 114.5 Host Python (pip, wheel, pyOpenSSL)
  - 114.6 Ollama — no LAN exposure
  - 114.7 OpenSSH Terrapin (CVE-2023-48795)
  - 114.8 ICMP timestamp (Low)
  - 114.9 TLS and certificate scans
  - 114.10 Maintainer security audits (CI parity)
- 115 Tenable / Nessus remediation
- 116 Tenable / Nessus remediation
  - 116.1 Prioritized action plan
  - 116.2 Finding classification table
  - 116.3 Operator commands (edge VM)
    - 116.3.1 Validate (read-only)
    - 116.3.2 Remediate host (maintenance window)
    - 116.3.3 Remediate Open-FDD containers



|         |                                            |
|---------|--------------------------------------------|
| 116.3.4 | Rescan                                     |
| 116.4   | Ollama unauthenticated access (Critical)   |
| 116.5   | pyOpenSSL and pip in containers            |
| 116.6   | TLS — expected vs production               |
| 116.7   | Residual findings after remediation        |
| 116.8   | Notes for IT / security team               |
| 116.9   | Repo-owned fixes (this document's release) |
| 117     | Security                                   |
| 118     | Security                                   |
| 119     | API routes                                 |
| 120     | API routes                                 |
| 120.1   | Health & audit                             |
| 120.2   | Auth                                       |
| 120.3   | Rules & FDD                                |
| 120.4   | Rule Lab playground                        |
| 120.5   | BRICK / data model                         |
| 120.6   | BACnet                                     |
| 120.7   | Faults (check-engine)                      |
| 120.8   | Timeseries & sites                         |
| 120.9   | Building & host                            |
| 120.10  | Local agent (Ollama)                       |
| 120.11  | Modbus                                     |
| 120.12  | JSON API (HTTP/HTTPS)                      |
| 120.13  | PyPI package (separate)                    |
| 120.14  | Errors                                     |
| 121     | Python package                             |
| 122     | Python package (open-fdd)                  |
| 122.1   | Modules                                    |
| 122.2   | When to use package-only                   |
| 122.3   | Versioning                                 |
| 122.4   | Offline Arrow example                      |
| 123     | Configuration reference                    |
| 124     | Configuration reference                    |
| 124.1   | Bridge / auth                              |
| 124.2   | BACnet                                     |
| 124.3   | Docker edge                                |
| 125     | Glossary                                   |
| 126     | Glossary                                   |
| 127     | Workspace env files                        |
| 128     | Workspace env files                        |
| 128.1   | Ansible / edge extras                      |
| 128.2   | Related docs                               |
| 129     | Appendix                                   |
| 130     | Appendix                                   |
| 131     | GL36 lab site (Acme)                       |
| 132     | GL36 lab site (Acme)                       |
| 132.1   | BACnet poll scope                          |
| 132.2   | Example FDD rules (Arrow)                  |
| 132.2.1 | Duct static pressure high (AHU)            |
| 132.2.2 | SAT too cold vs setpoint                   |
| 132.2.3 | Site rule stubs in the repo                |
| 132.2.4 | Local OA-T vs web weather (JSON API)       |

- 132.3 Deploy / upgrade (Ansible, this lab only)
- 132.4 AI-assisted data modeling (lab workflow)
- 133 Examples & lab notes
- 134 Examples & lab notes
- 135 Arrow-native runtime
- 136 Arrow-native runtime
  - 136.1 Authoring contract
  - 136.2 Package layout

*Generated June 9, 2026*

---

# 1 Home

## 2 Open-FDD

Open-FDD is an open-source **building edge platform** for BACnet integration, feather historian storage, Arrow-native fault detection, and operator dashboards — designed for **trusted LAN / OT edge** deployment behind Caddy, not direct public internet exposure.

### 2.1 Who it is for

| Role                            | What you get                                                              |
|---------------------------------|---------------------------------------------------------------------------|
| <b>IT / controls engineer</b>   | Pull three GHCR images, log in, poll BACnet, view faults                  |
| <b>Commissioning integrator</b> | Discover points, bind BRICK model, pin FDD rules in Rule Lab              |
| <b>Developer / maintainer</b>   | Extend the Operator Bridge, publish images, run CI and LAN security scans |

### 2.2 Start here

| Path              | Go to                                                           |
|-------------------|-----------------------------------------------------------------|
| <b>Try it</b>     | <a href="#">Quick Start — Run with Docker</a>                   |
| <b>Operate it</b> | <a href="#">Updating the stack</a> · <a href="#">Operations</a> |
| <b>Develop it</b> | <a href="#">Developer Guide</a> · <a href="#">Architecture</a>  |

## 2.3 v3 edge stack

Three primary containers on a Linux edge host:

| Container                 | Role                                                    |
|---------------------------|---------------------------------------------------------|
| <b>openfdd-bridge</b>     | Operator Bridge — API, dashboard, feather <b>ingest</b> |
| <b>openfdd-commission</b> | BACnet discover/read/write + <b>poll loop</b>           |
| <b>openfdd-mcp-rag</b>    | Doc-search sidecar for agent tools                      |

Host **Caddy** on :80 or :443 is the normal LAN front door. BACnet UDP :47808 is bound by **commission** only — the bridge ingests samples.csv into the feather historian.

Details: [Containers](#) · [Deployment modes](#)

**LAN / OT edge only.** Do not expose the Operator Bridge to the public internet without TLS, strong auth, and site review. BACnet writes are disabled by default — see [BACnet write guard](#).

## 2.4 Distribution

| Channel                                                                                  | Use when                                           |
|------------------------------------------------------------------------------------------|----------------------------------------------------|
| <b>GHCR</b> <a href="https://ghcr.io/bbartling/openfdd-">ghcr.io/bbartling/openfdd-*</a> | Production or trial edge deploy                    |
| <b>This repository</b>                                                                   | Custom builds, commissioning, Rule Lab development |
| <b>PyPI</b> <a href="https://pypi.org/project/open-fdd/">open-fdd</a>                    | Arrow runtime lint/test without full UI            |

## 2.5 License

MIT — see repository LICENSE.

# 3 Run with Docker images

# 4 Run with Docker images

Deploy Open-FDD on a Linux edge host using **three published GHCR images**. No git clone required.

## 4.1 Images

| GHCR image                           | Service    | Role                                                          |
|--------------------------------------|------------|---------------------------------------------------------------|
| ghcr.io/bbartling/openfdd-bridge     | bridge     | Operator<br>API,<br>dashboard,<br>historian<br>ingest         |
| ghcr.io/bbartling/openfdd-commission | commission | BACnet<br>discover/<br>read/write<br><b>and poll<br/>loop</b> |
| ghcr.io/bbartling/openfdd-mcp-rag    | mcp-rag    | Doc-search<br>sidecar                                         |

Edge hosts pull **latest** from GHCR (three images: bridge, commission, mcp-rag). The retired **openfdd-bacnet-poll** image is no longer published.

## 4.2 1. Install and validate Docker

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
newgrp docker    # or log out/in

docker --version
docker compose version
docker ps
docker run --rm hello-world
```

Expect: active / enabled for docker, docker in groups, hello-world succeeds.

## 4.3 2. Bootstrap the site (one script)

Downloads docker-compose.yml, creates workspace/, generates auth.env.local, and sets BACNET\_BIND from the host LAN NIC (e.g. ens192 → 10.200.200.185/24:47808):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh
```

Bootstrap + pull + start in one go:

```
bash /tmp/openfdd_edge_bootstrap.sh --start
```

**Auth** — bootstrap writes ~/open-fdd/workspace/auth.env.local (gitignored). This file holds the API signing secret and one username/password per role. Use it for your first dashboard login:

```
cat ~/open-fdd/workspace/auth.env.local
```

| Variable          | Role                                                    |
|-------------------|---------------------------------------------------------|
| OFDD_AUTH_SECRET  | Signs session tokens (do not share)                     |
| OFDD_OPERATOR_*   | Read-only operator                                      |
| OFDD_INTEGRATOR_* | <b>Default dashboard login</b> — commissioning + BACnet |
| OFDD_AGENT_*      | Agent / automation                                      |

Example shape (passwords are **random on first bootstrap only** — existing file is kept on re-runs):

```
OFDD_AUTH_SECRET=<random>
OFDD_OPERATOR_USER=operator
OFDD_OPERATOR_PASSWORD=<random>
OFDD_INTEGRATOR_USER=integrator
OFDD_INTEGRATOR_PASSWORD=<random>
OFDD_AGENT_USER=agent
OFDD_AGENT_PASSWORD=<random>
```

Sign in as **integrator** with the password from that file → First login and health check.

| Action                                    | Touches auth.env.local?                         |
|-------------------------------------------|-------------------------------------------------|
| First bash ...<br>bootstrap.sh<br>--start | Creates file with random passwords (if missing) |

| Action                                    | Touches auth.env.local?                                    |
|-------------------------------------------|------------------------------------------------------------|
| bash ...<br>bootstrap.sh<br>--start again | <b>Keeps</b> your file — does not overwrite                |
| bash ...<br>bootstrap.sh<br>--restart     | <b>Keeps</b> your file — only restarts bridge to reload it |
| nano<br>auth.env.local<br>+ --restart     | Uses <b>your</b> edited passwords                          |
| --force-auth                              | <b>Regenerates</b> random passwords (opt-in only)          |

The **bridge** container reads auth.env.local at startup only. After you change passwords (manual edit or --force-auth), recreate bridge (env files are not reloaded by restart alone):

```
cd ~/open-fdd && docker compose up -d --force-recreate bridge
```

Regenerate passwords and reload (stack already running):

```
bash /tmp/openfdd_edge_bootstrap.sh --force-auth --restart --show-secrets
```

Manual edit then reload:

```
nano ~/open-fdd/workspace/auth.env.local
bash /tmp/openfdd_edge_bootstrap.sh --restart
```

Options:

| Flag              | Purpose                                                                       |
|-------------------|-------------------------------------------------------------------------------|
| --start           | docker compose pull && up -d after layout; curls http://127.0.0.1:8765/health |
| --image-tag TAG   | default latest                                                                |
| --repo-ref BRANCH | branch for compose.edge.yml if not on master yet                              |
| --force-auth      | regenerate auth.env.local (restarts bridge if stack is up)                    |
| --restart         | recreate bridge to reload auth.env.local; curls /health after                 |
| --show-secrets    | print passwords at end (lab only)                                             |

From a repo checkout: ./scripts/openfdd\_edge\_bootstrap.sh --start

The script prints **BACnet NIC**, **BACNET\_BIND**, and file paths — **validate** commission.env before polling OT devices:

```
nano ~/open-fdd/workspace/bacnet/commissioning/commission.env
```

If you used `--start`, the stack is already up. Next step → First login and health check.

### 4.3.1 Manual start (optional)

Use this only if you ran bootstrap **without** `--start` and want to bring the stack up yourself:

```
cd ~/open-fdd
docker compose pull
docker compose up -d
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

Expected: **bridge, commission, mcp-rag** — all up. Then → First login and health check.

## 4.4 Long-term operation

### 4.4.1 Survive power cycles

All services use restart: unless-stopped. After reboot:

```
cd ~/open-fdd && docker compose ps
curl -sf http://127.0.0.1:8765/health
```

### 4.4.2 Start / stop / restart

```
cd ~/open-fdd
docker compose up -d           # idempotent
docker compose stop
docker compose restart commission
docker compose logs -f --tail 100 commission
```

Never run `docker compose down -v` or delete workspace/ on a live site.

### 4.4.3 LAN access

Put **Caddy** (or `nginx`) on port 80 → 127.0.0.1:8765, or expose 8765 through the firewall.

## 4.5 Deployment choices

| Mode                        | Compose services                                      | When to use                                                                               |
|-----------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Standard OT / BACnet</b> | bridge + commission + mcp-rag                         | Real buildings — commission polls BACnet into samples.csv; bridge ingests to feather      |
| <b>Non-BACnet / demo</b>    | bridge + mcp-rag (omit commission)                    | JSON API, Modbus-only, or lab without field BACnet — see <a href="#">Driver framework</a> |
| <b>TLS on OT LAN</b>        | Same stack + <b>host Caddy</b><br>OFDD_CADDY_MODE=tls | Encrypt browser↔edge; trust self-signed CA on operator PCs — not for public internet      |

Do **not** run the retired openfdd-bacnet-poll image or legacy openfdd-bacnet-poll systemd unit alongside Docker **commission** — that double-polls BACnet.

## 4.6 Next steps

→ [First login and health check](#)

→ [Updating the stack](#) — ./scripts/openfdd\_site\_update.sh

# 5 First login and health check

# 6 First login and health check

Quick checks after docker compose up -d on the edge host. Services use restart: unless-stopped — after a reboot, run docker compose ps once Docker is up (see [Run with Docker](#)).

## 6.1 Public health

```
curl -s http://127.0.0.1:8765/health
```

Through Caddy on port 80:



```
curl -s http://127.0.0.1/health
```

Expect "ok": true and "auth\_required": true when auth is configured.

## 6.2 Login

1. Open the dashboard (<http://<host-lan-ip>/> or :8765).
2. Sign in with credentials from `workspace/auth.env.local`.

```
curl -s -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"<from-auth-env>"}
```

## 6.3 Smoke checklist

| Check                          | Pass                                                                                            |
|--------------------------------|-------------------------------------------------------------------------------------------------|
| <code>docker compose ps</code> | bridge, commission, mcp-rag<br><b>Up</b>                                                        |
| <code>GET /health</code>       | 200, ok: true                                                                                   |
| Login                          | Token returned                                                                                  |
| BACnet poll                    | <code>samples.csv</code> growing<br>( <code>tail -1 workspace/bacnet/polls/samples.csv</code> ) |
| Historian                      | Files under <code>workspace/data/feather_store/</code>                                          |

## 6.4 Stack detail (authenticated)

Integrator token required:

```
TOKEN=$(curl -sf -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"..."}' | jq -r .token)
curl -sf http://127.0.0.1:8765/health/stack -H "Authorization:
Bearer $TOKEN" | jq .
```

Look for `bacnet_poll` green/yellow in the services list.

## 6.5 Troubleshooting

| Symptom        | Fix                                             |
|----------------|-------------------------------------------------|
| 401 everywhere | Configure <code>workspace/auth.env.local</code> |

| Symptom               | Fix                                                               |
|-----------------------|-------------------------------------------------------------------|
| Empty BACnet discover | Fix BACNET_BIND in commission.env — <a href="#">network setup</a> |
| No poll rows          | Commission container running? points.csv has enabled rows?        |
| LAN browser timeout   | Open firewall port 80 or 8765                                     |

Advanced probes (model graph, Rule Lab, compose profiles):  
[Developer health check](#).

## 7 Quick Start

## 8 Quick Start

Run Open-FDD on a Linux edge host using **three GHCR Docker images**. No git clone on the host.

| Step | Page                                                                          |
|------|-------------------------------------------------------------------------------|
| 1    | <a href="#">Run with Docker images</a> — bootstrap script + docker compose up |
| 2    | <a href="#">First login and health check</a>                                  |
| 3    | <a href="#">Updating the stack</a> — backup + pull                            |

**One-liner bootstrap** (edge host):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
  https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh --start
```

**Prerequisites:** Linux, Docker enabled on boot, BACnet LAN if polling OT equipment.

| **BACnet writes are disabled by default.** See [Write safety](#).

**Stack:** bridge + commission (BACnet + poll) + mcp-rag. Details: [Containers](#).

# 9 Updating the stack

## 10 Updating the stack

Upgrade GHCR images on the edge host: **backup workspace/, pull new tags, recreate containers**. No git pull on the host.

### 10.1 Before you start

1. SSH to the edge host (`cd ~/open-fdd`).
2. Pick a tag from [GitHub Packages](#) — default is **latest**.
3. Short maintenance window (containers restart briefly; restart: unless-stopped brings them back after reboot).

### 10.2 Fetch helper scripts (no git clone)

If you bootstrapped with `openfdd_edge_bootstrap.sh`, scripts are already under `~/open-fdd/scripts/`. Otherwise:

```
mkdir -p ~/open-fdd/scripts
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_backup.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_backup.sh
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_update.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_update.sh
chmod +x ~/open-fdd/scripts/openfdd_site_*.sh
```

### 10.3 One-command upgrade (recommended)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Pulls **:latest** by default, verifies all three images exist on GHCR, recreates containers, and hits `/health`.

## 10.4 Step by step

### 10.4.1 1. Backup site state

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives workspace/ (feather, BACnet CSVs, model, auth) under ~/openfdd-backups/<timestamp>/.

Custom backup location:

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/
openfdd_site_backup.sh
```

### 10.4.2 2. Verify images (optional)

```
export NEW_TAG="${NEW_TAG:-latest}"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:$
{NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:$
{NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:$
{NEW_TAG} >/dev/null &&
echo "All images exist for ${NEW_TAG}"
```

### 10.4.3 3. Pull and recreate

```
cd ~/open-fdd
./scripts/openfdd_site_update.sh
```

**Manual equivalent** (same as the script):

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

### 10.4.4 4. Verify

```
docker compose ps
curl -sf http://127.0.0.1:8765/health | jq .
tail -1 workspace/bacnet/polls/samples.csv
ls -lah workspace/data/feather_store/
```

→ Health check

## 10.5 Dashboard UI updates

If the release changed the React UI, copy a new workspace/api/static/app/ build onto the host before or after the image pull. Image pull alone may not change the browser bundle hash.

## 10.6 Rollback

Restore docker-compose.yml from backup and pull again, or re-run bootstrap from a known-good master commit. GHCR keeps only the **three newest** versions per image; early sites should rely on **latest** plus workspace/ backups.

workspace/ is untouched by image-only upgrades.

## 10.7 Safe cleanup

```
docker image prune -f
```

Never run docker compose down -v, docker volume prune, or delete workspace/ on a live site.

## 10.8 Next steps

- [Run with Docker images](#) — bootstrap a new host
- [Health check](#)

# 11 Local development

# 12 Local development

## 12.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+
chars)
```

## 12.2 Start stack

```
./scripts/build_and_test.sh           # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

| URL                        | Service         |
|----------------------------|-----------------|
| http://127.0.0.1:8765      | Bridge (direct) |
| http://127.0.0.1:8765/docs | OpenAPI         |

## 12.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env – set BACNET_BIND to your OT interface
```

## 12.4 Production-like local stack (Caddy, LAN scans)

Same shape as an Acme edge deploy: **prod React UI, Caddy on :80**, bridge on loopback **8765**, auth enabled. Use for OWASP ZAP or nmap from another machine on the LAN.

```
./scripts/pentest_production_stack.sh start    # build prod UI +
Caddy + auth.pentest.local
./scripts/pentest_production_stack.sh status   # bridge,
commission, caddy, mcp-rag
./scripts/pentest_production_stack.sh verify   # LAN IP, /health,
ZAP target URL
./scripts/pentest_production_stack.sh stop     # when done
```

Credentials: workspace/auth.pentest.local (generated on first start). ZAP target is usually http://<lan-ip>/ (benserver example: http://192.168.204.18/). If LAN clients cannot reach :80, run `sudo ./scripts/open_lan_port.sh 80`.

See also [Health checks \(developer\)](#) for probe details and [ZAP baseline expectations](#) for accepted Medium/Low findings.

## 12.5 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve
# http://127.0.0.1:4000/open-fdd/
```

# 13 Build Docker images

# 14 Build Docker images

## 14.1 Local build

```
./scripts/docker_build.sh
# Optional tar for air-gap:
./scripts/docker_build.sh --save
```

Images: openfdd-bridge, openfdd-commission, openfdd-mcp-rag.

## 14.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker addons** — manual dispatch; publishes **:latest** only and prunes GHCR (retired openfdd-bacnet-poll removed; keep 3 versions per image).

Maintainer checklist:

1. Green CI on master
2. Run **Publish Docker addons** workflow
3. Verify ghcr.io/bbartling/openfdd-{bridge,commission,mcp-rag}:latest
4. Edge hosts: docker compose pull && docker compose up -d

Details live in .github/workflows/ and scripts/docker\_build.sh — not duplicated here.

# 15 Developer Guide

# 16 Developer Guide

Clone, build, test, and contribute to Open-FDD.

| Topic         | Page                                      |
|---------------|-------------------------------------------|
| Local setup   | <a href="#">Local development</a>         |
| Docker builds | <a href="#">Build Docker images</a>       |
| Health checks | <a href="#">Health checks (developer)</a> |

| Topic            | Page                                         |
|------------------|----------------------------------------------|
| Tests & CI       | <a href="#">Tests and CI</a>                 |
| Security testing | <a href="#">Release &amp; LAN scan cycle</a> |
| PRs              | <a href="#">Contributing</a>                 |

AI-assisted contributors: see repository `AGENTS.md` for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

# 17 Tests and CI

# 18 Tests and CI

## 18.1 Local tests

```
./scripts/build_and_test.sh # UI + workspace bridge
pytest
python3 -m pytest open_fdd/tests -q # PyPI package tests
cd workspace/dashboard && npm test # dashboard unit tests (if
configured)
```

## 18.2 CI (GitHub Actions)

| Workflow                  | Trigger         | Checks                                                      |
|---------------------------|-----------------|-------------------------------------------------------------|
| ci.yml                    | PR / push       | pytest, bridge security audit, dashboard build, Jekyll docs |
| docs-pages.yml            | push master     | GitHub Pages site                                           |
| publish-open-fdd.yml      | tag open-fdd-v* | PyPI wheel                                                  |
| publish-docker-addons.yml | manual          | GHCR images                                                 |

## 18.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```



Fix any template or link errors before opening a PR that touches docs/.

## 18.4 Security-related CI

ci.yml includes bridge security tests (tests/workspace\_bridge/test\_security.py). That complements — but does not replace — LAN ZAP + Nmap scans on each patch revision. See Security testing cycle.

# 19 Contributing

# 20 Contributing

## 20.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (auth.env.local, Ansible secrets/\*.env.local, commissioning CSVs with real site data).
- Follow existing code style in workspace/api/ and workspace/dashboard/.

## 20.2 Pull request checklist

☐

Tests pass locally (./scripts/build\_and\_test.sh or scoped pytest)

☐

Docs updated if user-facing behavior changed

☐

docs/ Jekyll build succeeds

☐

No credentials in diff

☐

Auth/Caddy/header changes: re-run pentest verify + LAN ZAP and update ZAP baseline if expectations changed

## 20.3 Repository layout (short)

| Path                 | Contents                                 |
|----------------------|------------------------------------------|
| workspace/api/       | Operator Bridge (FastAPI)                |
| workspace/dashboard/ | React SPA                                |
| open_fdd/            | PyPI package (arrow_runtime, playground) |
| docker/              | Compose files and image contexts         |
| infra/ansible/       | Edge deploy playbooks                    |
| docs/                | This site (Jekyll + Just the Docs)       |

## 20.4 Security issues

Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

## 20.5 Agent-assisted development

Cursor/Claude contributors: read AGENTS.md and skills/ for automation boundaries. Public docs intentionally avoid agent playbook detail.

# 21 Health checks (developer)

# 22 Health checks (developer)

Extended validation for engineers with a **full git checkout** — CI, compose overlays, and pre-release gates.

IT operators on edge hosts should use Quick Start — health check and Deployment validation instead.

## 22.1 Clone and local stack

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local #
loopback dev only
./scripts/docker_build.sh # or: ./
scripts/openfdd_stack.sh prod
./scripts/openfdd_stack.sh up
```

GHCR production pull (no local build):

```
OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/openfdd_stack.sh prod
```

## 22.2 Stack health script

```
./scripts/stack_health_check.sh
OPENFDD_BASE_URL=http://192.168.204.18:8765 ./scripts/
stack_health_check.sh
./scripts/stack_health_check.sh --require-ollama
```

Probes: containers up, /health, /api/model/health, sites, tree, SPARQL graph (with integrator auth when configured).

## 22.3 Compose overlays

| Overlay                  | Use                                                             |
|--------------------------|-----------------------------------------------------------------|
| docker/compose.dev.yml   | Local dev bind-mount                                            |
| docker/compose.bench.yml | Host-network<br>BACnet lab                                      |
| docker/compose.prod.yml  | GHCR images<br>instead of local build                           |
| docker/compose.edge.yml  | IT edge template<br>(copy to ~/open-fdd/<br>docker-compose.yml) |

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml ps
docker compose -f docker/compose.dev.yml config --images
```

## 22.4 BACnet poll pipeline

```
tail -1 workspace/bacnet/polls/samples.csv
cat workspace/data/bacnet_ingest_state.json
docker compose logs --since 10m commission | grep -i poll
docker compose logs --since 10m bridge | grep -i ingest
```

## 22.5 pytest (bridge security / BACnet)

```
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
```

# 22.6 Pentest production stack (LAN ZAP)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

Then from a **LAN workstation**, run the packaged scan:

- Windows: scripts/security/Run-OpenFddSecurityScan.ps1
- macOS/Linux: scripts/security/run\_openfdd\_security\_scan.sh

See [scripts/security/README.md](#), [Security — ZAP baseline](#), and the full [security testing cycle](#). Host-side notes: workspace/deploy/PENTEST.md.

# 22.7 Post-deploy insurance (inventory hosts)

For automated checks against named inventory hosts, see `infra/ansible/scripts/post_deploy_check.sh` (maintainer tooling — not required for IT docker-pull sites).

# 23 Security testing cycle

# 24 Security testing cycle

How Open-FDD validates each **patch revision** before it ships to the live Acme HVAC bench — local automation, AI-assisted PR review, LAN passive scans, and edge post-deploy checks.

Related security docs: [ZAP baseline](#), [TLS and certificates](#), [LAN hardening](#), [Linux host hardening](#), [Tenable remediation](#), [Authenticated scanning \(roadmap\)](#).

## 24.1 Who owns what

| Concern                                         | Owner                    | Where                                            |
|-------------------------------------------------|--------------------------|--------------------------------------------------|
| HTTP security headers (CSP, X-Frame, COOP, ...) | <b>Bridge</b> middleware | workspace/api/openfdd_bridge/security_headers.py |
| TLS termination + HSTS                          | <b>Caddy</b>             |                                                  |

| Concern                 | Owner                                    | Where                                                                                                                                                      |
|-------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OT LAN TLS certificates | <b>Site integrator / maintainer</b>      | workspace/deploy/caddy/, Ansible Caddyfile.j2<br>./scripts/setup_caddy_certs.sh<br>→ workspace/deploy/caddy/certs/;<br>Ansible /etc/openfdd/caddy/ on edge |
| Pentest credentials     | Generated per stack                      | workspace/auth.pentest.local (never commit)                                                                                                                |
| LAN ZAP + Nmap scans    | <b>Developer workstation</b> on test LAN | scripts/security/                                                                                                                                          |

See [TLS and certificate ownership](#) for cert lifecycle.

## 24.2 Per-revision workflow

Typical cycle on a **control machine** (local dev) through a **lab edge host** (live bench):

flowchart LR

```

A[Local dev + pytest] --> B[Pentest stack verify]
B --> C[LAN ZAP + Nmap]
C --> D[PR + GitHub CI]
D --> E[CodeRabbit AI review]
E --> F[Merge + tag open-fdd-vX.Y.Z]
F --> G[GHCR + PyPI publish]
G --> H[Acme upgrade + post_deploy_check --full]
H --> I[Bump next patch locally]

```

### 24.2.1 1. Local functional tests

```

./scripts/build_and_test.sh
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
infra/ansible/scripts/bench_5007_arrow_battery.sh    # when FDD/
Arrow paths change

```

## 24.2.2 2. Pentest production stack (on Open-FDD host)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

Confirms Caddy :80 fronts the bridge, auth is enabled, BACnet writes disabled, and security headers are singular (no Caddy duplicates). Details: [Health checks — pentest](#).

## 24.2.3 3. Host + exposure checks (on edge VM)

After IT Nessus scans or before Acme deploy sign-off:

```
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip <lan-ip>
```

Host remediation (dry-run first): `./scripts/security/remediate_ubuntu_host.sh`

## 24.2.4 4. LAN security scan (from laptop)

Use the packaged scripts — do not hand-roll one-off docker/nmap commands unless debugging:

| Platform      | Command                                                      |
|---------------|--------------------------------------------------------------|
| Windows       | <code>.\scripts\security\Run-OpenFddSecurityScan.ps1</code>  |
| macOS / Linux | <code>./scripts/security/run_openfdd_security_scan.sh</code> |

Setup (Docker Desktop, Nmap): [scripts/security/README.md](#).

Target URL is usually `http://<lan-ip>/` (Caddy), not `:8765`.

Review `openfdd-security-report/90-quick-findings-summary.txt` and compare against [ZAP baseline expectations](#).

## 24.2.5 5. Pull request + automated review

- Open PR against master
- **GitHub Actions** (`ci.yml`): pytest, bridge security audit, dashboard build, Jekyll docs
- **CodeRabbit** (or similar AI PR review): style, security footguns, missing tests/docs

Do not merge with failing CI or unresolved duplicate-header regressions.

## 24.2.6 6. Release

```
git tag open-fdd-v3.0.4 # example
git push origin open-fdd-v3.0.4
```

Tag triggers GHCR :latest and PyPI publish workflows.

## 24.2.7 7. Lab edge validation

After pulling new images on a live bench VM:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
    infra/ansible/scripts/post_deploy_check.sh --limit
<inventory_host> --full
```

Re-run LAN ZAP + Nmap against the edge LAN IP if Caddy or auth behavior changed. Site-specific example: [GL36 lab note](#).

## 24.2.8 8. Next cycle

Bump patch version locally (pyproject.toml, open\_fdd/\_\_init\_\_.py) on master for the next development round — do not tag until the next release is ready.

## 24.3 What each scan layer covers

| Layer                          | Tool                                  | Depth                                                 |
|--------------------------------|---------------------------------------|-------------------------------------------------------|
| Unit / integration             | pytest, test_security.py              | Header<br>middleware,<br>auth gates                   |
| Stack smoke                    | pentest_production_stack.sh<br>verify | Live headers,<br>health, auth file                    |
| Passive web                    | ZAP baseline                          | Public routes,<br>CSP, cookies,<br>CORS               |
| Port exposure                  | Nmap scoped                           | One host,<br>known service<br>ports                   |
| Authenticated<br>API/dashboard | ZAP full + login                      | <b>Roadmap</b> —<br><u>Authenticated<br/>scanning</u> |

Baseline ZAP does **not** exercise integrator login or protected /api/\* routes deeply.

## 24.4 Open-source contributors

The scripts/security/ folder is part of the public repo so any fork can run the same LAN smoke against their bench. PRs that change Caddy, auth, or security\_headers.py should update [ZAP baseline](#) and re-run the scan scripts.

## 25 Expression cookbook (Arrow-native)

## 26 Expression cookbook (Arrow-native)

Reference for **Open-FDD 3.x Rule Lab**: every rule is a Python module with `apply_faults_arrow(table, cfg, context)` using `pyarrow.compute` — **no pandas, no YAML expression files, no NumPy DataFrames on the IoT edge**.

This page **replaces the legacy pandas type: expression YAML cookbook**. Each legacy recipe maps to a **fixed fault code** and an Arrow pattern below.

| Topic                 | Page                                               |
|-----------------------|----------------------------------------------------|
| Quick templates       | <a href="#">Arrow recipes</a>                      |
| Shared imports        | <a href="#">Python recipes</a>                     |
| Console window stats  | <a href="#">Lookback window</a>                    |
| Fault code catalog    | <a href="#">Fault codes</a>                        |
| Programmatic defaults | <code>open_fdd.arrow_runtime.sensor_catalog</code> |

### 26.1 Legacy → modern translation

| Legacy (pandas / YAML)                             | Arrow-native (3.x)                                              |
|----------------------------------------------------|-----------------------------------------------------------------|
| <code>type: expression + expression: string</code> | <code>apply_faults_arrow()</code> in <code>rules_py/*.py</code> |
| <code>RuleRunner.run(df, column_map=...)</code>    | Rule Lab batch on <b>feather PyArrow table</b>                  |
| <code>np.maximum(a, b)</code>                      | <code>pc.max(a, b)</code>                                       |
| <code>series.rolling(n).min()</code>               |                                                                 |



| Legacy (pandas / YAML)  | Arrow-native (3.x)                                                   |
|-------------------------|----------------------------------------------------------------------|
|                         | arrow_rolling_min(vals, n)<br>from<br>open_fdd.arrow_runtime.windows |
| series.diff().abs()     | arrow_abs_diff(vals, 1)                                              |
| normalize_cmd(x)        | pc.if_else(pc.greater(x, 1),<br>pc.divide(x, 100), x)                |
| params.max_temp in YAML | Module constant or<br>cfg["bounds_high"]                             |
| flag: rule_a_flag       | Rule metadata <b>fault_code</b> →<br>AHU-A ... VAV-C                 |
| Numeric codes VAV-03    | Letter codes <b>VAV-C</b> (see<br>LEGACY_CODE_MAP in bridge)         |

**Edge constraint:** Docker / BACnet poll path never imports pandas. Central portfolio Dash may use pandas for CSV analytics only.

## 26.2 How Arrow rules are structured

1. **Historian columns** — Feather column names from BACnet poll (oa-t, sa-t, stat\_zn-t, ...) or Brick labels via model export.
2. **Module constants** — Thresholds at top of file (bench style) or read from cfg for site tuning.
3. **apply\_faults\_arrow** — Returns a **boolean PyArrow array** (True = fault sample).
4. **fault\_code** — Set in Rule Lab metadata; must exist in GET /api/faults/catalog.

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask
```

```
FAULT_CODE = "VAV-C" # metadata in Rule Lab UI
```

```
def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "zone_temp", cfg)
```

### 26.2.1 Command scaling (0-1 vs 0-100 %)

BACnet often exposes **0-100** for damper/VFD commands. Scale explicitly:

```
def _norm_cmd(col):
    return pc.if_else(pc.greater(col, 1.0), pc.divide(col,
100.0), col)
```

## 26.2.2 Occupied hours (schedule gating)

Use `open_fdd.arrow_runtime.cookbook._unoccupied_mask` or compare local hour from timestamp column. Example: **fan on when unoccupied** → **BLD-C** / `schedule_compare`.

---

## 26.3 Sensor validation (bounds, flatline, rate of change)

Use `open_fdd.arrow_runtime.sensor_catalog.SENSOR_PROFILES` for defaults, or copy constants into `rules_py`. Tune per site in Rule Lab `cfg` or building-agent tuning brief.

### 26.3.1 Bounds (out of range) — `oob_rolling`

| Sensor kind                      | Min  | Max  | Flatline tol | Max $\Delta$ / hour | Max $\Delta$ / 15 min | Typical fault code            |
|----------------------------------|------|------|--------------|---------------------|-----------------------|-------------------------------|
| Zone temp (°F)                   | 55   | 90   | 0.10         | 4.0                 | 2.0                   | <b>VAV-C</b>                  |
| Supply air temp                  | 50   | 110  | 0.15         | 8.0                 | 3.0                   | <b>AHU-C, RTU-C</b>           |
| Return air temp                  | 55   | 95   | 0.10         | 3.0                 | 1.5                   | <b>AHU-D</b><br>(also mixing) |
| Mixed air temp                   | 40   | 110  | 0.15         | 6.0                 | 2.5                   | <b>AHU-D</b>                  |
| Outdoor air temp                 | −40  | 130  | 0.10         | 12.0                | 6.0                   | <b>BLD-B</b>                  |
| Duct static (inH <sub>2</sub> O) | −0.5 | 3.0  | 0.02         | 0.5                 | 0.25                  | <b>AHU-A</b>                  |
| Relative humidity (%)            | 0    | 100  | 1.0          | 15.0                | 8.0                   | <b>DC-C</b>                   |
| Chilled water (°F)               | 40   | 90   | 0.10         | 4.0                 | 2.0                   | <b>CH-D</b>                   |
| Hot water (°F)                   | 70   | 200  | 0.15         | 6.0                 | 3.0                   | <b>CH-D</b>                   |
| Condenser water (°F)             | 50   | 110  | 0.15         | 5.0                 | 2.5                   | <b>CH-A</b>                   |
| CO <sub>2</sub> (ppm, occupied)  | 400  | 1000 | 5.0          | 200                 | 80                    | <b>VAV-B</b><br>(ventilation) |

| Sensor kind        | Min | Max | Flatline tol | Max $\Delta$ / hour | Max $\Delta$ / 15 min | Typical fault code |
|--------------------|-----|-----|--------------|---------------------|-----------------------|--------------------|
| Discharge air temp | 45  | 120 | 0.15         | 10.0                | 4.0                   | VAV-E, HP-D        |

**Return air** uses a **narrow band** vs zone/OAT because RAT should track building return conditions, not outdoor extremes. When OAT/RAT/MAT are all present, prefer **mixing envelope** (below) over RAT bounds alone.

**CO<sub>2</sub> upper bound** is an occupied ventilation target; unoccupied rules may use 400–2000 ppm per policy.

### 26.3.2 Flatline (stuck sensor) — flatline\_1h

Default **12 samples  $\approx$  1 hour** at 5-minute poll. True when rolling max – min  $\leq$  tolerance.

```
from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_flatline_mask(table, "outdoor_air_temp", cfg)
# BLD-B
```

Bench references: workspace/data/rules\_py/bench\_oa-t\_flatline\_1h.py, bench\_stat\_zn-t\_flatline\_1h.py.

### 26.3.3 Rate of change (spike) — rate\_of\_change

Flags physically impossible step changes (comms glitch, substituted value).

```
from open_fdd.arrow_runtime.cookbook import rate_of_change_mask
from open_fdd.arrow_runtime.sensor_catalog import cfg_from_profile

def apply_faults_arrow(table, cfg, context=None):
    merged = cfg_from_profile("outdoor_air_temp", cfg)
    merged["samples_per_hour"] = 12 # 5-min poll
    return rate_of_change_mask(table, merged, col="oa-t")
```

Legacy **weather\_temp\_spike** (spike\_limit: 16 °F per step) → **BLD-B** with max\_per\_15min: 6 or stricter per-step limit.

### 26.3.4 Mixing envelope (MAT vs OAT/RAT) — mixing\_envelope

Legacy GL36 **Rule B / C** and **AHU-D**. MAT should sit between OAT and RAT ( $\pm$  tolerance) while fan runs.

```
from open_fdd.arrow_runtime.cookbook import mixing_envelope_mask

def apply_faults_arrow(table, cfg, context=None):
    return mixing_envelope_mask(
        table,
        {**cfg, "mixing_tol": 1.15},
        mat_col="ma-t",
        oat_col="oa-t",
        rat_col="ra-t",
        fan_col="supply-fan-speed-command",
    )
```

---

## 26.4 GL36-inspired AHU rules (legacy expression → fault code)

ASHRAE Guideline 36-style rules from the old YAML cookbook, translated to Arrow. Assign **fault\_code** per row in Rule Lab.

| Legacy rule                          | Summary                          | Suggested code             | Pattern         |
|--------------------------------------|----------------------------------|----------------------------|-----------------|
| Rule A — duct static low @ full fan  | SP below SP setpoint at high VFD | <b>AHU-A</b>               | custom_arrow    |
| Rule B — blend below band            | MAT below OAT/RAT envelope       | <b>AHU-D</b>               | mixing_envelope |
| Rule C — blend above band            | MAT above OAT/RAT envelope       | <b>AHU-D</b>               | mixing_envelope |
| Rule D — discharge cold when heating | SAT low vs MAT, heat valve open  | <b>AHU-B</b>               | custom_arrow    |
| Rule E — SAT low, full heating       | SAT below SP, valve > 90%        | <b>AHU-C / performance</b> | custom_arrow    |
| Rule F — SAT/ MAT                    | Econ mode,                       | <b>AHU-E</b>               | custom_arrow    |

| Legacy rule                                     | Summary                                       | Suggested code | Pattern         |
|-------------------------------------------------|-----------------------------------------------|----------------|-----------------|
| mismatch<br>econ                                | SAT $\neq$<br>MAT                             |                |                 |
| Rule G —<br>ambient warm<br>free cool           | OAT ><br>SAT SP,<br>econ<br>open, cool<br>off | <b>AHU-E</b>   | custom_arrow    |
| Rule H —<br>OAT/MAT<br>mismatch<br>econ+mech    | Mech +<br>econ, MAT<br>$\neq$ OAT             | <b>AHU-E</b>   | mixing_envelope |
| Rule I — OAT/<br>MAT<br>mismatch<br>econ-only   | Econ only,<br>MAT $\neq$<br>OAT               | <b>AHU-E</b>   | mixing_envelope |
| Rule J —<br>discharge<br>above blend<br>cooling | SAT ><br>MAT in<br>cooling                    | <b>AHU-B</b>   | custom_arrow    |
| Rule K —<br>discharge<br>above SP full<br>cool  | SAT > SP,<br>full<br>cooling                  | <b>AHU-C</b>   | custom_arrow    |
| Rule L —<br>cooling coil $\Delta T$<br>when off | CHW drop<br>when<br>valves<br>closed          | <b>CH-C</b>    | custom_arrow    |
| Rule M —<br>heating coil<br>$\Delta T$ when off | HW rise<br>when<br>valves<br>closed           | <b>AHU-B</b>   | custom_arrow    |

### 26.4.1 Rule A — duct static low at full speed (Arrow sketch)

```

import pyarrow.compute as pc

SP = "duct-static-pressure"
SP_SP = "duct-static-pressure-sp"
FAN = "supply-fan-speed-command"
SP_MARGIN = 0.12
DRV_HI = 0.93

def apply_faults_arrow(table, cfg, context=None):
    sp = pc.cast(table[SP], "float64")
    sp_sp = pc.cast(table[SP_SP], "float64")

```

```

        fan = pc.if_else(pc.greater(table[FAN], 1),
pc.divide(table[FAN], 100), table[FAN])
        low_sp = pc.less(sp, pc.subtract(sp_sp, SP_MARGIN))
        fan_hi = pc.greater_equal(fan, DRV_HI - 0.06)
    return pc.and_(low_sp, fan_hi)

```

## 26.5 Starter pack (VAV / AHU baseline)

Maps to legacy YAML starter filenames; use as first deploy bundle.

| Legacy YAML starter                        | Arrow approach                                       | Fault code |
|--------------------------------------------|------------------------------------------------------|------------|
| 01_vav_zone_temp_bounds_occupied           | sensor_bounds_mask(..., "zone_temp") + occupied mask | VAV-C      |
| 02_vav_zone_temp_flatline_occupied         | sensor_flatline_mask(..., "zone_temp") + occupied    | VAV-C      |
| 03_vav_damper_command_extreme_flatline     | flatline_1h_mask on damper cmd column                | VAV-D      |
| 04_ahu_runtime_outside_schedule            | after_hours_fan_satisfied_mask / run-hours script    | BL-C       |
| 05_ahu_duct_static_pressure_not_maintained | Rule A sketch above                                  | AH-A       |
| 06_ahu_internal_temp_sensor_bounds         | sensor_bounds_mask on SAT/MAT                        | AH-C       |
| 07_ahu_internal_temp_sensor_flatline       | sensor_flatline_mask on SAT                          | AH-C       |

### 26.5.1 Economizer starters

| Legacy name                                  | Fault code | Key logic                                                            |
|----------------------------------------------|------------|----------------------------------------------------------------------|
| ahu_econ_100oa_temp_tracking_fault           | AHU-E      | OA damper $\geq$ 95%, $ MAT - OAT $ and $ SAT - MAT  > tol$          |
| ahu_mech_cooling_when_free_cooling_available | AHU-E      | $OAT \leq SAT$<br>SP – margin,<br>low OA damper,<br>cooling valve on |

| Legacy name                               | Fault code | Key logic                                                 |
|-------------------------------------------|------------|-----------------------------------------------------------|
| ahu_oa_damper_excess_open_extreme_ambient | AHU-E      | OA $\geq$ 50%<br>when OAT<br>> 70 °F or<br>OAT < 40<br>°F |

## 26.6 VAV, central plant, heat pump

| Legacy recipe                  | Fault code | Notes                                      |
|--------------------------------|------------|--------------------------------------------|
| zone_reheat_warm_ambient       | VAV-A      | Reheat<br>open when<br>OAT ><br>cutoff     |
| zone_damper_valve_full_open    | VAV-D      | Damper ><br>97.5% for<br>rolling<br>window |
| dp_below_sp_pump_max           | CH-E       | Plant DP<br>low at max<br>pump<br>speed    |
| flow_high_pump_max             | CH-B       | High flow<br>at max<br>pump                |
| plant_supply_temp_deadband     | CH-D       | CHW SP $\pm$<br>band while<br>pump on      |
| chiller_excessive_runtime      | CH-F       | Rolling<br>sum of<br>chiller on<br>samples |
| hp_discharge_cold_when_heating | HP-D       | SAT low,<br>zone cold,<br>fan on           |

Rolling persistence in pandas `.rolling(n).min()` →  
`arrow_rolling_min + pc.and_` with current sample.

## 26.7 Opportunistic / ventilation

| Legacy recipe                 | Fault code                    |
|-------------------------------|-------------------------------|
| econ_active_warm_ambient      | <b>AHU-E</b>                  |
| mech_cool_when_econ_available | <b>AHU-E</b>                  |
| low_oa_fraction_estimated     | <b>VAV-B / BLD-A</b>          |
| preheat_excess_temp           | <b>AHU-B</b>                  |
| weather_temp_spike            | <b>BLD-B</b> (rate_of_change) |
| weather_gust_lt_wind          | <b>BLD-B</b> (custom compare) |

**OA fraction estimate** (no airflow meter):

```
# Guard: |RAT - OAT| > gap before dividing
oa_frac = (mat - rat) / (oat - rat) * 100
# Flag when oa_frac < oa_min_pct and fan on
```

Use pc.and\_ with gap check; skip divide where denominator near zero.

## 26.8 Data quality vs equipment faults

| Symptom                 | Pattern         | Code                       | Do not confuse with           |
|-------------------------|-----------------|----------------------------|-------------------------------|
| Flatline 1h             | flatline_1h     | <b>VAV-C, AHU-C, BLD-B</b> | Equipment off / damper closed |
| OOB sample              | oob_rolling     | sensor-specific            | True process excursion        |
| Spike between polls     | rate_of_change  | <b>BLD-B</b>               | Fast legitimate weather front |
| No new samples          | stale_points    | <b>BLD-D</b>               | Equipment fault               |
| MAT $\notin$ [OAT, RAT] | mixing_envelope | <b>AHU-D</b>               | Stratification during startup |

Always gate sensor faults with **fan on, valve open, or occupied** where appropriate — see [Sensor & data quality](#).



## 26.9 Metric (°C) equivalents

Set `cfg["temp_unit"] = "metric"` in Rule Lab or scale constants:

| Imperial                | Metric (approx) |
|-------------------------|-----------------|
| 55 °F                   | 12.8 °C         |
| 90 °F                   | 32.2 °C         |
| 50 °F                   | 10.0 °C         |
| 110 °F                  | 43.3 °C         |
| 2.0 °F tol              | 1.1 °C          |
| 0.15 inH <sub>2</sub> O | ~37 Pa          |

`open_fdd.playground.temp_units` documents UI field keys for °F/°C toggle.

---

## 26.10 Binding rules to the fault catalog

1. Pick a **letter code** from [Fault codes](#) — never invent suffixes.
2. In Rule Lab, set **fault\_code** on the rule row.
3. Batch FDD aggregates into `GET /api/faults/status` and portfolio rollup.
4. Legacy numeric codes (AHU-03) migrate via `LEGACY_CODE_MAP` in the bridge.

```
curl -s http://127.0.0.1:8765/api/faults/catalog | jq  
' .families[].codes[] | select(.code=="VAV-C") '
```

---

## 26.11 Pandas YAML → Arrow checklist (commissioning)

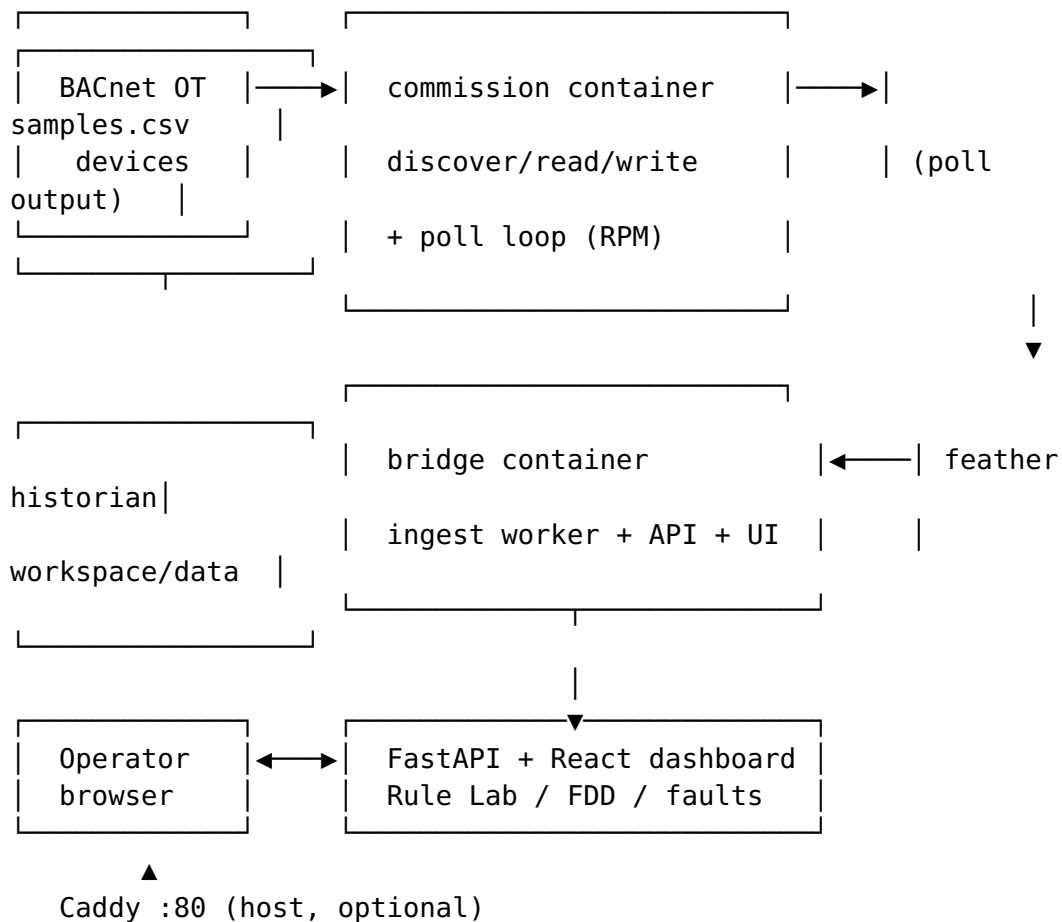
- ☐ Replace each type: expression YAML with a `rules_py` module
- ☐ Map inputs Brick labels → feather column names in model/poll CSV
- ☐ Set `fault_code` per rule (letter suffix)
- ☐ Run Rule Lab kit on 7-day feather window; confirm flag rate sane
- ☐ Apply `sensor_catalog` defaults; tune bounds for site climate
- ☐

Enable building-agent check-in; verify faults in portfolio rollout

**Next:** [Arrow recipes](#) · [Fault codes](#) · [Rule Lab](#)

## 27 System overview

## 28 System overview



### 28.1 Docker edge stack (production)

Three GHCR images — see [Containers](#):

| Container         | BACnet OT                  | Operator LAN                         |
|-------------------|----------------------------|--------------------------------------|
| <b>bridge</b>     | No (HTTP only)             | Yes, via Caddy :80 or loopback :8765 |
| <b>commission</b> | Yes (host network, :47808) | No (internal :8767)                  |

| Container | BACnet OT | Operator LAN        |
|-----------|-----------|---------------------|
| mcp-rag   | No        | No (internal :8090) |

BACnet **polling** is not a separate container — it runs inside **commission**. The bridge only **ingests** poll CSV into feather.

## 28.2 Components

| Layer                    | Responsibility                                                                           |
|--------------------------|------------------------------------------------------------------------------------------|
| <b>Operator Bridge</b>   | Auth, REST/WebSocket API, compiled dashboard, Rule Lab, historian ingest                 |
| <b>BACnet commission</b> | Who-Is, read/write (gated), <b>scheduled poll loop</b> → <code>samples.csv</code>        |
| <b>Historian</b>         | Feather files per point; local-first retention                                           |
| <b>FDD runner</b>        | <code>workspace/data/rules_py/*.py</code> on timer or <code>POST /api/rules/batch</code> |
| <b>Model</b>             | Brick TTL + <code>model.json</code> bindings ( <code>fdd_input</code> , equipment tree)  |
| <b>MCP RAG</b>           | Optional doc retrieval for agent tools                                                   |
| <b>Caddy</b>             | Host reverse proxy <code>:80</code> → bridge (typical edge)                              |
| <b>Ollama</b>            | Optional host LLM for building insight (not required for FDD)                            |

## 28.3 Deployment

IT operators: [Quick Start — Docker](#). Modes and Caddy: [Deployment modes](#).

## 28.4 Optional PyPI-only path

`pip install open-fdd` ships Arrow runtime + playground lint helpers **without** the Operator Bridge UI. Use for offline rule tests or CI. See [Appendix — Python package](#).

## 29 Arrow recipes

## 30 Arrow recipes (default)

Open-FDD 3.0 Rule Lab rules use `apply_faults_arrow(table, cfg, context)`, `pyarrow.compute`, and **module constants** (no `config.json`).

For the full legacy pandas/YAML → Arrow translation, sensor bound tables, and GL36 fault-code mapping, see **Expression cookbook (Arrow-native)**.

### 30.1 Simple threshold

```
import pyarrow.compute as pc

VALUE_COLUMN = "zone_temp"
MAX_ZONE_TEMP = 78.0

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table[VALUE_COLUMN], MAX_ZONE_TEMP)
```

### 30.2 Supply air temp high

```
import pyarrow.compute as pc

VALUE_COLUMN = "sa-t"
HIGH_F = 75.0

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_F)
```

### 30.3 Fan commanded but no airflow

```
import pyarrow.compute as pc

FAN_CMD = "fan-cmd"
AIRFLOW = "supply-cfm"
FAN_ON = 0.5
MIN_CFM = 500.0
```

```
def apply_faults_arrow(table, cfg, context=None):
    fan_on = pc.greater(table[FAN_CMD], FAN_ON)
    low_airflow = pc.less(table[AIRFLOW], MIN_CFM)
    return pc.and_(fan_on, low_airflow)
```

## 30.4 Flatline (1 h rolling window)

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.windows import arrow_rolling_max,
arrow_rolling_min

VALUE_COLUMN = "oa-t"
WINDOW_SAMPLES = 12
FLATLINE_TOLERANCE = 0.1

def apply_faults_arrow(table, cfg, context=None):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    spread = pc.subtract(arrow_rolling_max(vals, WINDOW_SAMPLES),
arrow_rolling_min(vals, WINDOW_SAMPLES))
    return pc.less_equal(pc.abs(spread), FLATLINE_TOLERANCE)
```

## 30.5 Sensor out of range (catalog defaults)

```
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "outdoor_air_temp", cfg) #
fault_code: BLD-B
```

## 30.6 Sensor flatline + spike (catalog)

```
from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask,
rate_of_change_mask
from open_fdd.arrow_runtime.sensor_catalog import cfg_from_profile

def apply_faults_arrow(table, cfg, context=None):
    merged = cfg_from_profile("zone_temp", cfg)
    merged["samples_per_hour"] = 12
    flat = sensor_flatline_mask(table, "zone_temp", cfg)
    spike = rate_of_change_mask(table, merged, col="stat_zn-t")
    return pc.or_(flat, spike) # import pyarrow.compute as pc
```

Bounds table: [Expression cookbook — sensor validation](#).

More templates ship in `open_fdd.playground.arrow_templates` and via `GET /api/playground/arrow-templates`.

Bench examples: `workspace/data/rules_py/bench_*.py`.

Dev kit zip layout: [Rule Lab — dev kit zip](#).

# 31 Deployment modes

# 32 Deployment modes

How Open-FDD is typically run: local development, lab LAN trials, and production edge on a trusted OT/IT network. Open-FDD is designed for **building LAN or OT edge** deployment — not direct exposure to the public internet.

## 32.1 Mode summary

| Mode            | Stack                              | BACnet                       | Front door                                 | Auth                                        |
|-----------------|------------------------------------|------------------------------|--------------------------------------------|---------------------------------------------|
| Local dev       | Compose on control machine         | Optional bench simulator     | 127.0.0.1:8765                             | OFDD_AUTH_DISABLED=1 optional on loopback   |
| Lab LAN         | Three GHCR containers              | Real or simulator VLAN       | Caddy :80 or direct :8765                  | Required unless explicit insecure lab flags |
| Edge production | Three GHCR containers + host Caddy | OT NIC via <b>commission</b> | <b>Caddy</b> :80 or :443 → loopback bridge | <b>Required</b> — workspace/auth.env.local  |

## 32.2 v3 container stack (all edge modes)

| Image              | Role                                                            |
|--------------------|-----------------------------------------------------------------|
| openfdd-bridge     | Operator Bridge API, dashboard, <b>feather historian ingest</b> |
| openfdd-commission | BACnet discover/read/write + <b>poll loop</b>                   |
| openfdd-mcp-rag    |                                                                 |

| Image | Role                               |
|-------|------------------------------------|
|       | Doc-search sidecar for agent tools |

**openfdd-bacnet-poll is retired.** BACnet polling runs inside **commission** only. Do not run the legacy poll container or openfdd-bacnet-poll systemd unit alongside Docker commission — that double-polls the OT network.

See [Containers](#) for ports, networks, and persistence.

## 32.3 Caddy HTTP (typical edge)

- Caddy listens on building LAN :80
- Reverse-proxies to bridge at 127.0.0.1:8765
- Bridge stays loopback-only; operators bookmark `http://<edge-ip>/`

Bootstrap and compose: [Quick Start — Docker](#).

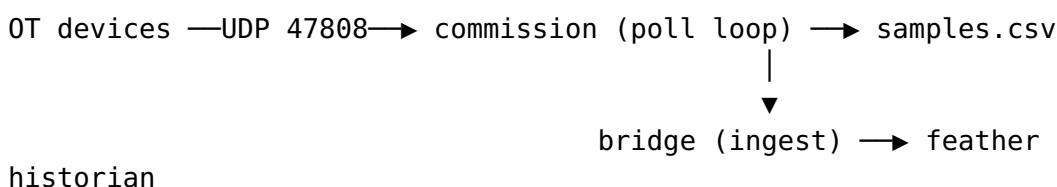
## 32.4 Caddy TLS (OT LAN)

For HTTPS on the edge LAN (self-signed or site CA):

1. Generate certs: `./scripts/setup_caddy_certs.sh` (control machine) or site PKI
2. Start with `OFDD_CADDY_MODE=tls`
3. Trust the CA on operator PCs

Details: [TLS and certificates](#). Security headers are set by the **Operator Bridge**; Caddy adds **HSTS** only in TLS mode.

## 32.5 BACnet data path



The bridge thread named `bacnet_poll_worker` is **ingest only** — it does not bind BACnet.

## 32.6 What not to do on production edges

- Port-forward :8765 to the public internet without TLS and strong auth
- Run `docker compose down -v` or delete workspace/ on a live site
- Deploy the retired `openfdd-bacnet-poll` image alongside commission

## 32.7 Related

| Topic            | Page                                     |
|------------------|------------------------------------------|
| First deploy     | <a href="#">Quick Start — Docker</a>     |
| Upgrade          | <a href="#">Updating the stack</a>       |
| LAN security     | <a href="#">LAN hardening</a>            |
| Lab example site | <a href="#">Examples — GL36 lab note</a> |

## 33 Python recipes (Arrow)

## 34 Python recipes (Arrow)

All Rule Lab Python rules use `apply_faults_arrow(table, cfg, context)` on PyArrow tables with `pyarrow.compute` and **module constants**.

See **Arrow recipes** for the canonical cookbook (flatline, spread, OOB, schedule faults).

Shared helpers live in `open_fdd.arrow_runtime.cookbook` and `open_fdd.arrow_runtime.windows`. For console validation, copy `_kit_lookback_stats` from [Lookback window helper](#).

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max
import pyarrow.compute as pc

VALUE_COLUMN = "oa-t"
WINDOW_SAMPLES = 12
FLATLINE_TOLERANCE = 0.1

def apply_faults_arrow(table, cfg, context=None):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    spread = pc.subtract(arrow_rolling_max(vals, WINDOW_SAMPLES),
```



```
arrow_rolling_min(vals, WINDOW_SAMPLES))
    return pc.less_equal(pc.abs(spread), FLATLINE_TOLERANCE)
```

# 35 Containers

# 36 Containers

Open-FDD v3 edge sites run **three** GHCR containers plus optional **host** services (Caddy, Ollama). There is no separate BACnet poll container — **field BACnet polling lives in commission**.

**Quick answer:** Who binds BACnet UDP 47808? **openfdd-commission**. Bridge only ingests samples.csv into the feather historian.

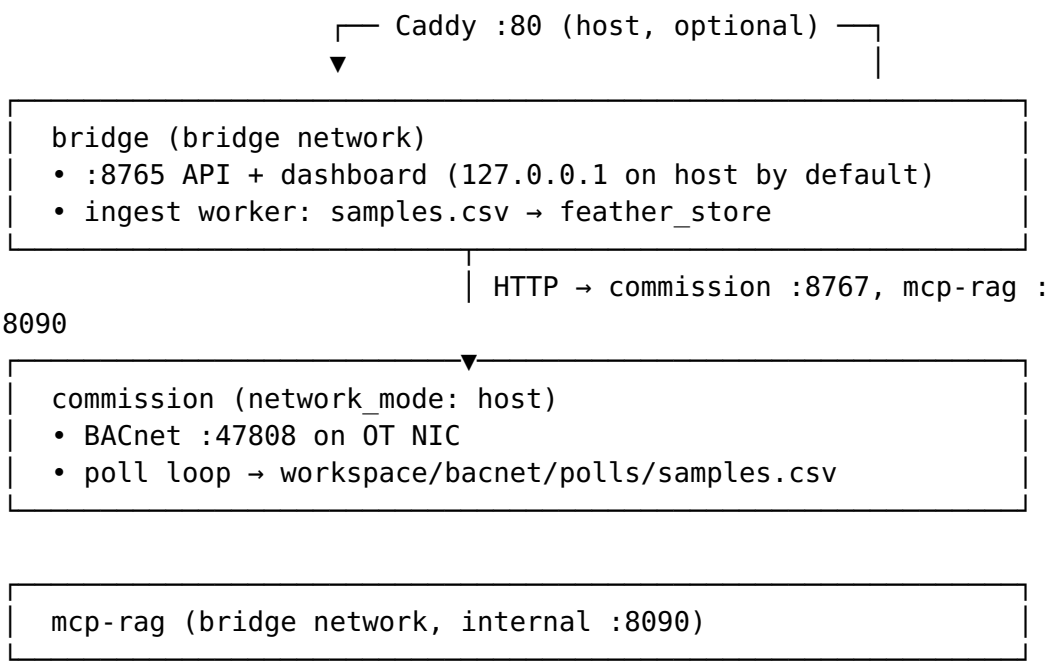
## 36.1 GHCR images

| Image                                | Compose service | Role                                                 |
|--------------------------------------|-----------------|------------------------------------------------------|
| ghcr.io/bbartling/openfdd-bridge     | bridge          | Operator API, React dashboard, feather <b>ingest</b> |
| ghcr.io/bbartling/openfdd-commission | commission      | BACnet discover / read / write + <b>poll loop</b>    |
| ghcr.io/bbartling/openfdd-mcp-rag    | mcp-rag         | Doc-search sidecar for agent tools                   |

Tag: **latest** on [GitHub Packages](#). Retired: openfdd-bacnet-poll (poll runs inside **commission**). GHCR retains only the **three newest** versions per image.

Template: docker/compose.edge.yml → copy to ~/open-fdd/docker-compose.yml.

# 36.2 Stack diagram



# 36.3 Responsibilities

| Concern                             | Container                                                                            |
|-------------------------------------|--------------------------------------------------------------------------------------|
| BACnet Who-Is / read / write        | <b>commission</b>                                                                    |
| BACnet scheduled RPM reads          | <b>commission</b> (bacnet_poll_loop)                                                 |
| CSV -> feather historian            | <b>bridge</b> (feather ingest worker — bacnet_poll_worker module name is historical) |
| Operator login, Rule Lab, faults UI | <b>bridge</b>                                                                        |
| Agent doc search                    | <b>mcp-rag</b>                                                                       |

The bridge thread named bacnet\_poll\_worker is **ingest only** — it does not bind BACnet.

# 36.4 Network and ports (typical edge)

| Endpoint     | Reachable from     | Notes                   |
|--------------|--------------------|-------------------------|
| Caddy :80    | Building LAN       | Reverse proxy to bridge |
| Bridge :8765 | Loopback (default) |                         |

| Endpoint           | Reachable from           | Notes                                                |
|--------------------|--------------------------|------------------------------------------------------|
| Commission : 8767  | Bridge / loopback        | Bind 127.0.0.1 in compose<br>HTTP API for BACnet ops |
| Commission : 47808 | OT BACnet LAN            | UDP;<br>requires network_mode: host                  |
| MCP :8090          | Bridge container network | Not exposed to LAN by default                        |

## 36.5 Long-lived deployment

All compose services set restart: unless-stopped. After host reboot:

```
sudo systemctl enable docker
cd ~/open-fdd && docker compose ps
```

If containers are down: `docker compose up -d` (idempotent).

See [Run with Docker images](#).

## 36.6 Persistent data

State lives on the **host** under `workspace/` (bind mount). Containers are replaceable; **backup workspace/** before image upgrades.

| Path                                            | Contents                                              |
|-------------------------------------------------|-------------------------------------------------------|
| <code>workspace/data/feather_store/</code>      | Historian                                             |
| <code>workspace/data/*.json</code>              | Model, rules, FDD results                             |
| <code>workspace/bacnet/commissioning/</code>    | <code>commission.env</code> , <code>points.csv</code> |
| <code>workspace/bacnet/polls/samples.csv</code> | Poll output                                           |
| <code>workspace/auth.env.local</code>           | Login secrets                                         |
| <code>workspace/api/static/app/</code>          | Dashboard bundle (if updated separately)              |

## 36.7 Optional host services

| Service        | Role                                                        |
|----------------|-------------------------------------------------------------|
| Caddy          | TLS or plain HTTP :80 → 127.0.0.1:8765                      |
| Ollama         | Optional LLM on :11434 (not in the three-image stack)       |
| FDD loop timer | Optional openfdd-fdd-loop.timer on host (docker exec batch) |

## 37 Lookback window helper

## 38 Lookback window helper

Rules that depend on **time history** (flatline, spread, streaks, trim/respond validation) should print a clear window summary in the Rule Lab console or local `run_test.py` output.

### 38.1 Recommended pattern (copy into `rule.py`)

Use module constants for the lookback you intend; the bridge passes the full feather window into `apply_faults_arrow` based on batch `lookback_hours` (default **1 h** on the scheduled loop; **24 h** when using **Update all records** with chunks).

```
import pyarrow.compute as pc

LOOKBACK_HOURS = 3 # tune: 1, 3, 6, 12, 24

def _kit_lookback_stats(table, *, hours=None):
    """Dev helper – prints row count, timestamp span, and
    configured lookback."""
    h = hours if hours is not None else LOOKBACK_HOURS
    if "timestamp" not in table.column_names:
        print(f"lookback={h}h rows={table.num_rows} (no timestamp
column)")
    return
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
```

```

        tmax = pc.max(ts).as_py()
        if tmin is None or tmax is None:
            print(f"lookback={h}h rows={table.num_rows} start=None
stop=None span=0.00h")
            return
        span_h = (tmax - tmin).total_seconds() / 3600.0
        print(
            f"lookback={h}h rows={table.num_rows} "
            f"start={tmin.isoformat()} stop={tmax.isoformat()}
span={span_h:.2f}h"
        )

    def _kit_value_stats(table, column):
        vals = pc.cast(table[column], "float64")
        print(
            f"column={column} min={pc.min(vals).as_py():.2f} "
            f"max={pc.max(vals).as_py():.2f}
mean={pc.mean(vals).as_py():.2f}"
        )

    def apply_faults_arrow(table, cfg, context=None):
        _kit_lookback_stats(table)
        _kit_value_stats(table, "oa-t")
        # ... fault logic ...

```

## 38.2 Why not hide this behind a library call yet?

A future `open_fdd.arrow_runtime.kit.lookback_stats(table, hours=3)` helper is reasonable once the Algorithms tab shares the same kit export path. For now, keeping the helper **inline in rule.py** matches the constants-first Rule Lab style and prints the same fields in:

- Local `run_test.py`
- Rule Lab **Quick test** console
- Batch run logs

## 38.3 Lookback vs rolling samples

| Concept                       | Controlled by                                       | Typical use                   |
|-------------------------------|-----------------------------------------------------|-------------------------------|
| <b>Historian<br/>lookback</b> | Batch <code>lookback_hours</code> ,<br>FDD loop env | How many rows are<br>in table |

| Concept               | Controlled by                                       | Typical use                           |
|-----------------------|-----------------------------------------------------|---------------------------------------|
| <b>Rolling window</b> | WINDOW_SAMPLES constant<br>(~12 ≈ 1 h @ 5 min poll) | Flatline, spread,<br>consecutive-true |

Always call `_kit_lookback_stats(table)` first so operators can confirm the table span matches expectations before interpreting rolling-window faults.

## 38.4 Batch intervals

| Trigger                                         | Default lookback                        |
|-------------------------------------------------|-----------------------------------------|
| openfdd-fdd-loop /<br>OFDD_FDD_INTERVAL_MINUTES | 1 h                                     |
| Rule Lab <b>Update all records</b>              | 24 h (chunk_hours: 6, use_chunks: true) |
| Rule Lab Quick test                             | 3 h (UI default)                        |

See [Windowing & debugging](#) for `arrow_rolling_min / arrow_consecutive_true`.

# 39 Data flow

# 40 Data flow

## 40.1 Telemetry ingest

Open-FDD supports three commissioning drivers. Each appends long-format samples to workspace CSVs and writes **feather shards** tagged by source (bacnet, modbus, json\_api).

### 40.1.1 BACnet

1. **Commission poll loop** (inside the commission container) reads enabled BACnet points from `points.csv` on a schedule (host network, BACnet :47808).
2. RPM results append to `workspace/bacnet/polls/samples.csv`.
3. **Bridge ingest worker** loads new rows into `feather_store/bacnet/`.

BACnet devices → commission (poll loop) → `samples.csv` → bridge (ingest) → `feather_store/bacnet`

### 40.1.2 Modbus TCP

1. Operator configures registers on the **Modbus** tab (or `read_and_store` API).
2. Poll worker reads holding/input registers via Modbus TCP.
3. Samples append to `workspace/modbus/polls/samples.csv` → `feather_store/modbus/`.

### 40.1.3 JSON API (HTTP/HTTPS)

1. Operator configures REST endpoints on the **JSON API** tab — GET/POST, Bearer or Basic auth, `${ENV:VAR}` placeholders from `json_api.env.local`, optional TLS verify off for self-signed OT gateways.
2. Poll worker issues HTTP requests (one GET can fan out to multiple JSON paths, e.g. OpenWeather `web-oat-t / web-rh / web-weather-desc`).
3. Samples append to `workspace/json_api/polls/samples.csv` → `feather_store/json_api/`.
4. FDD batch **merges** `json_api` columns with BACnet/Modbus on the same site (nearest timestamp, 30 min tolerance) for cross-source rules.

OT REST / weather API → bridge JSON API worker → `samples.csv` → `feather_store/json_api`

↘ merged

into FDD frame with `bacnet/modbus`

Showcase: [OpenWeatherMap bundle](#). Details: [Driver framework](#), [BACnet polling](#), [JSON API driver](#), [Containers](#).

## 40.2 Rule evaluation

1. Rules are Python files in `workspace/data/rules_py/` with metadata in `rules_store.json`.
2. **Bindings** map rules to points via the Brick model (`fdd_input` tags).
3. Batch runner (`POST /api/rules/batch` or host timer) produces `fdd_results.json`.
4. Dashboard **faults** view and check-engine status read aggregated results.

## 40.3 Model sync

- Commissioning imports `update_model.json` / TTL graph.
- `POST /api/model/sync-ttl` refreshes SPARQL-backed tree used by the UI.

# 40.4 Retention

Feather files grow on disk; retention is configurable (workspace/data.env.local). Image upgrades must **preserve** workspace/data/ — backup before docker compose up -d --force-recreate. See [Updating the stack](#).

# 41 Architecture

# 42 Architecture

Open-FDD v3 edge: **three Docker containers** (bridge, commission, mcp-rag), BACnet polling in **commission**, feather ingest in **bridge**, optional host Caddy and Ollama.

| Page                             | Topic                                                  |
|----------------------------------|--------------------------------------------------------|
| <a href="#">System overview</a>  | Components and diagram                                 |
| <a href="#">Deployment modes</a> | Local dev, lab LAN, edge production, Caddy HTTP/TLS    |
| <a href="#">Data flow</a>        | BACnet / Modbus / JSON API → feather → FDD → dashboard |
| <a href="#">Containers</a>       | GHCR images, ports, persistence, retired poll image    |
| <a href="#">Driver framework</a> | Shared commissioning pattern for OT sources            |

**Operators:** [Quick Start](#) — [Docker](#) — no git clone on the edge host.

**BACnet polling:** [Polling](#) — commission container only.

# 43 Rule Cookbook

# 44 Rule Cookbook

Practical patterns for **Arrow-native Rule Lab** (apply\_faults\_arrow) on feather historian PyArrow tables — **no pandas on the IoT edge**.



| Page                                             | Use when                                                                                 |
|--------------------------------------------------|------------------------------------------------------------------------------------------|
| <b><u>Expression cookbook (Arrow-native)</u></b> | <b>Full reference</b> — legacy YAML/pandas translation, sensor bounds, GL36, fault codes |
| <u>Arrow recipes</u>                             | Short templates — threshold, flatline, OOB, fan/schedule                                 |
| <u>Python recipes</u>                            | Shared <code>open_fdd.arrow_runtime.cookbook</code> imports                              |
| <u>Lookback window helper</u>                    | <code>_kit_lookback_stats</code> — print start/stop timestamps and span                  |
| <u>Windowing &amp; debugging</u>                 | Rolling windows, batch runtime, Rule Lab console                                         |
| <u>GL36 algorithm stubs</u>                      | Doc-only supervisory patterns (trim & respond, plant resets)                             |

Programmatic sensor defaults:

`open_fdd.arrow_runtime.sensor_catalog.SENSOR_PROFILES.`

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "zone_temp", cfg) #
fault_code: VAV-C
```

Future graph ML (sklearn / PyG) runs in a separate training service — see [GitHub issue #211](#).

## 45 Windowing & debugging

## 46 Windowing & debugging

### 46.1 Lookback vs rolling window

|               | Historian lookback                                                                   | Rolling window                                               |
|---------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Set by        | Batch <code>lookback_hours</code> , FDD loop, Quick test UI                          | <code>WINDOW_SAMPLES</code> constant in <code>rule.py</code> |
| Typical       | 1 h scheduled; 24 h Update all                                                       | 12 samples $\approx$ 1 h @ 5 min poll                        |
| Validate with | <code>_kit_lookback_stats(table)</code> — see <a href="#">Lookback window helper</a> | Rule logic + console flagged count                           |

## 46.2 Rolling windows (Arrow)

Use `open_fdd.arrow_runtime.windows` for sample-based rolling min/max and consecutive-true streaks:

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max, arrow_consecutive_true
```

Cookbook masks (`flatline_1h_mask`, `spread_1h_mask`, `oob_mask`) default to ~12 samples (~1 h at 5 min poll).

Bench rules use constants: `WINDOW_SAMPLES`, `FLATLINE_TOLERANCE`, `OAT_LOW` / `OAT_HIGH`, etc. Legacy `cfg` keys still work for uploaded rules that read `cfg.get(...)`.

## 46.3 Rule Lab console

Quick-test and batch responses include backend: `arrow`, `ms`, and flagged row counts. Arrow summary events appear in the event console on fault hits.

# 47 GL36 algorithm stubs

## 48 GL36 algorithm stubs (doc-only)

Draft **supervisory** PyArrow patterns for the future Algorithms tab. These mirror Niagara GL36 blocks documented in:

**README\_TRIM\_RESPOND.md**

FDD rules return **fault masks**. Algorithms return **setpoints or request integers** — the stubs below use `apply_algorithm_arrow` as the planned entryptpoint. For early testing in Rule Lab, you can adapt the logic into advisory FDD rules (boolean “sequence unhealthy” flags).

### 48.1 Shared lookback helper

```
import pyarrow.compute as pc
```

```
LOOKBACK_HOURS = 6
```

```

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

```

## 48.2 VAV zone cooling requests (0-3)

Simplified from GL36 VAV box request generator — zone temp vs cooling setpoint bands.

```

"""GL36 VAV cooling requests – doc stub."""

import pyarrow.compute as pc

ZONE_TEMP = "zn-t"
ZONE_SP = "zn-t-sp"
ZONE_DEMAND = "zn-cool-loop"
HIGH_DIFF_F = 5.0
MED_DIFF_F = 3.0
LOOP_ON = 95.0
LOOP_OFF = 85.0

def apply_algorithm_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    tz = pc.cast(table[ZONE_TEMP], "float64")
    sp = pc.cast(table[ZONE_SP], "float64")
    diff = pc.subtract(tz, sp)
    demand = pc.cast(table[ZONE_DEMAND], "float64")
    # Last row request level for kit demo (full port needs timer
state)

    last_diff = diff[-1].as_py() if table.num_rows else 0.0
    last_demand = demand[-1].as_py() if table.num_rows else 0.0
    if last_diff >= HIGH_DIFF_F:
        req = 3
    elif last_diff >= MED_DIFF_F:
        req = 2
    elif last_demand > LOOP_ON:
        req = 1
    else:
        req = 0
    print(f"cooling_requests={req} dT={last_diff:.1f}F
loop={last_demand:.0f}%")
    return {"cooling_requests": req}

```

## 48.3 AHU duct static trim & respond (advisory fault)

Detect **duct static stuck high** while VAV pressure requests are low — trim & respond may not be trimming.

```
"""AHU duct static trim advisory – doc stub."""

import pyarrow.compute as pc

DUCT_SP = "duct-static"
DUCT_SP_MAX = 1.50
DUCT_SP_TRIM_TARGET = 0.80
VAV_PRESS_REQ_SUM = "vav-pressure-req-sum"
LOOKBACK_HOURS = 12

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    sp = pc.cast(table[DUCT_SP], "float64")
    reqs = pc.cast(table[VAV_PRESS_REQ_SUM], "float64")
    stuck_high = pc.and_(pc.greater(sp, DUCT_SP_TRIM_TARGET),
pc.less(reqs, 1.0))
    return pc.and_(stuck_high, pc.greater(sp, DUCT_SP_MAX * 0.9))
```

## 48.4 Chiller plant enable (valve request counting)

From GL36 §5.18.15.2 — count AHUs with CHW valve above threshold.

```
"""Chiller plant enable advisory – doc stub."""

import pyarrow.compute as pc

AHU_VALVE_COLS = ["ahu1-clg-vlv", "ahu2-clg-vlv", "ahu3-clg-vlv"]
REQ_THRESH = 95.0
DIS_THRESH = 10.0
MIN_AHUS_REQ = 2

def apply_algorithm_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    requesting = 0
    for col in AHU_VALVE_COLS:
        if col not in table.column_names:
            continue
        v = pc.cast(table[col], "float64")[-1].as_py()
        if v >= REQ_THRESH:
```

```

        requesting += 1
        enable = requesting >= MIN_AHUS_REQ
        print(f"chiller_enable={enable} requesting_ahus={requesting}/
{MIN_AHUS_REQ}")
        return {"chiller_enable": enable, "requesting_ahus":
requesting}

```

## 48.5 Hot water plant HWST trim & respond (advisory)

Flags **HWST trimmed too low** while heating requests remain high.

```

"""HWST trim advisory – doc stub."""

import pyarrow.compute as pc

HWST = "hw-supply-t"
HWST_SP = "hw-supply-t-sp"
HEAT_REQ_SUM = "hw-reset-req-sum"
LOW_HWST_F = 120.0
LOOKBACK_HOURS = 6

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    temp = pc.cast(table[HWST], "float64")
    reqs = pc.cast(table[HEAT_REQ_SUM], "float64")
    return pc.and_(pc.less(temp, LOW_HWST_F), pc.greater(reqs,
2.0))

```

## 48.6 Chilled water plant (DP + CHWST reset advisory)

Flags **CHWST at design cold** with low cooling requests — possible stuck 100% plant loop.

```

"""CHW plant reset advisory – doc stub."""

import pyarrow.compute as pc

CHWST = "chw-supply-t"
CHWST_DESIGN_COLD_F = 44.0
COOL_REQ_SUM = "chw-reset-req-sum"
LOOKBACK_HOURS = 6

def apply_faults_arrow(table, cfg, context=None):

```

```

    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    temp = pc.cast(table[CHWST], "float64")
    reqs = pc.cast(table[COOL_REQ_SUM], "float64")
    return pc.and_(pc.less(temp, CHWST_DESIGN_COLD_F + 1.0),
pc.less(reqs, 1.0))

```

## 48.7 Testing later

1. Bind historian columns in **Model & assignments** (fdd\_input / brick types).
2. Copy a stub into Rule Lab as an FDD advisory rule, or wait for the **Algorithms** tab kit export.
3. Run python run\_test.py from a downloaded kit and confirm \_kit\_lookback\_stats prints expected start / stop / span.

Reference implementation (Niagara Java):  
[README\\_TRIM\\_RESPOND.md](#).

# 49 Dashboard overview

# 50 Dashboard overview

## 50.1 Primary areas

| Area                           | Purpose                                                      |
|--------------------------------|--------------------------------------------------------------|
| <b>Home / Building insight</b> | Check-engine status (green/yellow/red), active fault summary |
| <b>Trends</b>                  | Point history from feather store                             |
| <b>Faults</b>                  | Rule hits grouped by equipment family                        |
| <b>Rule Lab</b>                | Edit, lint, and test Python FDD rules (PyArrow kit zip)      |
| <b>Algorithms</b>              | Supervisory GL36 trim & respond (coming soon)                |
| <b>Model &amp; assignments</b> | Equipment tree, FDD rule pins, commissioning JSON            |
| <b>BACnet</b>                  | Discovery, reads, commission workflows                       |
| <b>Settings / health</b>       | Stack status, auth mode                                      |

## 50.2 Live updates

Dashboard tiles use REST polling and WS /ws/dashboard for selected live views (ticket auth via POST /api/auth/ws-ticket).

## 50.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See [Auth and roles](#).

# 51 Auth and roles

## 52 Auth and roles

### 52.1 Production (LAN / edge)

- Set OFDD\_AUTH\_SECRET and role passwords in workspace/auth.env.local.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

| Role       | Typical access                   |
|------------|----------------------------------|
| viewer     | Read trends, faults, model tree  |
| operator   | Run batch FDD, acknowledge views |
| integrator | Rule Lab, bindings, model import |
| commission | BACnet writes (plus write guard) |
| agent      | Agent tools / app-edit gates     |
| admin      | Full configuration               |

Login: POST /api/auth/login → Bearer JWT on API calls.

### 52.2 Localhost dev

OFDD\_BRIDGE\_HOST=127.0.0.1 with OFDD\_AUTH\_DISABLED=1 is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: [Security](#).

# 53 Rule Lab

# 54 Rule Lab

Rule Lab authors **Arrow-native** Python FDD rules against the feather historian. Bench rules use **module constants** at the top of rule.py — no browser config panel, no config.json in the dev kit zip.

```
"""Bench OA-T out of bounds (Arrow)."""

import pyarrow.compute as pc

VALUE_COLUMN = "oa-t"
OAT_LOW = 68.0
OAT_HIGH = 88.0
LOOKBACK_HOURS = 1

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def _kit_value_stats(table):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    print(
        f"column={VALUE_COLUMN} min={pc.min(vals).as_py():.2f} "
        f"max={pc.max(vals).as_py():.2f}
mean={pc.mean(vals).as_py():.2f}"
    )

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    _kit_value_stats(table)
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    return pc.or_(pc.less(vals, OAT_LOW), pc.greater(vals,
OAT_HIGH))
```



## 54.1 Workflow

1. **Download kit** — PyArrow zip (see below)
2. **Edit constants** locally (VALUE\_COLUMN, limits, LOOKBACK\_HOURS, WINDOW\_SAMPLES)
3. **Run** `pip install -r requirements.txt` then `python run_test.py`
4. **Upload** `rule.py` on Rule Lab (integrator)

Upload validation (Phase B): AST parse, forbidden imports, `apply_faults_arrow(table, cfg, context=None)` signature.

The bridge runs rules on **PyArrow tables** via `open_fdd.arrow_runtime`, and persists `.py` sources under `workspace/data/rules_py/`.

- **Quick test** — POST `/api/playground/test-rule` (default 3 h lookback)
- **Batch** — POST `/api/rules/batch / openfdd-fdd-loop timer (1 h default lookback)`
- **Update all records** — 24 h lookback, 6 h chunks (`use_chunks: true`)
- **Templates** — GET `/api/playground/arrow-templates`

Pin points to rules via **Model & assignments** (`/model`) commissioning JSON or BACnet tree right-click. Export shows Rule Lab **names** on each point (`fdd_rules_linked`); import uses rule **ids** (`fdd_rule_ids`).

Bench seed: `python3 scripts/setup_bench_afdd.py` — imports `bench_import_model.json` and bench cookbook rules with matching id/name pairs.

## 54.2 Dev kit zip (download)

**Download kit** on Rule Lab exports `openfdd-rule-kit-<rule_id>.zip` with:

| File                        | Purpose                                                                                                            |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>rule.py</code>        | Arrow rule + <b>constants</b> at top; optional <code>_kit_lookback_stats / _kit_value_stats</code> helpers         |
| <code>data.py</code>        | <code>SITE_ID</code> , <code>LOOKBACK_HOURS</code> , <code>load_table()</code> reading <code>sample.feather</code> |
| <code>sample.feather</code> | Full historian window for the kit lookback (all rows, not a CSV preview)                                           |
| <code>run_test.py</code>    | Runs <code>rule.apply_faults_arrow(table, {}, context=...)</code> and prints <code>flagged=N</code>                |

| File             | Purpose                                                    |
|------------------|------------------------------------------------------------|
| requirements.txt | open-fdd>=3.0.1 and pyarrow                                |
| README.md        | Install and run instructions                               |
| column_map.json  | BRICK logical keys → feather column names                  |
| kit_meta.json    | Export metadata (site, columns, row count, lookback hours) |

**Not included:** config.json (retired — tune thresholds as Python constants).

Local test:

```
unzip openfdd-rule-kit-*.zip
cd openfdd-rule-kit-*
pip install -r requirements.txt
python run_test.py
```

Expected console output:

```
site=demo lookback_h=3 rows=...
lookback=3h rows=... start=... stop=... span=2.98h
column=oa-t min=... max=... mean=...
flagged=...
```

## 54.3 Lookback windows

Rules that need multi-hour history should call `_kit_lookback_stats(table)` (see [Lookback window helper](#)). Pass `hours=1, 3, 6, 12, or 24` to validate timestamp span in the console.

Rolling-window rules (flatline, spread) use `WINDOW_SAMPLES` (~12 samples ≈ 1 h at 5 min poll) — see [Windowing & debugging](#).

## 54.4 Algorithms (supervisory — coming soon)

GL36 trim & respond and plant reset sequences will live on the **Algorithms** tab with the same zip kit shape but apply `algorithm_arrow` outputs. Reference: [README\\_TRIM\\_RESPOND](#).

## 54.5 Related

- [Model workflow](#)
- [Arrow recipes](#)

- [GL36 algorithm stubs](#) (doc-only)

## 55 JSON API driver

## 56 JSON API driver

Poll **HTTP or HTTPS** REST endpoints on the OT LAN (or public APIs such as [OpenWeatherMap](#)) and store extracted JSON values in the feather historian (source=json\_api).

Use this driver for gateways, micro-PLCs, IoT hubs, vendor APIs, and **web weather** when you want to cross-check shaky local outdoor-air sensors against a reference.

### 56.1 Supported features

| Feature                | Support                                                                                 |
|------------------------|-----------------------------------------------------------------------------------------|
| Methods                | GET, POST                                                                               |
| Transport              | http:// and https://                                                                    |
| TLS certificate verify | On by default; disable for self-signed OT gateways                                      |
| Authentication         | None, <b>Bearer token</b> , <b>HTTP Basic</b> , or <b>query-string API keys via env</b> |
| Env placeholders       | \${ENV:VAR} or \${VAR} in URL, headers, body, bearer token                              |
| Custom headers         | Optional headers map (merged with auth headers)                                         |
| Value extraction       | Dot-path into JSON body (title, main.temp, weather.0.description)                       |
| Multi-extract poll     | One HTTP request per URL group; fan-out to multiple json_path rows                      |
| Polling                | <b>1 / 5 / 15 / 30 min</b> or <b>1 hour</b> (shared with BACnet/Modbus)                 |
| Historian              | workspace/json_api/polls/<br>samples.csv → feather_store/<br>json_api/                  |
| FDD merge              |                                                                                         |

| Feature | Support                                                                      |
|---------|------------------------------------------------------------------------------|
|         | Scheduled FDD merges bacnet + modbus + json_api columns by nearest timestamp |

## 56.2 Secrets: json\_api.env.local

Copy workspace/json\_api.env.example → workspace/json\_api.env.local (gitignored). The bridge loads this file at startup and when run\_local.sh starts the stack.

Use placeholders in commissioning URLs so API keys never land in git:

```
https://api.openweathermap.org/data/2.5/weather?q=${ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=${ENV:OPENWEATHER_UNITS}
```

Check which vars are configured: GET /api/json-api/env/status.

## 56.3 OpenWeatherMap showcase (recommended demo)

This is the flagship example of what the generic JSON API driver can do — **one URL, three historian columns**, env-based API key, browser tree like BACnet/Modbus.

### 56.3.1 1. Configure env

```
cp workspace/json_api.env.example workspace/json_api.env.local
# Edit: OPENWEATHER_API_KEY, OPENWEATHER_CITY, OPENWEATHER_UNITS
(imperial = °F)
```

Restart the bridge (or ./scripts/run\_local.sh restart) so env vars load.

### 56.3.2 2. Register from the UI

1. Sign in as **integrator** → **JSON API** tab.
2. Open the **OpenWeatherMap showcase** panel.
3. Use the **OpenWeatherMap** preset → **Register** (default poll 30 min; choose interval in the form).

Three endpoints appear under api.openweathermap.org in the commissioning tree:

| Label            | JSON path             | Historian column | Units       |
|------------------|-----------------------|------------------|-------------|
| web-oat-t        | main.temp             | web-oat-t        | degF / degC |
| web-rh           | main.humidity         | web-rh           | %           |
| web-weather-desc | weather.0.description | web-weather-desc | text        |

Poll worker deduplicates: **one GET** serves all three extractions per cycle.

### 56.3.3 3. Headless / script

```
# Standalone fetch (like the original playground tester)
python3 scripts/openweather_json_api_example.py

# Register via REST (integrator login)
python3 scripts/openweather_json_api_example.py --register --base
http://127.0.0.1:8000
```

### 56.3.4 4. FDD: local OA-T vs web weather

With BACnet oa-t and JSON API web-oat-t in the same site, scheduled FDD merges both sources. Example rule:

workspace/data/rules\_py/oat\_vs\_web\_spread\_1h.py — flags when  $|oa-t - web-oat-t| > 8 \text{ }^{\circ}\text{F}$  (stuck or drifting outdoor-air sensor).

Enable it in Rule Lab, bind to your OA-T BACnet point, and run **Update all records**.

## 56.4 Commissioning (UI)

The **JSON API** tab follows [Home Assistant sensor.rest](#):

1. Sign in → **RESTful sensor (JSON API)** tab.
2. Pick a **preset** (JSONPlaceholder, DummyJSON, httpbin, Open-Meteo, OpenAQ, OpenWeather) or enter a **resource URL**.
3. Add one or more **sensors** — each row is a `value_json_path` + historian **label** (multi-value from one GET, like HA `json_attributes`).
4. Click **Test resource** — probes without saving; shows extracted values and raw JSON.
5. Click **Register & poll** — writes `endpoints.csv`, enables polling at the selected interval.
6. Poll choices match BACnet/Modbus: **1 / 5 / 15 / 30 min** or **1 hour**.

## 56.4.1 API for agents

| Endpoint                                 | Purpose                                                    |
|------------------------------------------|------------------------------------------------------------|
| GET /api/json-api/presets                | Full preset catalog + poll intervals (human + AI readable) |
| POST /api/json-api/test                  | Probe resource + multi-sensor extraction                   |
| POST /api/json-api/register-bundle       | Register one resource with many sensors                    |
| POST /api/json-api/presets/{id}/register | One-click preset registration                              |

## 56.4.2 Bench preset

Use **JSONPlaceholder** — **todo title** → **Test resource** to verify outbound HTTPS without field hardware.

## 56.5 Authentication examples

### 56.5.1 Query-string API key via env (OpenWeather)

```
{
  "url": "https://api.openweathermap.org/data/2.5/weather?q=${ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=${ENV:OPENWEATHER_UNITS}",
  "method": "GET",
  "json_path": "main.temp",
  "label": "web-oat-t",
  "auth_type": "none"
}
```

### 56.5.2 Bearer token (API key)

```
{
  "url": "https://192.168.10.50/api/v1/status",
  "method": "GET",
  "json_path": "sensors.zone_temp",
  "label": "zone-temp",
  "auth_type": "bearer",
  "bearer_token": "${ENV:GATEWAY_API_KEY}",
  "verify_tls": false
}
```

### 56.5.3 HTTP Basic

```
{
  "url": "http://192.168.10.50/data.json",
  "method": "GET",
  "json_path": "value",
  "label": "sensor-value",
  "auth_type": "basic",
  "basic_user": "operator",
  "basic_password": "changeme"
}
```

### 56.5.4 POST with JSON body

```
{
  "url": "https://gateway.local/api/read",
  "method": "POST",
  "json_path": "result.pv",
  "label": "pv",
  "body": { "point": "AHU-1_SAT", "command": "read" },
  "auth_type": "bearer",
  "bearer_token": "..."
}
```

Credentials in endpoints.csv are redacted in API responses (\*\*\*).  
Prefer \${ENV:...} for production keys.

## 56.6 REST API

| Method | Path                              | Role        | Description                                         |
|--------|-----------------------------------|-------------|-----------------------------------------------------|
| GET    | /api/json-api/env/status          | operator+   | Which env vars are configured (no secret values)    |
| POST   | /api/json-api/presets/openweather | integrator+ | Register 3-point OpenWeather bundle + optional poll |
| GET    | /api/json-api/driver/tree         | operator+   | Commissioning tree grouped by host                  |
| GET    | /api/json-api/poll/status         | operator+   | Poll worker status                                  |
| POST   | /api/json-api/poll/once           | integrator+ | Run one poll cycle                                  |
| POST   | /api/json-api/request             | operator+   | One-shot HTTP request (no store)                    |
| POST   | /api/json-api/read_and_store      | integrator+ | Request + CSV + feather ingest                      |

| Method | Path                              | Role        | Description                  |
|--------|-----------------------------------|-------------|------------------------------|
| POST   | /api/json-api/refresh             | operator+   | Re-fetch one endpoint        |
| PATCH  | /api/json-api/endpoint/poll       | integrator+ | Enable/disable poll interval |
| DELETE | /api/json-api/endpoint/{point_id} | integrator+ | Remove endpoint              |

Full route list: [API routes](#).

## 56.7 Typical OT URLs

| Device type    | Example URL                                         | Notes                                |
|----------------|-----------------------------------------------------|--------------------------------------|
| Web weather    | https://api.openweathermap.org/data/2.5/weather?... | Env-based appid; 3 historian columns |
| Gateway status | http://192.168.1.50/api/status                      | Often plain HTTP on LAN              |
| HTTPS BMS API  | https://10.0.0.12/rest/points/zone1                 | May need verify_tls: false           |
| Local mock     | https://jsonplaceholder.typicode.com/todos/1        | Bench / CI only                      |

## 56.8 Limitations

- Response body must be **JSON** (not XML or plain text).
- Numeric paths (e.g. main.temp) are stored as strings in poll CSV; FDD rules cast with `pc.cast(..., "float64")`.
- Outbound requests originate from the **bridge** container/host — ensure routing and firewall rules allow the edge box to reach the target (OT subnet or public internet for weather).

## 56.9 Disable polling (save API quota)

Stop scheduled calls without deleting endpoints: JSON API tree → select OpenWeather points → **Stop polling**, or integrator API:

```
curl -X PATCH http://127.0.0.1:8765/api/json-api/endpoint/poll \
-H "Authorization: Bearer $TOKEN" \
```



```
-d '{"point_id":"ja-api-openweathermap-org-get-web-oat-  
t","enabled":false,"poll_interval_s":0}'
```

API keys live in `json_api.env.local` (gitignored) — never committed; audit logs do not record expanded URLs with `appid=`. See [Logging and audit](#).

## 56.10 Smoke test

```
python3 scripts/smoke_multi_driver_stack.py
```

Validates JSON API ingest alongside BACnet and Modbus, feather isolation, BRICK scope, and FDD batch.

# 57 Algorithms

## 58 Algorithms (coming soon)

The **Algorithms** dashboard tab (`/algorithms`) is a placeholder for **supervisory Python sequences** — the mirror image of Rule Lab FDD:

|            | FDD (Rule Lab)                                       | Algorithms (planned)                                              |
|------------|------------------------------------------------------|-------------------------------------------------------------------|
| Purpose    | Detect faults                                        | Compute setpoints, request levels, plant enables                  |
| Entrypoint | <code>apply_faults_arrow(table, cfg, context)</code> | <code>apply_algorithm_arrow(table, cfg, context)</code> (planned) |
| Output     | Boolean fault mask                                   | Numeric outputs / structured dict                                 |
| Historian  | Same feather_store<br>PyArrow tables                 | Same                                                              |

Until the tab ships, draft patterns live in this doc and in [GL36 algorithm stubs](#).

### 58.1 GL36 Trim & Respond reference

Open-FDD algorithms will follow the same ASHRAE Guideline 36 trim & respond methodology already implemented for Niagara in:

**[README\\_TRIM\\_RESPOND.md](#)**

That guide covers:

- VAV zone request generators (cooling + static pressure)
- AHU duct static pressure reset
- AHU supply air temperature reset
- Central plant chiller / boiler enable and HWST / CHWST trim & respond

Porting those blocks to PyArrow is in progress; the dashboard tab shows **COMING SOON** until upload, batch, and BACnet write paths are wired.

## 58.2 Planned workflow (same kit shape as Rule Lab)

1. Download **algorithm kit** zip from the Algorithms tab (future)
2. Edit **constants** at the top of algorithm.py (no config.json)
3. Run `python run_test.py` locally against sample.feather
4. Upload algorithm.py when satisfied

Kit files will match the Rule Lab zip layout (see [Rule Lab — dev kit zip](#)).

## 58.3 Lookback windows

Rules and algorithms that need multi-hour history should use the shared **lookback stats helper** documented in [Lookback window helper](#). Pass `hours=1, 3, 6, 12, or 24` to print row count, timestamp start/stop, and span for console validation.

Scheduled batch defaults: **1 h** lookback on the FDD loop; **Update all records** in Rule Lab uses **24 h** with 6 h chunks.

## 58.4 AHU supervisory checks (doc-only stubs)

These are **not** uploaded rules yet — copy-paste references for future algorithm or FDD work.

### 58.4.1 Duct static pressure too high

Flags sustained high duct static (possible stuck setpoint, blocked filter, or trim/respond not trimming).

```
"""AHU duct static high – doc stub (Arrow constants)."""
```

```
import pyarrow.compute as pc
```

```

VALUE_COLUMN = "duct-static"
HIGH_INWC = 1.20
LOOKBACK_HOURS = 3
MIN_SAMPLES_ABOVE = 6

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    high = pc.greater(vals, HIGH_INWC)
    # Require several consecutive highs (~30 min at 5 min poll)
    from open_fdd.arrow_runtime.windows import
arrow_consecutive_true

    streak = arrow_consecutive_true(high, MIN_SAMPLES_ABOVE)
    return streak

```

## 58.4.2 Supply air temperature too cold (cooling coil over-delivering)

```

"""AHU SAT too cold vs setpoint – doc stub."""

import pyarrow.compute as pc

SAT_COLUMN = "sa-t"
SP_COLUMN = "sa-t-sp"
COLD_DELTA_F = 5.0
LOOKBACK_HOURS = 1

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

```

```
def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

## 58.5 Plant supervisory checks (doc-only)

See [GL36 algorithm stubs](#) for chiller plant enable, HWST trim & respond, and CHW DP/CHWST reset patterns aligned with the Niagara README.

## 58.6 Related

- [Rule Lab](#) — live FDD authoring
- [Model workflow](#) — point bindings
- [Windowing & debugging](#) — rolling windows

# 59 Model workflow

# 60 Model workflow

## 60.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (workspace/data/data\_model.ttl)
- **Point registry** — BACnet inventory → series\_id for historian bindings
- **Health score** — orphan points, missing fdd\_input, stale telemetry

## 60.2 Typical integrator flow

1. Import or sync BACnet point registry.
2. Map equipment classes (AHU, VAV, ...) in the model UI — set brick\_type on points and feeds on equipment (AHU→VAV).
3. Tag points with fdd\_input roles (zone temp, SAT, damper cmd, ...).

- 4. Bind Rule Lab rules to points, equipment, or BRICK classes (bindings in rules\_store.json).
- 5. Run POST /api/model/sync-ttl after bulk imports.
- 6. Validate on **Trend plot** with FDD overlays (?fdd=1) — faults render on a right-hand 0/1 axis per device scope.

AI Agent and commissioning JSON import can bulk-apply steps 2-4; integrator should review model health before production FDD. See [FDD and assignments](#) and [Ollama and analytics](#).

Export: GET /api/model/export (TTL/JSON). Commissioning bundle: GET /api/model/commissioning-export — includes per-point fdd\_rules\_linked (Rule Lab names) plus fdd\_rules[] catalog. Use **Import / export** tab preview table or **Point → FDD rule pins** before editing raw JSON. API detail in [Appendix](#).

## 61 Driver framework

## 62 Driver framework

Open-FDD edge commissioning uses a **shared driver pattern** for OT data sources. Each driver polls field devices on a schedule, stores long-format samples in workspace CSVs, and ingests into the **feather historian** under a distinct source tag for FDD and plots.

### 62.1 Drivers

| Driver                   | Protocol                                                  | Feather source | Commissioning tab |
|--------------------------|-----------------------------------------------------------|----------------|-------------------|
| <a href="#">BACnet</a>   | MS/TP / IP (:47808)                                       | bacnet         | BACnet            |
| Modbus                   | Modbus TCP                                                | modbus         | Modbus            |
| <a href="#">JSON API</a> | HTTP/HTTPS REST (+ <a href="#">OpenWeather showcase</a> ) | json_api       | JSON API          |

All three drivers share the same operator workflow:

- 1. **Discover or configure** — add devices/endpoints to the driver registry.
- 2. **Request once** — on-demand read to verify connectivity and JSON path / register map.

3. **Request & store** — append to poll CSV and write a feather shard.
4. **Enable polling** — 1 / 5 / 10 / 15 minute intervals via tree right-click or bulk toolbar.
5. **Poll all now** — force one worker cycle without waiting for the interval.

## 62.2 Data layout

```
workspace/
  bacnet/commissioning/  points.csv, discovered inventory
  bacnet/polls/          samples.csv
  modbus/commissioning/  registers.csv
  modbus/polls/          samples.csv
  json_api/commissioning/ endpoints.csv
  json_api/polls/        samples.csv
  data/feather_store/
    bacnet/{site_id}/    shard-*.feather
    modbus/{site_id}/
    json_api/{site_id}/
```

## 62.3 Historian and FDD

- **Plot tab** — numeric columns from bacnet and modbus sources (GET /api/timeseries/plot?source=...).
- **String JSON fields** — stored in feather under json\_api; use the JSON API tree or feather directly (plot is numeric-only).
- **FDD batch** — merges bacnet + modbus + json\_api historian columns by nearest timestamp; rules use BRICK-bound external\_id names (e.g. compare BACnet oa-t vs JSON API web-oat-t — see [JSON API](#)).
- **BRICK model** — bind points in **Model & assignments** commissioning JSON; sync TTL for SPARQL scope.

## 62.4 Rule Lab (Arrow upload/download)

1. **Download kit** — zip with rule.py (constants at top), data.py, sample.feather (~3h), run\_test.py, requirements.txt.
2. Local test: pip install -r requirements.txt then edit constants in rule.py and python run\_test.py.
3. **Upload rule.py** — Arrow-only; must define apply\_faults\_arrow(table, cfg, context=None).
4. Browser shows **read-only** source; use Quick test + Update all records.

API: GET /api/rules/export-kit, POST /api/rules/upload.

# 62.5 Smoke validation

From repo root on a running stack (<http://127.0.0.1:8765>):

```
python3 scripts/smoke_multi_driver_stack.py
python3 scripts/validate_modbus_temp_e2e.py --no-server
```

# 62.6 Related docs

- [Data flow](#) — poll → CSV → feather → FDD
- [API routes](#) — REST reference
- [BACnet polling](#) — commission container loop

# 63 Operator Bridge

# 64 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

| Page                                 | Topic                                            |
|--------------------------------------|--------------------------------------------------|
| <a href="#">Dashboard overview</a>   | Main UI areas                                    |
| <a href="#">Auth and roles</a>       | Login and permissions                            |
| <a href="#">Rule Lab</a>             | Python FDD rule authoring (PyArrow kit zip)      |
| <a href="#">Algorithms</a>           | Supervisory GL36 sequences (coming soon)         |
| <a href="#">Model workflow</a>       | Brick bindings and imports                       |
| <a href="#">FDD and assignments</a>  | Rule bindings, BRICK classes, commissioning JSON |
| <a href="#">Ollama and analytics</a> | Home briefing, zone levers, model RAM tiers      |

REST details: [Appendix — API routes](#). Env files: [Workspace env files](#).

# 65 Ollama and analytics

## 66 Ollama and analytics

### 66.1 What runs without login

The **Building status** home page (/) and GET /openfdd-agent/building-insight are **public** (same policy as fault status). Anyone on the LAN can read the briefing sentence and underlying numeric levers; only the **AI Agent** chat and stack revision endpoints require sign-in.

### 66.2 Ollama setup

1. Copy workspace/ollama.env.example → workspace/ollama.env.local.
2. Pick a model for available RAM (comments in the example file):

| RAM tier | Suggested model |
|----------|-----------------|
| 4 GB Pi  | qwen3:0.6b      |
| 8 GB     | qwen3:1.7b      |
| 16 GB    | qwen3:4b        |
| 32 GB    | qwen3:8b        |
| 64 GB    | qwen3:14b       |

1. Set OFDD\_OLLAMA\_BASE\_URL (default http://127.0.0.1:11434).
2. Optional: OFDD\_OLLAMA\_GPU\_MODE (cpu / auto / gpu), OFDD\_OLLAMA\_TIMEOUT\_S for slow CPUs.
3. Bootstrap helper: ./scripts/bootstrap\_ollama.sh.

Bridge reads env via run\_local.sh / compose. Model resolution: workspace/api/openfdd\_bridge/ollama\_profiles.py.

### 66.3 Automatic analytics (deterministic, always on)

These pandas calculations run on a refresh interval and feed the home dashboard **whether or not Ollama is up**:



| Lever                     | Module                                      | Refresh env                               | What it computes                                                                                                                                                          |
|---------------------------|---------------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zone temperature snapshot | zone_temp_analytics.py                      | OFDD_ZONE_TEMP_INTERVAL_S (default 3600s) | Site-wide and per AHU zone temperature averages, spread out-of-band computed using BRICK feeds (AHU→VAV) which are modeled; falls back to all Zone_Air_Temperature points |
| Device poll health        | device_poll_health.py                       | stack health cycle                        | BACnet device liveness seen / stale poll detection                                                                                                                        |
| Operational brief         | building_insight.py → get_operational_brief | same as insight                           | Compact numerical status for integrators                                                                                                                                  |
| Root-cause hints          | root_cause_hints.py                         | on insight refresh                        | Multi-zone fault chains using brick:feeds from                                                                                                                            |

Shared lookback window: OFDD\_ANALYTICS\_LOOKBACK\_DAYS (default 14) in operational\_analytics.py.

API routes (auth optional unless noted):

- GET /openfdd-agent/building-insight — sentence + zone snapshot + fault sentences
- GET /openfdd-agent/zone-temp-analytics
- GET /openfdd-agent/device-poll-health
- GET /openfdd-agent/ollama/health — integrator diagnostics

## 66.4 When Ollama is used

building\_insight.get\_building\_insight():

1. Refreshes zone + device snapshots (pandas).
2. Calls ollama\_client.health().
3. If Ollama is reachable and should\_use\_ollama\_for\_insight() passes, sends a **compact context** (status, zone levers, feeds chains) to the configured model for a one-sentence briefing (source: "ollama").
4. On failure, uses \_fallback\_sentence() (source: "deterministic").

Context assembly: building\_insight.\_compact\_context(). Prompt template references BRICK feeds and root\_cause\_hints.

# 66.5 AI Agent (authenticated)

/agent page uses POST /openfdd-agent/chat with tool access (agent\_tools.py): BRICK graph, scope bundles, rule binding patches, model health, etc. This is separate from the public home briefing.

# 66.6 Code map

```
workspace/api/openfdd_bridge/
  building_insight.py      # cached home briefing + Ollama
sentence
  zone_temp_analytics.py  # zone temp levers (BRICK feeds)
  device_poll_health.py   # BACnet poll staleness
  operational_analytics.py # lookback window + methodology dict
  root_cause_hints.py     # multi-fault BRICK chain hints
  ollama_client.py        # HTTP client to Ollama
  ollama_profiles.py      # RAM tiers / model names
```

Dashboard consumers: BuildingStatusPage.tsx,
StackStatusStrip.tsx (WebSocket /ws/dashboard).

# 67 FDD and assignments

# 68 FDD and assignments

## 68.1 What the product actually does

Open-FDD separates **data modeling** (BRICK equipment, points, feeds relationships) from **rule assignment** (which FDD rules apply to which points, equipment, or BRICK classes). Both are editable in **Model & assignments** and via commissioning JSON import.

### 68.1.1 BRICK modeling (integrator + AI-assisted)

| Concern        | Where it lives          | Notes                                                      |
|----------------|-------------------------|------------------------------------------------------------|
| Equipment tree | ModelService + TTL sync | AHU, VAV, sensors; equipment_type and brick_type on points |

| Concern              | Where it lives                       | Notes                                                                   |
|----------------------|--------------------------------------|-------------------------------------------------------------------------|
| brick:feeds          | model_feeds.py,<br>ttl_service.py    | AHU→VAV→zone relationships; used by zone analytics and root-cause hints |
| Point registry       | BACnet inventory → model points      | fdd_input roles (oa-t, stat_zn-t, ...), series_id / historian column    |
| Commissioning import | POST /api/model/commissioning-import | Bulk JSON: sites, equipment, points, optional fdd_rules                 |

**AI Agent** (agent\_tools.py) can read slim\_brick\_graph, scope\_bundle, patch rule bindings, and upsert points — but it does **not** auto-generate a full site model without operator review. Expected workflow:

1. BACnet discover/poll populates point registry.
2. Integrator or agent tags brick\_type, fdd\_input, and feeds on equipment.
3. POST /api/model/sync-ttl writes workspace/data/data\_model.ttl.
4. Model health (model\_health.py) flags orphan points and missing fdd\_input.

### 68.1.2 Rule assignment (three binding kinds)

Saved rules in rules\_store.json carry bindings:

```
{
  "point_ids": ["5007-analog-input-1173"],
  "equipment_ids": ["bench-1"],
  "brick_types": ["Outside_Air_Temperature_Sensor"]
}
```

| Binding kind  | Effect                                            |
|---------------|---------------------------------------------------|
| point_ids     | Rule runs for that historian point / series       |
| equipment_ids | Rule runs for all points on equipment (mass bind) |
| brick_types   |                                                   |

| Binding kind | Effect                                                |
|--------------|-------------------------------------------------------|
|              | Rule runs for any point with that BRICK class on site |

Assign via:

- **Model & assignments** UI — equipment/point chips, rule pin menu on trend plot
- **Rule Lab** — bindings panel
- **Commissioning import** — `fdd_rules[].bindings` and per-point `fdd_rule_ids`
- **AI Agent** — `patch_rule_binding` tool

Runtime evaluation: `fdd_runner.py` (batch FDD) and `plot_readings.evaluate_fault_plots()` (trend overlays). Plot scope filters rules to those bound to **selected telemetry** (point, equipment, or BRICK class); unbound rules still evaluate site-wide.

### 68.1.3 CLASS vs device

- **BRICK class** (`brick_type` on points, or `brick_types` on rule bindings) — template-style assignment (“all `Zone_Air_Temperature_Sensor`”).
- **Device / equipment** (`equipment_id`, BACnet device instance) — instance assignment (“this VAV-12”).
- **Point** (`point_ids`) — single-sensor assignment.

`applies_to` on legacy rules is deprecated; use bindings in 3.0+.

## 68.2 Commissioning JSON shape

Export: `GET /api/model/commissioning-export`. Import accepts `import_ready_json` wrappers (see `commissioningImport.ts`).

### 68.2.1 Rule names vs ids (Rule Lab alignment)

Rule Lab shows **human names** (Bench OA-T flatline 1h). Saved rules use stable **ids** (`bench-oa-t-flatline-1h` from setup scripts, or a UUID for uploaded custom rules). Commissioning export includes both so you never have to cross-reference sections by hand:

| Field                              | Direction       | Purpose                                |
|------------------------------------|-----------------|----------------------------------------|
| <code>points[].fdd_rule_ids</code> | import + export | Assignment surface — array of rule ids |

| Field                     | Direction          | Purpose                                                                |
|---------------------------|--------------------|------------------------------------------------------------------------|
| points[].fdd_rules_linked | <b>export only</b> | [{id, name}] — Rule Lab names beside each point                        |
| fdd_rules[].id            | import + export    | Stable rule key (matches Rule Lab / rules_store.json)                  |
| fdd_rules[].name          | import + export    | Same label as Rule Lab dropdown                                        |
| fdd_rules[].source_file   | export             | Basename of deployed .py when present (e.g. bench_oa-t_flatline_1h.py) |

On import, fdd\_rules\_linked is stripped (like fdd\_rule\_ids — not stored in model.json). Use the **Point → FDD rule pins** table on Model & assignments to read names without parsing JSON.

Minimum fields integrators/AI should produce:

- sites, equipment[] with id, equipment\_type, optional feeds[]
- points[] with site\_id, equipment\_id, brick\_type, fdd\_input, BACnet IDs, optional fdd\_rule\_ids
- fdd\_rules[] with id, name, bindings, enabled (optional source\_file on export)

After import, verify in **Trend plot** with ?fdd=1 — fault lanes appear on the right-hand 0/1 axis for bound rules.

## 68.3 Related

- [Model workflow](#)
- [Rule Lab](#)
- [Trend plot / fault overlays](#)

# 69 Network setup

## 70 Network setup

### 70.1 Bind address

Commission and poll containers need the **OT interface** IP and UDP **47808**:

```
# workspace/bacnet/commissioning/commission.env  
BACNET_BIND=192.168.1.50/24:47808  
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses `network_mode: host` on Linux edges.

### 70.2 Discovery range

```
DISCOVER_LOW=1  
DISCOVER_HIGH=4194302  
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

### 70.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

## 71 Discover and read

## 72 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

| Capability     | Notes                                           |
|----------------|-------------------------------------------------|
| Who-Is / I-Am  | Device discovery on OT subnet                   |
| Read property  | Present value, object name, units               |
| RPM            | Bulk reads for inventory export                 |
| Write property | <b>Gated</b> — see <a href="#">Write safety</a> |

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

## 72.1 Operator workflow

1. Confirm [network setup](#).
2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.

# 73 Polling

# 74 Polling

**BACnet field reads run in the `openfdd-commission` container.**  
The Operator Bridge does not bind UDP 47808.

**Retired:** `openfdd-bacnet-poll` image and `openfdd-bacnet-poll` systemd unit. Poll loop lives in **commission** only — do not run both.

BACnet polling has two stages:

1. **Field reads** — commission poll loop → `workspace/bacnet/polls/samples.csv`
2. **Historian ingest** — bridge background worker → `workspace/data/feather_store/`

## 74.1 Commission poll loop (default)

The **commission** container runs the poll loop at startup. It reads enabled rows from `points.csv` and appends BACnet RPM results to `samples.csv`.

| Setting     | Location                                                   |
|-------------|------------------------------------------------------------|
| BACnet bind | <code>workspace/bacnet/commissioning/commission.env</code> |

| Setting        | Location                                  |
|----------------|-------------------------------------------|
| Enabled points | workspace/bacnet/commissioning/points.csv |
| Poll output    | workspace/bacnet/polls/samples.csv        |
| Historian      | workspace/data/feather_store/             |

Check activity:

```
tail -1 workspace/bacnet/polls/samples.csv
docker compose logs --since 5m commission | grep -i poll
```

## 74.2 Bridge ingest worker

Inside **bridge**, a background thread watches `samples.csv` and ingests new rows into feather. Disable only for debugging:  
`OFDD_DISABLE_POLL_WORKER=1`.

## 74.3 Health

Dashboard stack health (GET `/health/stack`) reports `bacnet_poll` status from the commission agent — not a separate container.

Stale points trigger data-quality fault patterns — see [Sensor quality faults](#).

# 75 Write safety

# 76 Write safety

**BACnet writes can change live plant behavior.** Open-FDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

## 76.1 Defaults

| Control            | Default                              |
|--------------------|--------------------------------------|
| Supervisory writes | <b>Off</b>                           |
| Write allowlist    | Empty — must enumerate device/object |
| Audit              | Commands logged when writes enabled  |



## 76.2 Enable writes (commission role)

1. Set env flag documented in [Security](#) (OFDD\_BACNET\_WRITE\_ENABLED or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/ note where API supports it.
4. Test on bench hardware before production OT.

## 76.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

# 77 BACnet

# 78 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

| Page                              | Topic                   |
|-----------------------------------|-------------------------|
| <a href="#">Network setup</a>     | Bind address and NIC    |
| <a href="#">Discover and read</a> | Commission agent        |
| <a href="#">Polling</a>           | Historian driver        |
| <a href="#">Write safety</a>      | Defaults and allowlists |

# 79 Convention

## 80 Fault code convention

### 80.1 Format

FAMILY-SUFFIX — suffix is **1-3 letters** (e.g. VAV-C, AHU-B). Numeric suffixes like VAV-03 are avoided (collision with physical equipment names).

### 80.2 Categories

| Category                | Meaning                              |
|-------------------------|--------------------------------------|
| performance_degradation | Efficiency or capacity drift         |
| simultaneous_heat_cool  | Heating and cooling fighting         |
| sensor_fault            | Flatline, OOB, inconsistent readings |
| io_fault                | Command vs feedback mismatch         |

### 80.3 Severity

| Level    | Operator response                                         |
|----------|-----------------------------------------------------------|
| info     | Log; trend for maintenance window                         |
| warning  | Investigate within days                                   |
| critical | Prompt attention; may affect comfort/energy significantly |

### 80.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into GET `/api/faults/status` by family.

### 80.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`, `mixing_envelope`, `rate_of_change`) — see [Expression cookbook \(Arrow-native\)](#).

Legacy pandas type: expression YAML rules are **not** used on the edge in 3.x; translate to rules\_py Arrow modules and bind **fault\_code** in Rule Lab.

## 81 Live site update

## 82 Live site update

Upgrade GHCR images on a **live edge host** that already has ~/open-fdd/ — no git pull on the host.

**IT operators:** prefer Quick Start — Updating the stack and openfdd\_site\_backup.sh / openfdd\_site\_update.sh on the edge host. This page adds control-machine UI deploy and Ansible paths.

### 82.1 Layout on the edge host

```
~/open-fdd/  
  docker-compose.yml  
  workspace/          # site state – backup first
```

workspace/ holds BACnet commissioning, poll CSV, feather historian, BRICK model, FDD rules, and auth.env.local. Image upgrades must **preserve** it.

See Backup and restore before any upgrade.

### 82.2 Concepts

| Term             | Meaning                                                               |
|------------------|-----------------------------------------------------------------------|
| <b>Container</b> | Running instance (bridge, commission, mcp-rag) — recreated on upgrade |
| <b>Image</b>     | GHCR package (tag latest or dated tag)                                |
| workspace/       | Open-FDD site state on disk — <b>do not delete</b>                    |

## 82.3 1. Verify new images on GHCR

Three images must exist for every tag:

```
export NEW_TAG="${NEW_TAG:-latest}"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:${NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${NEW_TAG} >/dev/null &&
echo "All three images exist for ${NEW_TAG}"
```

## 82.4 2. Update dashboard UI (when release changed React)

**Image pull alone does not update the browser UI.** The Operator Bridge serves workspace/api/static/app/ from the bind mount **before** image-baked assets.

**From control machine (full repo):**

```
cd /path/to/open-fdd
./scripts/build_operator_dashboard.sh prod
cd infra/ansible
RUN_POST_CHECK=0 ./deploy.sh ui --limit <inventory_host>
```

Confirm hash on edge:

```
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.' | head -1
```

**One command (UI + images + insurance check):**

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit <inventory_host>
```

## 82.5 3. Pull and recreate (edge host)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Manual equivalent:

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
```

```
docker compose ps
curl -sf http://127.0.0.1:8765/health
```

## 82.6 4. Ansible image-only (control machine)

```
OPENFDD_IMAGE_TAG=latest RUN_POST_CHECK=1 ./scripts/
upgrade_edge_ghcr.sh --limit <inventory_host>
```

## 82.7 5. Validate

→ [Deployment validation](#)

## 82.8 Rollback

Restore docker-compose.yml from backup, pull previous tag, docker compose up -d --force-recreate. workspace/ is unchanged. Details: [Backup and restore](#).

## 82.9 Related

- [Updating the stack](#) — edge-host helper scripts
- [Containers](#) — three-image architecture

# 83 AHU faults

# 84 AHU faults

Air handling unit codes (sample from catalog).

| Code  | Title                              | Severity | Detection summary          | Likely causes                       | Operator checks                     |
|-------|------------------------------------|----------|----------------------------|-------------------------------------|-------------------------------------|
| AHU-A | Supply fan performance degradation | warning  | Spread/runtime vs baseline | Belt slip, dirty filters, VFD limit | Trend speed vs airflow; filter dP   |
| AHU-B | Simultaneous heating and cooling   | critical | Heat + cool outputs        | Sequencing bug, leaking valve       | Trend coil commands; valve feedback |

| Code         | Title                               | Severity | Detection summary               | Likely causes                  | Operator checks                          |
|--------------|-------------------------------------|----------|---------------------------------|--------------------------------|------------------------------------------|
|              |                                     |          | active together                 |                                |                                          |
| <b>AHU-C</b> | SAT sensor fault                    | warning  | Flatline / OOB SAT              | Failed sensor, bad calibration | SAT trend; compare to coil discharge     |
| <b>AHU-D</b> | MAT inconsistent with OAT/RAT       | warning  | MAT outside OAT-RAT envelope    | MAT/OAT/RAT sensor errors      | Verify damper positions; cross-check OAT |
| <b>AHU-E</b> | Economizer not economizing          | warning  | Free cool available, OA minimum | Stuck damper, lockout          | OA damper vs OAT trend                   |
| <b>AHU-F</b> | Damper command vs feedback mismatch | warning  | Cmd $\neq$ feedback             | Stuck actuator, linkage        | Compare cmd/feedback trends              |

**Example rule:** flatline\_1h on SAT → tag **AHU-C**. Recipe: [Python recipes](#).

## 85 Backup and restore

## 86 Backup and restore

workspace/ on the edge host is the **live site state** — feather historian, BACnet commissioning files, BRICK model, FDD rules, auth, and dashboard static bundle. Container images are replaceable; **backup workspace/ before every upgrade**.

### 86.1 Quick backup (edge host)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives land under ~/openfdd-backups/<timestamp>/.

Custom location:

```
BACKUP_ROOT=~/.openfdd-backups/manual ./scripts/  
openfdd_site_backup.sh
```

## 86.2 Manual backup (SSH)

Container processes may write files as **root:root** with mode **0600**.  
Use **sudo** for a full archive:

```
cd ~/open-fdd  
  
export BACKUP_ROOT="$HOME/openfdd-backups/$(date +%Y%m%d-%H%M%S)"  
mkdir -p "$BACKUP_ROOT"  
  
cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before"  
docker compose ps > "$BACKUP_ROOT/docker-compose-ps-before.txt"  
docker compose config --images > "$BACKUP_ROOT/docker-images-  
before.txt"  
  
sudo tar --xattrs --acls -czf "$BACKUP_ROOT/workspace-full.tgz"  
workspace  
sudo chown "$USER:$USER" "$BACKUP_ROOT/workspace-full.tgz"  
  
du -h "$BACKUP_ROOT/workspace-full.tgz"
```

## 86.3 What to protect

| Path                                   | Contents                                     |
|----------------------------------------|----------------------------------------------|
| workspace/data/<br>feather_store/      | Feather historian                            |
| workspace/bacnet/<br>commissioning/    | points.csv, commission.env                   |
| workspace/bacnet/<br>polls/samples.csv | Poll output                                  |
| workspace/data/*.json                  | Model, rules store, FDD results              |
| workspace/<br>auth.env.local           | Login secrets                                |
| workspace/api/static/<br>app/          | Dashboard bundle (if deployed<br>separately) |

## 86.4 Restore after a bad upgrade

1. Stop containers: `docker compose stop` (do **not** use `down -v`)
2. Restore `docker-compose.yml` from backup
3. If data corruption: extract selective files from `workspace-full.tgz` — model, rules, feather shards

4. docker compose pull && docker compose up -d --force-recreate
5. Verify: [Deployment validation](#)

## 86.5 Safe cleanup

```
docker image prune -f
```

**Never on live sites:** docker compose down -v, docker volume prune, docker system prune -a --volumes, or deleting workspace/.

## 86.6 Related

- [Updating the stack](#)
- [Live site update](#)

# 87 VAV faults

# 88 VAV faults

Variable air volume terminal codes (sample).

| Code  | Title                         | Severity | Detection summary                  | Likely causes                   | Operator checks                          |
|-------|-------------------------------|----------|------------------------------------|---------------------------------|------------------------------------------|
| VAV-A | Zone heating/cooling overlap  | warning  | Heat and cool at terminal          | Reheat valve stuck, SAT too low | Zone temp vs SAT; valve cmd              |
| VAV-B | Poor zone temperature control | warning  | Persistent deviation from setpoint | Oversized box, bad tuning       | PI loop trends; occupancy                |
| VAV-C | Zone temperature sensor fault | warning  | Flatline / OOB zone temp           | Failed zone sensor              | Flatline test; compare to neighbor zones |
| VAV-D | Minimum airflow not met       | warning  | Flow below                         | Damper stuck,                   | Airflow vs damper cmd                    |



| Code         | Title                             | Severity | Detection summary         | Likely causes            | Operator checks |
|--------------|-----------------------------------|----------|---------------------------|--------------------------|-----------------|
|              |                                   |          | minimum cmd               | pressure issue           |                 |
| <b>VAV-E</b> | Discharge temp deviation from SAT | warning  | DAT far from SAT setpoint | Coil issue, sensor drift | DAT vs SAT SP   |

**Example rule:** oob\_rolling on zone temp → **VAV-C** or comfort bounds per site policy.

## 89 RTU faults

## 90 RTU faults

Packaged rooftop unit codes (sample).

| Code         | Title                        | Severity | Detection summary           | Likely causes              | Operator checks                 |
|--------------|------------------------------|----------|-----------------------------|----------------------------|---------------------------------|
| <b>RTU-A</b> | Compressor short cycling     | warning  | Rapid on/off cycles         | Low charge, oversized unit | Run-time histogram              |
| <b>RTU-B</b> | Economizer / OA damper fault | warning  | OA behavior vs conditions   | Actuator, sensor           | OA damper vs enthalpy/OAT       |
| <b>RTU-C</b> | Supply air temperature fault | warning  | SAT flatline or OOB         | Sensor failure             | SAT trend vs outdoor conditions |
| <b>RTU-D</b> | Simultaneous heat and cool   | critical | Heat + cool stages together | Control board, relay stuck | Stage status trends             |

Bind rules to packaged unit fdd\_input points (SAT, OAT, compressor cmd, ...).

# 91 Deployment validation

## 92 Deployment validation

Non-destructive checks after deploy or image upgrade. Run from the **edge host** (smoke) or from a **control machine** with the full repo (insurance suite).

### 92.1 Edge host smoke (5 minutes)

```
cd ~/open-fdd

docker compose ps
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health

tail -1 workspace/bacnet/polls/samples.csv
du -sh workspace/data/feather_store/

docker compose logs --since 20m | grep -Ei "error|exception|
traceback|critical" | tail -20 || true
```

Via Caddy (building LAN):

```
curl -sf http://<edge-ip>/health
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.' | head -1
```

### 92.2 Browser checklist

1. Open `http://<edge-ip>/` (Caddy :80, not only loopback :8765)
2. Integrator login (`workspace/auth.env.local` on the edge host)
3. **Host stats** — container image tag / git SHA
4. **BACnet** — driver tree shows devices; last poll timestamp recent
5. **Model & assignments** — commissioning export returns JSON
6. **Rule Lab** — quick-test a saved rule (backend: arrow)

### 92.3 Control-machine insurance check

From a machine with the full open-fdd repo and Ansible inventory:

```
cd infra/ansible/scripts
./post_deploy_check.sh --limit <inventory_host> --full
```

Or by IP:

```
./post_deploy_check.sh --host <edge-ip> --ssh-user <deploy-user>
--full
```

Checks include: Caddy → React dashboard, /health/stack, MCP RAG, BACnet tree, BRICK/SPARQL, Docker log scan.

**Do not run on live commissioned sites without approval:** acme\_operational\_verify.sh with BACnet rediscover — use --skip-discover for smoke only. See [Examples — GL36 lab](#).

## 92.4 LAN security smoke (optional)

From a workstation on the same LAN as the edge:

- Windows: scripts/security/Run-OpenFddSecurityScan.ps1
- macOS/Linux: scripts/security/run\_openfdd\_security\_scan.sh

Expected findings: [ZAP baseline](#).

## 92.5 Arrow / FDD rules (3.0+)

On control machine:

```
./scripts/validate_fdd_backends.sh --docker
```

On edge after upgrade:

```
grep -R "apply_faults_arrow" workspace/data/rules_py | wc -l
```

## 92.6 Related

- [Health check](#)
- [Security testing cycle](#) (maintainers)

# 93 Sensor and data quality

## 94 Sensor and data quality

Sensor faults use cookbook patterns `flatline_1h`, `oob_rolling`, `rate_of_change`, and `mixing_envelope`. Full thresholds: [Expression cookbook — sensor validation](#).

### 94.1 Catalog codes

| Code         | Title                               | Severity | Pattern                       | Detection summary                       |
|--------------|-------------------------------------|----------|-------------------------------|-----------------------------------------|
| <b>BLD-B</b> | Outdoor air sensor fault            | warning  | flatline, oob, rate_of_change | OAT flatline, OOB, or unrealistic spike |
| <b>BLD-D</b> | Stale telemetry                     | critical | stale_points                  | No new samples within threshold         |
| <b>AHU-C</b> | SAT sensor fault                    | warning  | flatline_1h, oob_rolling      | Supply air temp stuck or out of band    |
| <b>AHU-D</b> | MAT inconsistent with OAT/RAT       | warning  | mixing_envelope               | Mixed air outside OAT-RAT envelope      |
| <b>VAV-C</b> | Zone temperature sensor fault       | warning  | flatline_1h, oob_rolling      | Zone temp stuck or 55–90 °F band breach |
| <b>RTU-C</b> | Supply air temperature fault        | warning  | flatline_1h, oob_rolling      | Packaged unit SAT fault                 |
| <b>CH-D</b>  | CHW supply temperature sensor fault | warning  | flatline_1h, oob_rolling      | Plant CHW sensor fault                  |
| <b>DC-C</b>  | Datacom sensor anomaly              | warning  | flatline_1h                   | CRAH/room sensor flatline or RH OOB     |
| <b>HP-D</b>  |                                     | warning  |                               |                                         |

| Code | Title                                                | Severity | Pattern                     | Detection summary                               |
|------|------------------------------------------------------|----------|-----------------------------|-------------------------------------------------|
|      | Discharge/<br>suction<br>temperature<br>sensor fault |          | flatline_1h,<br>oob_rolling | Heat pump<br>refrigerant/<br>air temp<br>sensor |

## 94.2 Default bounds (imperial, occupied)

| Sensor                     | Min     | Max                    | Flatline tol | Max $\Delta$ /hr |
|----------------------------|---------|------------------------|--------------|------------------|
| Zone temp                  | 55 °F   | 90 °F                  | 0.10 °F      | 4 °F             |
| Supply air temp            | 50 °F   | 110 °F                 | 0.15 °F      | 8 °F             |
| Return air temp            | 55 °F   | 95 °F                  | 0.10 °F      | 3 °F             |
| Duct static                | −0.5    | 3.0 inH <sub>2</sub> O | 0.02         | 0.5              |
| RH                         | 0 %     | 100 %                  | 1.0 %        | 15 %             |
| CHW                        | 40 °F   | 90 °F                  | 0.10 °F      | 4 °F             |
| HW                         | 70 °F   | 200 °F                 | 0.15 °F      | 6 °F             |
| Condenser water            | 50 °F   | 110 °F                 | 0.15 °F      | 5 °F             |
| CO <sub>2</sub> (occupied) | 400 ppm | 1000 ppm               | 5 ppm        | 200 ppm          |

Return-air checks use **narrow bands**; when MAT/OAT/RAT are available, prefer **mixing envelope** over RAT bounds alone.

## 94.3 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use stale\_points for dropout detection (**BLD-D**).
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

## 94.4 Operator workflow

1. Confirm **BLD-D** (stale) is resolved before trusting other sensor codes.
2. Cross-check **BLD-B** OAT against weather service or JSON API driver.
3. For **VAV-C** / **AHU-C**, compare neighbor zones / coil behavior before replacing sensor.

# 95 Logging and audit

## 96 Logging and audit

Open-FDD uses a **two-tier logging model** familiar to IT and security teams: structured **audit/error JSONL** for forensics and compliance, plus **service stdout** for engineering troubleshooting. Defaults include **age pruning**, **size rotation in MB**, and **Docker log caps** — no operator cron required.

### 96.1 Log types

| Tier                  | Location                                                         | Format                        | Who reads it                                     |
|-----------------------|------------------------------------------------------------------|-------------------------------|--------------------------------------------------|
| <b>Audit</b>          | workspace/logs/audit.jsonl                                       | JSON Lines                    | Security, MSI, integrator API                    |
| <b>Errors</b>         | workspace/logs/error.jsonl                                       | JSON Lines                    | Support, integrator API                          |
| <b>Service stdout</b> | Docker: docker compose logs ·<br>Dev: workspace/.local-run/*.log | Plain text (uvicorn, workers) | Engineering                                      |
| <b>Historian</b>      | workspace/data/feather_store/                                    | Feather                       | FDD, plots — separate retention (data.env.local) |

Structured audit logs are **not** mixed into uvicorn access logs. That separation matches common practice: SIEM-friendly JSONL for auth and OT commands; unstructured stdout for stack traces and poll worker noise.

### 96.2 Audit record schema

Each line in `audit.jsonl` / `error.jsonl` is one JSON object:

| Field      | Example                   | Notes                         |
|------------|---------------------------|-------------------------------|
| @timestamp | 2026-06-07T19:57:21+00:00 | UTC ISO-8601                  |
| event_id   | UUID                      | Unique per event              |
| event_type | auth.login.failure        | Stable taxonomy (see below)   |
| severity   | warning                   | debug ... critical            |
| outcome    | failure                   | success, failure, denied, ... |

| Field      | Example                         | Notes                                                          |
|------------|---------------------------------|----------------------------------------------------------------|
| service    | openfdd-bridge                  |                                                                |
| host       | hostname                        |                                                                |
| action     | login                           | Human-readable verb                                            |
| actor      | {username, role}                | After successful auth                                          |
| client     | {ip, user_agent}                | Respects<br>OFDD_TRUST_X_FORWARDED_FOR<br>behind Caddy         |
| http       | method, path, status            | When HTTP-triggered                                            |
| resource   | BACnet object, model id,<br>... | When applicable                                                |
| detail     | Sanitized map                   | <b>Passwords, tokens,<br/>secrets stripped by key<br/>name</b> |
| request_id | UUID                            | On audited HTTP paths                                          |

### 96.2.1 Auth events (industry-standard login trail)

| event_type         | When                                                   |
|--------------------|--------------------------------------------------------|
| auth.login.success | Valid credentials                                      |
| auth.login.failure | Bad password<br>(generic client<br>message)            |
| auth.login.lockout | Rate limit<br>exceeded (default<br>5 failures / 5 min) |

Login rate limits and lockouts are **audited**; responses never reveal whether the username exists.

### 96.2.2 OT / commissioning events

| event_type      | When                                                |
|-----------------|-----------------------------------------------------|
| bacnet.command  | Supervisory write (allowed, denied, dry-run)        |
| bacnet.discover | Who-Is / point discovery                            |
| model.write     | BRICK model mutations                               |
| agent.tool      | Agent tool calls (prompts hashed/truncated)         |
| api.access      | Sensitive POST/PUT/PATCH/DELETE (BACnet,<br>ingest) |
| error.unhandled | Unhandled exception (also in error.jsonl)           |

**Not audited:** routine GETs (dashboard reads, plot data, health). This reduces noise while keeping **auth** and **field commands** on the trail.

### 96.2.3 Secret redaction

Never logged: passwords, bearer tokens, `OFDD_AUTH_SECRET`, private keys. Detail objects drop keys containing password, token, secret, authorization. Agent tool arguments use additional hashing for prompts and rule source.

Opt-in full prompt logging (lab only): `OFDD_AUDIT_LOG_PROMPTS=1`.

## 96.3 Integrator API (read logs in the UI stack)

| Method | Path                                     | Role       |
|--------|------------------------------------------|------------|
| GET    | <code>/api/audit/events?limit=100</code> | integrator |
| GET    | <code>/api/audit/errors?limit=100</code> | integrator |
| GET    | <code>/api/audit/summary</code>          | integrator |

Use these for post-incident review without shell access. Forward `workspace/logs/*.jsonl` to SIEM (rsyslog, Filebeat, Wazuh) on production edges.

## 96.4 Retention and rotation (defaults)

Configured in `workspace/data.env.local` (copy from `data.env.example`):

| Variable                               | Default   | Behavior                                                                                                                            |
|----------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>OFDD_LOG_RETENTION_DAYS</code>   | <b>90</b> | Drop audit/error JSONL lines older than N days; delete aged <code>.local-run</code> and archived <code>*.log / audit/*.jsonl</code> |
| <code>OFDD_LOG_MAX_MB</code>           | <b>50</b> | When <code>audit.jsonl</code> or <code>error.jsonl</code> exceeds N MB, archive to <code>audit.{UTC}.jsonl</code> and start fresh   |
| <code>OFDD_LOCAL_RUN_LOG_MAX_MB</code> | <b>25</b> |                                                                                                                                     |



| Variable                       | Default | Behavior                                                                                                                            |
|--------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| OFDD_LOG_ROTATE_INTERVAL_HOURS | 6       | Rotate workspace/.local-run/*.log (bridge, Caddy, FDD loop, ...) Background retention while bridge is running (not only at restart) |

Rotation runs at **bridge startup** and on the **scheduled interval**. No logrotate.d entry is required for audit JSONL.

Optional path overrides:

```
OFDD_AUDIT_LOG_PATH=/var/log/openfdd/audit.jsonl
OFDD_ERROR_LOG_PATH=/var/log/openfdd/error.jsonl
```

## 96.5 Docker edge: json-file caps (not journald for app audit)

Production stacks use **Docker Compose** (docker/compose.edge.yml). Container stdout uses the **json-file driver with size limits**:

```
logging:
  driver: json-file
  options:
    max-size: "25m"
    max-file: "5"
```

That caps each service at roughly **125 MB** of rotated container logs on disk — the same pattern many teams use instead of unbounded docker logs.

### Why audit JSONL stays on disk (not journald)?

- Append-only files under workspace/logs/ survive container recreate and bind-mount to the host.
- JSONL lines import cleanly into SIEM without journal field parsing.
- OT incident response often needs **months** of auth/BACnet write history on the edge box.

**When journald still applies:** native systemd units (Caddy, Ollama on bare metal) — use `journalctl -u <unit>` per your Linux runbook. Bridge audit remains in `workspace/logs/` even when uvicorn stdout goes to Docker or `.local-run/`.

## 96.6 Local dev (run\_local.sh)

| File                                | Service                 |
|-------------------------------------|-------------------------|
| workspace/.local-run/bridge.log     | Bridge / uvicorn        |
| workspace/.local-run/commission.log | BACnet commission agent |
| workspace/.local-run/caddy.log      | Caddy                   |
| workspace/.local-run/fdd_loop.log   | Scheduled FDD           |
| workspace/.local-run/mcp_rag.log    | MCP RAG                 |
| workspace/.local-run/ollama.log     | Ollama                  |

These are **engineering logs**. Structured audit still lands in `workspace/logs/audit.jsonl` when auth is enabled.

## 96.7 Quick checks

```
# Last auth failures
grep 'auth.login' workspace/logs/audit.jsonl | tail -5

# Integrator API (after login)
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8765/
api/audit/summary | jq .

# Docker edge
docker compose logs bridge --tail 50
```

## 96.8 Comparison to typical web apps

| Practice                        | Open-FDD                                  |
|---------------------------------|-------------------------------------------|
| Structured auth audit           | Yes — JSONL with actor, IP, outcome       |
| Login rate limit + logout audit | Yes                                       |
| Secret redaction in logs        | Yes — key-based + agent sanitization      |
| Request correlation ID          | Yes — on audited mutation paths           |
| Centralized stdout JSON         | No — stdout is plain text; audit is JSONL |

| Practice                 | Open-FDD                                          |
|--------------------------|---------------------------------------------------|
| Full HTTP access log     | No — mutations and auth only (OT noise reduction) |
| Log rotation / size caps | Yes — defaults above + Docker max-size            |
| SIEM-ready export        | Yes — file tail / Filebeat on workspace/logs/     |

For BACnet write safety and credential generation, see [SECURITY.md](#) and [Secrets and auth](#).

## 96.9 Related

- [JSON API driver](#) — OpenWeatherMap showcase, env-based API keys (json\_api.env.local)
- [Live site update](#) — docker compose logs after upgrades
- [Data flow](#) — historian retention (OFDD\_FEATHER\_\*) is separate from audit logs

# 97 Fault Codes

## 98 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

| Page                                      | Content                      |
|-------------------------------------------|------------------------------|
| <a href="#">Convention</a>                | Naming, severity, categories |
| <a href="#">AHU faults</a>                | Air handling units           |
| <a href="#">VAV faults</a>                | Terminal units               |
| <a href="#">RTU faults</a>                | Packaged rooftops            |
| <a href="#">Sensor &amp; data quality</a> | Stale, flatline, comms       |

Live catalog API: GET /api/faults/catalog on the Operator Bridge.

# 99 Operations

# 100 Operations

Runbooks for live edge hosts after the initial [Quick Start](#) deploy.

| Page                                  | When to use                                 |
|---------------------------------------|---------------------------------------------|
| <a href="#">Live site update</a>      | Pull new GHCR tags, preserve workspace/     |
| <a href="#">Backup and restore</a>    | Archive workspace/ before upgrades          |
| <a href="#">Logging and audit</a>     | Auth audit trail, rotation, Docker log caps |
| <a href="#">Deployment validation</a> | Post-upgrade smoke and insurance checks     |

Site-specific lab notes (example BACnet scope, GL36 rules):  
[Examples & lab notes](#).

For first-time deploy: [Quick Start — Docker](#).

## 101 LAN hardening

## 102 LAN hardening

Open-FDD targets **trusted building LAN / OT edge** deployment. The Operator Bridge is not intended for direct public internet exposure.

Full mode matrix: [Deployment modes](#).

### 102.1 Auth and bind

| Mode      | Bridge bind                    | Auth                                              |
|-----------|--------------------------------|---------------------------------------------------|
| Local dev | 127.0.0.1                      | Optional<br>OFDD_AUTH_DISABLED=1<br>on loopback   |
| Lab LAN   | 0.0.0.0                        | Required unless<br>explicit insecure<br>lab flags |
|           | Loopback + <b>Caddy</b> on LAN |                                                   |

| Mode               | Bridge bind | Auth                                                      |
|--------------------|-------------|-----------------------------------------------------------|
| Edge<br>production |             | <b>Required</b> —<br>OFDD_AUTH_SECRET +<br>role passwords |

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are set).

## 102.2 Network exposure

- Put **Caddy** on :80 / :443; keep bridge :8765 on loopback when possible.
- Do not port-forward the Operator Bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; management UI on IT VLAN with firewall rules.
- BACnet **writes** are gated by default — [BACnet write guard](#).

## 102.3 Rule Lab

Rule Lab runs **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

## 102.4 TLS and LAN security scans

| Topic                                | Page                                                                       |
|--------------------------------------|----------------------------------------------------------------------------|
| Certificates (self-signed / site CA) | <a href="#">TLS and certificates</a>                                       |
| ZAP + Nmap bench smoke               | <a href="#">ZAP baseline</a> · <a href="#">scripts/security/</a>           |
| Ubuntu host + Tenable/Nessus         | <a href="#">Linux host hardening</a> · <a href="#">Tenable remediation</a> |
| Release test cycle (maintainers)     | <a href="#">Security testing</a>                                           |

## 102.5 Ollama on the edge

Ollama has **no built-in auth**. Bind OLLAMA\_HOST=127.0.0.1:11434 on the host; never expose :11434 on the OT LAN. Open-FDD bridge reaches Ollama server-side only. Validation: [scripts/security/check\\_openfdd\\_exposure.sh](#).

# 103 ZAP baseline (local bench)

# 104 ZAP baseline (local bench)

Passive OWASP ZAP baseline against the **pentest production stack** on benser server is the standard LAN security smoke before Acme edge deploys.

Full per-revision workflow (pytest → PR → GHCR → Acme):  
Developer — security testing cycle.

Packaged scan scripts (Windows + Mac/Linux): scripts/security/README.md.

## 104.1 Run

**On the Open-FDD host:**

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

**From a LAN workstation** (after verify passes):

```
# Windows
.\scripts\security\Run-OpenFddSecurityScan.ps1

# macOS / Linux
./scripts/security/run_openfdd_security_scan.sh --url http://
192.168.204.18
```

ZAP target (example): `http://192.168.204.18/` — Caddy on `:80` only; bridge `:8765` stays loopback. TLS bench: see TLS and certificates.

## 104.2 Expected results (HTTP bench mode)

| Finding                       | Severity | Status                                                         |
|-------------------------------|----------|----------------------------------------------------------------|
| CSP style-src 'unsafe-inline' | Medium   | <b>Accepted</b> — Vite/React inline styles; TODO to nonce/hash |

| Finding                                     | Severity | Status                                                                |
|---------------------------------------------|----------|-----------------------------------------------------------------------|
| Missing COEP                                | Low      | <b>Accepted</b> — require-corp not enabled (breaks some assets)       |
| Port 80 open (Caddy)                        | Info     | <b>Expected</b> — intentional LAN entry                               |
| Ports 8765, 5173, 8000, 8080 closed         | —        | <b>Good</b>                                                           |
| Port 443 filtered                           | —        | <b>Expected</b> in HTTP mode; use OFDD_CADDY_MODE=tls for HTTPS bench |
| Duplicate Referrer-Policy / X-Frame-Options | Low      | <b>Fixed</b> — bridge owns headers; Caddy does not duplicate          |

## 104.3 Header ownership

| Layer                               | Sets                                                                                                                        |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Bridge</b> (security_headers.py) | CSP, X-Frame-Options: DENY, frame-ancestors 'none', COOP, CORP, Permissions-Policy, Referrer-Policy, X-Content-Type-Options |
| <b>Caddy HTTP</b>                   | Reverse proxy only; strips Server banners                                                                                   |
| <b>Caddy TLS</b>                    | Adds Strict-Transport-Security only                                                                                         |

## 104.4 Authenticated scan (later)

Baseline scan hits ~4 public endpoints (/health, public agent insight routes). Deep API/dashboard coverage requires ZAP with integrator login — see [Authenticated scanning \(roadmap\)](#).

## 104.5 After fixes

Re-run pentest\_production\_stack.sh verify, then the packaged LAN scan from scripts/security/. Confirm response headers show **one** X-Frame-Options: DENY and no duplicate Referrer-Policy.

# 105 TLS and certificates

## 106 TLS and certificates

Who generates, stores, and renews TLS material for Open-FDD on OT LANs — and how that ties into ZAP/Nmap bench testing.

Developer release cycle: [Security testing cycle](#).

### 106.1 Ownership

| Environment                         | Who owns certs         | Storage                                                                     |
|-------------------------------------|------------------------|-----------------------------------------------------------------------------|
| Local / benserver bench             | Developer / maintainer | workspace/<br>deploy/caddy/<br>certs/<br>(cert.pem,<br>key.pem)             |
| Ansible edge (Acme, customer sites) | Site integrator        | /etc/openfdd/<br>caddy/ on the<br>VM<br>(playbook-<br>managed or<br>manual) |
| Public internet                     | <b>Not in scope</b>    | Open-FDD<br>targets OT<br>LAN; use site<br>PKI or self-<br>signed           |

Open-FDD does **not** ship Let's Encrypt or public CA automation. OT benches use **self-signed** or customer-provided certs.

### 106.2 Generate bench certs (local)

```
./scripts/setup_caddy_certs.sh  
# or: ./scripts/setup_caddy_certs.sh --cn openfdd.local --out  
workspace/deploy/caddy/certs
```

Then start with TLS mode:

```
OFDD_CADDY_MODE=tls ./scripts/run_local.sh  
# or pentest stack with OFDD_CADDY_MODE=tls
```



Caddy serves :443 and redirects :80 → HTTPS. Caddy adds **Strict-Transport-Security** only; all other security headers remain on the **bridge** (ZAP baseline).

## 106.3 ZAP / Nmap with TLS bench

When `OFDD_CADDY_MODE=tls`:

- Nmap: expect **443 open**, 80 may redirect
- ZAP: use `https://<lan-ip>/` (add `-k trust` or import `cert.pem` on the scan workstation)
- PowerShell: `.\Run-OpenFddSecurityScan.ps1 -TargetUrl "https://192.168.204.18"`
- Bash: `./run_openfdd_security_scan.sh --url https://192.168.204.18`

Self-signed certs produce ZAP TLS warnings — expected on OT LAN.

## 106.4 Renewal

Self-signed certs from `setup_caddy_certs.sh` default to **365 days**. Re-run the script before expiry or bake renewal into site runbooks.

Edge Ansible deploys should document whether the integrator rotates certs manually or via site PKI.

## 106.5 Enterprise CA on Caddy (edge)

When IT requires **clean Nessus SSL plugins** (no self-signed / untrusted chain), install a site-provided certificate on the edge VM instead of `setup_caddy_certs.sh` output.

Typical layout (Ansible / manual):

```
/etc/openfdd/caddy/cert.pem # full chain (leaf + intermediates)
/etc/openfdd/caddy/key.pem  # private key (0600, root or caddy
user)
```

Ansible TLS mode (`caddy_mode: tls`) already points Caddy at those paths — see `infra/ansible/templates/Caddyfile.j2`.

### Internal enterprise CA workflow:

1. Generate CSR on the edge host or request cert from site PKI for the operator hostname (e.g. `openfdd.acme.local`).
2. Install `cert.pem` + `key.pem` under `/etc/openfdd/caddy/`.

3. Distribute the **CA root** to admin laptops (trust store) so browsers and ZAP scans validate the chain.
4. Monitor expiry:

```
openssl x509 -in /etc/openfdd/caddy/cert.pem -noout -dates
```

### **Caddyfile snippet (TLS, enterprise cert):**

```
:443 {  
    tls /etc/openfdd/caddy/cert.pem /etc/openfdd/caddy/key.pem  
    header {  
        Strict-Transport-Security "max-age=31536000;  
includeSubDomains"  
    }  
    reverse_proxy 127.0.0.1:8765  
}
```

Lab/self-signed mode still produces expected **Medium** Tenable findings — document as accepted risk or switch to enterprise CA. See [Tenable remediation](#).

## **106.6 Caddy internal CA mode (lab)**

`./scripts/setup_caddy_certs.sh` generates a **self-signed** pair for bench TLS. Trust `cert.pem` on scan workstations (`-k curl`, ZAP import) or accept browser warnings. Not suitable when IT mandates trusted chains.

## **106.7 Secrets**

- **Never commit** `key.pem` or production cert bundles
- `workspace/deploy/caddy/certs/` is for local bench only unless your `.gitignore` already excludes generated keys (verify before push)

# 107 Authenticated scanning (roadmap)

# 108 Authenticated scanning (roadmap)

ZAP baseline is **unauthenticated** — it checks public pages, response headers, and a handful of routes reachable without login. That is the right default for every docker image / patch revision smoke on a test LAN.

For **deep** coverage of integrator dashboards, /api/\* routes, WebSocket sessions, and role-gated writes, plan a separate **authenticated** scan — only on fake/bench stacks with explicit approval.

Release context: [Security testing cycle](#).

## 108.1 Current state (3.0.x)

| Capability                                         | Status                                  |
|----------------------------------------------------|-----------------------------------------|
| ZAP baseline via scripts/security/                 | <b>Shipped</b>                          |
| Pentest creds in workspace/<br>auth.pentest.local  | <b>Shipped</b> (host-local, not in git) |
| ZAP full / active scan automation                  | <b>Not shipped</b> — manual only        |
| ZAP authenticated context (login script + session) | <b>Roadmap</b>                          |
| CI running ZAP against PR previews                 | <b>Not planned</b> (LAN bench only)     |

## 108.2 Why baseline is not enough

After login, the bridge exposes model editing, BACnet/JSON API configuration, Rule Lab, commissioning export, and write-gated commands. Baseline ZAP never exercises:

- Session cookies / JWT persistence
- CSRF or auth bypass on mutating endpoints
- IDOR on site-scoped resources
- Rate limits and audit logging on failed auth

## 108.3 Planned approach (contributors welcome)

1. **Bench-only target** — same pentest stack; never point active scans at live OT controllers
2. **Login hook** — ZAP zap-full-scan.py or Automation Framework with:
  - POST /api/auth/login using integrator creds from env (not committed)
  - Regex or JSON extractor for session token / cookie
3. **Scope file** — include /api/model/\*, /api/commissioning/\*, dashboard static assets; exclude BACnet UDP and unrelated LAN hosts
4. **Report parity** — extend scripts/security/ with opt-in - Authenticated / --zap-full flags and separate report prefix (22-zap-authenticated-\*)
5. **Gate** — document accepted findings; fail release only on High/Critical regressions

## 108.4 Manual authenticated ZAP today

1. Start pentest stack; note creds in workspace/auth.pentest.local
2. Open ZAP desktop or docker with API port exposed
3. Configure **Context** → **Authentication** → script or form-based login to /api/auth/login
4. Run **Spider** then **Active Scan** on http://<lan-ip>/ only
5. Compare with baseline report in openfdd-security-report/

Do **not** run active scans against production Acme without a maintenance window and BACnet write lock.

## 108.5 Related

- [scripts/security/README.md](#) — baseline setup
- [Secrets and auth](#) — env file layout
- [LAN hardening](#) — bind and auth modes

# 109 Secrets and auth

## 110 Secrets and auth

### 110.1 Files (gitignored)

| File                                       | Contents                                                                                              |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| workspace/<br>auth.env.local               | JWT secret, role passwords                                                                            |
| workspace/<br>json_api.env.local           | REST API keys for JSON API <code>\${ENV:VAR}</code><br>URLs (e.g. OpenWeather<br>OPENWEATHER_API_KEY) |
| workspace/<br>data.env.local               | Historian + <b>audit log retention</b><br>(OFDD_LOG_*, OFDD_FEATHER_*)                                |
| workspace/<br>ollama.env.local             | Ollama URL, model, RAM tier, GPU<br>mode                                                              |
| workspace/<br>mcp.env.local                | MCP RAG sidecar                                                                                       |
| workspace/<br>caddy.env.local              | Reverse proxy TLS/HTTP mode                                                                           |
| workspace/<br>pentest.env.local            | LAN security scan stack                                                                               |
| infra/ansible/secrets/<br><host>.env.local | SSH deploy secrets (maintainers)                                                                      |

Copy from \*.example templates only. Full inventory: [Workspace env files](#).

### 110.2 Required variables (production)

| Variable                        | Purpose                  |
|---------------------------------|--------------------------|
| OFDD_AUTH_SECRET                | HMAC signing (32+ chars) |
| OFDD_INTEGRATOR_USER / PASSWORD | Rule Lab, bindings       |
| OFDD_OPERATOR_USER / PASSWORD   | Operations               |

### 110.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

## 110.4 Backup

Back up workspace/data/ and auth env in secure operator vault — not in git.

Audit JSONL under workspace/logs/ may contain client IPs and usernames — treat like security logs. See [Logging and audit](#).

## 111 BACnet write allowlists

## 112 BACnet write allowlists

Supervisory writes require:

1. **Feature flag** enabled in environment (off by default).
2. **Allowlist** file at workspace/bacnet/write\_allowlist.json (copy from [write\\_allowlist.example.json](#)). If the flag is on but the allowlist is missing, writes are **denied**.
3. **Integrator** role on API calls.
4. **Audit** log review after commissioning sessions.

Allowlist entries support device\_instance, object\_identifier, property\_identifier, optional priority\_min/priority\_max, value\_min/value\_max, and allowed\_values for binary/multistate targets.

Lab-only override: OFDD\_BACNET\_WRITE\_ALLOW\_ANY=1 (audited; not for production edges).

See operator guide: [BACnet write safety](#).

Never enable blanket write access for agent or integrator automation without human approval per site.

## 113 Linux host hardening

## 114 Linux host hardening

Generic **Ubuntu edge VM** guidance for Open-FDD deployments scanned by IT tools (Tenable/Nessus, Qualys, etc.). This complements application-level [LAN hardening](#) and [TLS](#).

Nessus-specific finding tables: [Tenable remediation](#).

## 114.1 Scope

| Layer                                | Owner                       | Repo support                                        |
|--------------------------------------|-----------------------------|-----------------------------------------------------|
| Ubuntu kernel, OpenSSH, apt packages | <b>Site IT / integrator</b> | scripts/security/remediate_ubuntu_host.sh           |
| Docker engine, compose, Caddy        | <b>Integrator</b>           | Ansible + check_openfdd_exposure.sh                 |
| Open-FDD containers (pip/pyOpenSSL)  | <b>Open-FDD maintainers</b> | docker/python-security-constraints.txt, GHCR images |
| Self-signed TLS on OT LAN            | <b>Expected lab finding</b> | Enterprise CA path in TLS                           |

## 114.2 Quick operator workflow

On the edge VM (SSH):

```
cd ~/open-fdd
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip "$
(hostname -I | awk '{print $1}')
```

Remediate host OS (maintenance window):

```
./scripts/security/remediate_ubuntu_host.sh # dry-run
sudo ./scripts/security/remediate_ubuntu_host.sh --apply
# if /var/run/reboot-required exists:
sudo reboot
./scripts/security/check_host_security.sh
```

Pull new Open-FDD images after a release that pins container pip/pyOpenSSL:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
```

## 114.3 Ubuntu version and EOL

Supported targets: **Ubuntu 22.04 LTS** or **24.04 LTS** (64-bit).  
Rebuild hosts still on **16.04** or **18.04** — apt upgrade cannot clear an SEoL finding.

Validate a surprising 16.04 Nessus hit:

```
lsb_release -a
cat /etc/os-release
# Scan credential / stale DNS / container banner mismatch?
```

## 114.4 Kernel and package updates

```
sudo apt update
sudo apt full-upgrade -y
uname -r
test -f /var/run/reboot-required && echo "reboot required"
sudo reboot # maintenance window
```

Optional status tools:

```
ubuntu-security-status # or: pro status
apt list --upgradable | grep -i security
```

## 114.5 Host Python (pip, wheel, pyOpenSSL)

Tenable often reports **host** pip, wheel, and pyOpenSSL versions installed by Ubuntu packages or admin pip install. Prefer **apt** on the host:

```
python3 -m pip show pip setuptools wheel pyOpenSSL
sudo apt install --only-upgrade python3-pip python3-openssl
python3-wheel
```

Open-FDD **containers** pin toolchain versions at image build — see docker/python-security-constraints.txt. Rescan after docker compose pull.

## 114.6 Ollama — no LAN exposure

Ollama has **no built-in authentication** in typical deployments. Treat LAN exposure as **Critical**.



| Rule                             | Implementation                                                                            |
|----------------------------------|-------------------------------------------------------------------------------------------|
| Bind loopback only               | OLLAMA_HOST=127.0.0.1:11434 in systemd (ollama.service) or dev bootstrap                  |
| Bridge calls server-side         | OFDD_OLLAMA_BASE_URL=http://127.0.0.1:11434 or host.docker.internal:11434 from containers |
| Never publish : 11434 on 0.0.0.0 | Compose uses 127.0.0.1:11434:11434 when Docker Ollama is enabled                          |

Validation:

```
ss -tulpn | grep 11434
curl -sf http://127.0.0.1:11434/api/tags | head
curl -sf --max-time 3 http://<edge-lan-ip>:11434/api/tags
# must FAIL from another host
```

Firewall (if management host needs GPU Ollama — rare):

```
# Example: block LAN → 11434; allow loopback only (adjust
interface names)
sudo ufw deny 11434/tcp
sudo ufw allow from 127.0.0.0/8 to any port 11434 proto tcp
```

## 114.7 OpenSSH Terrapin (CVE-2023-48795)

Apply Ubuntu security updates first (openssh-server, openssh-client). Verify:

```
ssh -V
sudo sshd -T | grep -Ei '^(ciphers|macs|kexalgorithms)'
```

Manual algorithm hardening is **optional** and can lock out clients — document rollback (sshd\_config.bak) before changes. Do not automate cipher stripping in repo scripts.

## 114.8 ICMP timestamp (Low)

Nessus may report ICMP timestamp disclosure. Optional hardening at host firewall or upstream network — low priority for isolated OT VLANs.

Example (nftables — site-specific):

```
# Block ICMP timestamp / type 13 – verify with IT before applying
sudo nft add rule inet filter input icmp type timestamp-request
drop
```

# 114.9 TLS and certificate scans

Self-signed or internal CA certificates produce **Medium** Nessus SSL findings (untrusted, self-signed, expiry). That is **expected** for lab/OT encryption-in-transit. For clean SSL plugin results, install a **site enterprise CA** certificate on Caddy — [TLS and certificates](#).

# 114.10 Maintainer security audits (CI parity)

```
./scripts/security/run_maintainer_audits.sh
```

Matches GitHub Actions operator-bridge-security job where possible.

# 115 Tenable / Nessus remediation

# 116 Tenable / Nessus remediation

Remediation plan for **IT/Tenable.io Nessus** findings on an Open-FDD **OT/LAN edge** deployment (e.g. Acme GL36 lab VM). Open-FDD runs behind a building firewall on Ubuntu with Docker Compose, Caddy, and optional host Ollama.

Related: [Linux host hardening](#), [TLS and certificates](#), [LAN hardening](#), [Security testing cycle](#).

## 116.1 Prioritized action plan

| Priority | Action                                                               | Owner           |
|----------|----------------------------------------------------------------------|-----------------|
| P0       | Confirm Ubuntu is <b>not</b> 16.04/18.04; rebuild if EOL             | IT / integrator |
| P0       | Bind Ollama to <b>127.0.0.1</b> only; verify LAN cannot reach :11434 | Integrator      |
| P0       |                                                                      | IT / integrator |

| Priority  | Action                                                                | Owner           |
|-----------|-----------------------------------------------------------------------|-----------------|
|           | apt full-upgrade + <b>reboot</b> for kernel USNs (8254, 8278, 8373)   |                 |
| <b>P1</b> | Upgrade host pip/wheel/pyOpenSSL via apt or documented pip path       | IT / integrator |
| <b>P1</b> | Pull GHCR images built with docker/python-security-constraints.txt    | Integrator      |
| <b>P2</b> | OpenSSH packages current; review Terrapin after patch                 | IT              |
| <b>P2</b> | TLS: accept self-signed findings <b>or</b> install enterprise CA cert | IT decision     |
| <b>P3</b> | ICMP timestamp: optional firewall rule                                | Network / IT    |

## 116.2 Finding classification table

| Nessus finding                            | Severity | Class                            | Remediation                                                                                                                            |
|-------------------------------------------|----------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Canonical Ubuntu Linux SEoL (16.04.x)     | Critical | Host OS / <b>false positive?</b> | lsb_release -a — rebuild VM on 24.04 LTS if truly 16.04; if host is 22.04/24.04, check scan target IP, credentials, stale asset record |
| Ollama Unauthenticated Access             | Critical | Deployment config                | OLLAMA_HOST=127.0.0.1:11434; no 0.0.0.0:11434; firewall deny; see <a href="#">Ollama</a>                                               |
| Ubuntu kernel USN-8254-1                  | Critical | Host OS                          | sudo apt update && sudo apt full-upgrade -y && sudo reboot                                                                             |
| pyOpenSSL 22.0.x < 26.0.0 Buffer Overflow | Critical | Host pip <b>and</b> container    | Host: apt upgrade python3-openssl or pip ≥26; Container: GHCR image with constraints file                                              |
| Ubuntu pip USN-8344-1                     | High     |                                  | apt full-upgrade; image build upgrades pip>=26                                                                                         |

| Nessus finding                       | Severity | Class                                   | Remediation                                                 |
|--------------------------------------|----------|-----------------------------------------|-------------------------------------------------------------|
|                                      |          | Host OS + container build               |                                                             |
| SSL Certificate Cannot Be Trusted    | Medium   | <b>Accepted risk</b> (lab) / deployment | Self-signed Caddy — use enterprise CA for clean scan        |
| SSL Self-Signed Certificate          | Medium   | <b>Accepted risk</b> (lab)              | Same as above                                               |
| SSL Certificate Expiry               | Medium   | Deployment / monitoring                 | openssl x509 -dates; renew setup_caddy_certs.sh or site PKI |
| SSH Terrapin (CVE-2023-48795)        | Medium   | Host OS                                 | Upgrade openssh-server; validate ssh -V                     |
| Ubuntu wheel USN-8221-1              | Medium   | Host OS                                 | apt full-upgrade                                            |
| Ubuntu kernel USN-8278-1, USN-8373-1 | Medium   | Host OS                                 | apt full-upgrade + reboot                                   |
| pyOpenSSL 0.14.x Security Bypass     | Medium   | Host + container                        | Same as pyOpenSSL Critical — upgrade to $\geq 26$           |
| ICMP Timestamp Request               | Low      | Network / optional host                 | Firewall drop; often accepted on isolated OT VLAN           |

**Class legend:** (1) repo-owned fix · (2) Docker base/image · (3) host OS · (4) deployment/config · (5) accepted risk · (6) validate false positive

## 116.3 Operator commands (edge VM)

### 116.3.1 Validate (read-only)

```

cd ~/open-fdd
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip "$
(hostname -I | awk '{print $1}')"
lsb_release -a
uname -r
test -f /var/run/reboot-required && cat /var/run/reboot-
required.pkgs
python3 -m pip show pip setuptools wheel pyOpenSSL 2>/dev/null |
grep -E '^(Name|Version):'
ss -tulpn | grep -E ':80|:443|:8765|:11434'
curl -sf http://127.0.0.1:11434/api/tags | head -3

```

From a **different LAN workstation** (must fail):

```
curl -sf --max-time 3 http://<edge-ip>:11434/api/tags
```

### 116.3.2 Remediate host (maintenance window)

```
./scripts/security/remediate_ubuntu_host.sh # dry-run
./scripts/security/remediate_ubuntu_host.sh --apply
sudo reboot # if reboot-required
./scripts/security/check_host_security.sh
```

### 116.3.3 Remediate Open-FDD containers

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
docker compose pull && docker compose up -d
```

### 116.3.4 Rescan

After reboot and image pull, request IT **Nessus rescan** of the edge IP. Compare plugin output to the table above.

## 116.4 Ollama unauthenticated access (Critical)

**Not an Open-FDD code bug** — Ollama's API is unauthenticated by design. Risk is **network exposure**.

Repo defaults (3.0.5+):

- Systemd: Environment=OLLAMA\_HOST=127.0.0.1:11434 in infra/ansible/templates/ollama.service.j2
- Compose Ollama (optional profile): 127.0.0.1:11434:11434 only
- Acme-style edge: host Ollama + bridge via host.docker.internal:11434 (container → host loopback path)
- post\_deploy\_check.sh warns if :11434 listens on 0.0.0.0

Re-apply Ansible Ollama unit after upgrade:

```
ansible-playbook -i inventory.yml ollama_bootstrap.yml --limit
<host> -e enable_ollama=true
```

# 116.5 pyOpenSSL and pip in containers

Docker build installs `docker/python-security-constraints.txt` before application requirements:

- `pip>=26.0, setuptools>=75.0, wheel>=0.45`
- `pyOpenSSL>=26.0.0, cryptography>=44.0.0`

CI `pip-audit` runs against `bridge` + `bacnet` requirements with the same constraints.

# 116.6 TLS — expected vs production

| Mode                              | Nessus SSL plugins                         | When to use                 |
|-----------------------------------|--------------------------------------------|-----------------------------|
| Caddy<br>HTTP :80                 | No TLS plugins                             | Dev / HTTP-only<br>VLAN     |
| Caddy<br>self-<br>signed :<br>443 | Medium: untrusted, self-<br>signed, expiry | OT LAN encryption,<br>lab   |
| Enterprise<br>CA cert<br>on Caddy | Clean SSL plugins (if chain<br>trusted)    | IT mandates clean<br>Nessus |

See [TLS — enterprise CA](#).

# 116.7 Residual findings after remediation

Even with a patched host and loopback-only Ollama, IT may still see:

- **SSL** Medium on self-signed TLS (accepted for lab; document in risk register)
- **ICMP** Low (optional network block)
- **Container** plugins if Nessus scans Docker overlay networks — clarify scan scope with IT

# 116.8 Notes for IT / security team

1. Scan the **edge VM IP**, not individual container IPs, unless container CVE correlation is required.
2. Open-FDD **bridge :8765** should be **loopback-only**; LAN entry is **Caddy :80/443**.

- 3. **BACnet UDP 47808** on commission is expected OT traffic — out of scope for web/plugin remediation.
- 4. Do not expose Ollama **11434** to the building LAN; agent chat is server-side only.
- 5. Kernel CVEs require **host reboot** — schedule with operations.
- 6. For authenticated web/API depth, see roadmap Authenticated scanning.

## 116.9 Repo-owned fixes (this document’s release)

| Item                             | Location                                                     |
|----------------------------------|--------------------------------------------------------------|
| Container pip/<br>pyOpenSSL pins | docker/python-security-constraints.txt,<br>docker/Dockerfile |
| Ollama loopback<br>systemd       | infra/ansible/templates/<br>ollama.service.j2                |
| Host/exposure check<br>scripts   | scripts/security/check_*.sh,<br>remediate_ubuntu_host.sh     |
| Post-deploy Ollama<br>bind warn  | infra/ansible/scripts/<br>post_deploy_check.sh               |
| CI pip-audit +<br>constraints    | .github/workflows/ci.yml                                     |

## 117 Security

## 118 Security

LAN deployment security for the Operator Bridge and BACnet write guard. Open-FDD is for **trusted edge / OT LAN** — not direct public internet exposure.

| Page                                | Topic                                       |
|-------------------------------------|---------------------------------------------|
| <u>LAN hardening</u>                | Auth and bind modes                         |
| <u>ZAP baseline (local bench)</u>   | Pentest stack, expected warnings            |
| <u>TLS and certificates</u>         | Cert ownership, bench TLS mode              |
| <u>Linux host hardening</u>         | Ubuntu edge VM, SSH, kernel,<br>Ollama bind |
| <u>Tenable / Nessus remediation</u> | IT scan findings, operator<br>runbook       |
|                                     | Deep ZAP / protected API                    |

| Page                                             | Topic                            |
|--------------------------------------------------|----------------------------------|
| <a href="#">Authenticated scanning (roadmap)</a> |                                  |
| <a href="#">Secrets and auth</a>                 | Env files                        |
| <a href="#">BACnet write allowlists</a>          | Supervisory write gates          |
| <a href="#">Release &amp; LAN scan cycle</a>     | Per-revision ZAP + CI workflow   |
| <a href="#">Logging and audit</a>                | Auth audit trail, rotation, SIEM |

## 119 API routes

## 120 API routes

REST API served by **openfdd-bridge** (default `http://127.0.0.1:8765`).  
Production: Caddy on :80 proxies to the bridge.

**OpenAPI:** GET /docs and GET /redoc when the bridge runs.

**Auth:** JWT via POST /api/auth/login. Most routes require Authorization: Bearer <token>. Roles: viewer, operator, integrator, commission, agent, admin. BACnet writes require commission role + write safety. Dev: OFDD\_AUTH\_DISABLED on loopback only.

**WebSocket:** WS /ws/dashboard — POST /api/auth/ws-ticket, then pass the ticket via Sec-WebSocket-Protocol: ofdd.ws, <ticket>. Query ?ticket= is dev-only when OFDD\_WS\_ALLOW\_QUERY\_TICKET=1. Tickets are short-lived and not interchangeable with the Bearer token.

### 120.1 Health & audit

| Method | Path                | Role   | Description                                                         |
|--------|---------------------|--------|---------------------------------------------------------------------|
| GET    | /health             | public | Minimal liveness (ok, service, version, auth_required)              |
| GET    | /health/stack       | auth   | Stack traffic-light (verbose URLs/bind with OFDD_DEBUG_DIAGNOSTICS) |
| POST   | /api/auth/ws-ticket | auth   | Short-lived WebSocket ticket                                        |
| GET    |                     | auth   | Audit counters                                                      |



| Method | Path               | Role | Description              |
|--------|--------------------|------|--------------------------|
|        | /api/audit/summary |      |                          |
| GET    | /api/audit/events  | auth | Audit log (query params) |
| GET    | /api/audit/errors  | auth | Recent API errors        |

\*Exact public set depends on OFDD\_AUTH\_DISABLED and middleware.

## 120.2 Auth

| Method | Path             | Description              |
|--------|------------------|--------------------------|
| POST   | /api/auth/login  | Username/password → JWT  |
| GET    | /api/auth/me     | Current user             |
| GET    | /api/auth/status | Auth mode / logout hints |

## 120.3 Rules & FDD

| Method | Path                         | Role        | Description                              |
|--------|------------------------------|-------------|------------------------------------------|
| GET    | /api/rules/saved             | user        | Rule index                               |
| POST   | /api/rules/save              | operator+   | Create/update rule metadata              |
| GET    | /api/rules/saved/{id}/source | user        | Python source                            |
| PUT    | /api/rules/saved/{id}/source | operator+   | Write .py file                           |
| GET    | /api/rules/assignments       | user        | Point bindings                           |
| POST   | /api/rules/bind              | integrator+ | Bind rule to point                       |
| POST   | /api/rules/bindings          | integrator+ | Bulk bindings                            |
| POST   | /api/rules/batch             | operator+   | Run all enabled rules → fdd_results.json |

| Method   | Path              | Role | Description |
|----------|-------------------|------|-------------|
| GET/POST | /api/rules/drafts | user | Draft rules |

## 120.4 Rule Lab playground

| Method | Path                         | Description                                        |
|--------|------------------------------|----------------------------------------------------|
| POST   | /api/playground/lint         | AST lint Python rule                               |
| POST   | /api/playground/test-rule    | Run apply_faults_arrow() on feather PyArrow window |
| POST   | /api/playground/run-script   | Execute script mode                                |
| GET    | /api/playground/sample-frame | Sample DataFrame for editor                        |

## 120.5 BRICK / data model

| Method   | Path                   | Description                  |
|----------|------------------------|------------------------------|
| GET      | /api/model/sites       | Site list                    |
| GET      | /api/model/tree        | Equipment tree (SPARQL)      |
| GET      | /api/model/graph       | RDF graph fragment           |
| GET      | /api/model/scope       | Scoped query                 |
| GET      | /api/model/export      | Export TTL/JSON              |
| GET      | /api/model/health      | Model point/equipment counts |
| GET      | /api/model/ttl         | TTL metadata                 |
| POST     | /api/model/sync-ttl    | Regenerate TTL from JSON     |
| POST     | /api/model/import      | Import model payload         |
| POST     | /api/model/sites       | Create site                  |
| GET/POST | /api/model/bacnet-sync | BACnet ↔ model sync state    |

## 120.6 BACnet

| Method | Path                                              | Role       | Description                  |
|--------|---------------------------------------------------|------------|------------------------------|
| GET    | /config/<br>bacnet                                | read       | Commission URL /<br>config   |
| GET    | /api/<br>bacnet/<br>commission/<br>status         | read       | Commission agent<br>health   |
| GET    | /api/<br>bacnet/<br>server/<br>points             | read       | Server point table           |
| GET    | /api/<br>bacnet/<br>inventory                     | read       | Discovered inventory         |
| POST   | /api/<br>bacnet/<br>whois                         | commission | Who-Is                       |
| POST   | /api/<br>bacnet/<br>discover                      | commission | Discover devices             |
| POST   | /api/<br>bacnet/read                              | commission | Read property                |
| POST   | /api/<br>bacnet/<br>read-<br>multiple             | commission | RPM read                     |
| POST   | /api/<br>bacnet/<br>write                         | write      | Supervisory write<br>(gated) |
| POST   | /api/<br>bacnet/<br>import-to-<br>model           | integrator | Inventory → BRICK            |
| POST   | /api/<br>bacnet/<br>driver/<br>sync-<br>discovery | commission | Sync driver tree             |
| POST   | /api/<br>bacnet/<br>poll/once                     | commission | Trigger poll cycle           |
| GET    | /api/<br>bacnet/<br>poll/status                   | read       | Poll worker status           |

| Method | Path               | Role | Description                    |
|--------|--------------------|------|--------------------------------|
| POST   | /ingest/<br>bacnet | read | Ingest samples into<br>feather |

Bind address: BACNET\_BIND in commission env — see [BACnet network setup](#). Operator guide: [Discover and read](#).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |  
 PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |  
 PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |  
 DELETE | /api/bacnet/driver/point/{point\_id} | commission | Remove point from registry |  
 DELETE | /api/bacnet/driver/device/{device\_instance} | commission | Remove device |  
 DELETE | /api/bacnet/driver/registry | commission | Clear CSVs + model sync |

## 120.7 Faults (check-engine)

| Method | Path                    | Description                |
|--------|-------------------------|----------------------------|
| GET    | /api/faults/catalog     | Fixed fault codes          |
| GET    | /api/faults/status      | GREEN/YELLOW/RED aggregate |
| GET    | /api/faults/tree        | Fault tree by equipment    |
| GET    | /api/faults/code/{code} | Code metadata              |

## 120.8 Timeseries & sites

| Method | Path                       | Description         |
|--------|----------------------------|---------------------|
| GET    | /api/timeseries/sites      | Feather sites       |
| GET    | /api/timeseries/series     | Series metadata     |
| GET    | /api/timeseries/plot       | Plot payload        |
| GET    | /api/timeseries/readings   | Raw readings        |
| GET    | /api/sites                 | Site list           |
| GET    | /api/sites/{site_id}/frame | Wide frame for site |

# 120.9 Building & host

| Method | Path                 | Description      |
|--------|----------------------|------------------|
| GET    | /api/building/status | Building summary |
| GET    | /api/building/alerts | Active alerts    |
| GET    | /api/host/stats      | CPU/mem/disk     |

# 120.10 Local agent (Ollama)

Prefix `/openfdd-agent`. Role `agent` for tool execution.

| Method | Path                            | Description                                  |
|--------|---------------------------------|----------------------------------------------|
| GET    | /openfdd-agent/context          | Composed prompt context                      |
| GET    | /openfdd-agent/tools            | Tool manifest                                |
| POST   | /openfdd-agent/tool             | Execute tool (e.g. <code>rules.save</code> ) |
| GET    | /openfdd-agent/ollama/health    | Ollama reachability                          |
| GET    | /openfdd-agent/building-insight | Check-engine narrative context               |
| GET    | /openfdd-agent/zone-temps       | Zone temperature insight                     |
| POST   | /openfdd-agent/chat             | Chat completion (catalog-bound)              |

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see [Architecture overview](#).

# 120.11 Modbus

| Method | Path                    | Role      | Description        |
|--------|-------------------------|-----------|--------------------|
| GET    | /api/modbus/driver/tree | operator+ | Commissioning tree |

| Method | Path                       | Role        | Description                          |
|--------|----------------------------|-------------|--------------------------------------|
| GET    | /api/modbus/poll/status    | operator+   | Poll worker status                   |
| POST   | /api/modbus/poll/once      | integrator+ | Force poll cycle                     |
| POST   | /api/modbus/read_registers | operator+   | One-shot register read               |
| POST   | /api/modbus/read_and_store | operator+   | Read + CSV + feather (source=modbus) |
| POST   | /api/modbus/refresh        | operator+   | Re-read one register                 |
| PATCH  | /api/modbus/register/poll  | integrator+ | Enable poll interval                 |

See [Driver framework](#).

## 120.12 JSON API (HTTP/HTTPS)

| Method | Path                              | Role        | Description                                         |
|--------|-----------------------------------|-------------|-----------------------------------------------------|
| GET    | /api/json-api/env/status          | operator+   | Env var configured flags (no secret values)         |
| POST   | /api/json-api/presets/openweather | integrator+ | Register 3-point OpenWeather bundle + optional poll |
| GET    | /api/json-api/driver/tree         | operator+   | Endpoints grouped by host                           |
| GET    | /api/json-api/poll/status         | operator+   | Poll worker status                                  |
| POST   | /api/json-api/poll/once           | integrator+ | Force poll cycle                                    |
| POST   | /api/json-api/request             | operator+   | One-shot GET/POST (no store)                        |
| POST   | /api/json-api/read_and_store      | operator+   | Request + CSV + feather (source=json_api)           |
| POST   | /api/json-api/refresh             | operator+   | Re-fetch endpoint                                   |
| PATCH  | /api/json-api/endpoint/poll       | integrator+ | Enable poll interval                                |
| DELETE |                                   | integrator+ | Remove endpoint                                     |

| Method | Path                                      | Role | Description |
|--------|-------------------------------------------|------|-------------|
|        | /api/json-api/<br>endpoint/<br>{point_id} |      |             |

**Outbound auth:** `auth_type = none | bearer | basic`; optional `verify_tls` (default `true`). See [JSON API driver](#).

## 120.13 PyPI package (separate)

Library-only HTTP is **not** bundled. See [Python package](#) for `open_fdd.arrow_runtime` (Arrow rules offline).

## 120.14 Errors

JSON error bodies; stack traces hidden unless `OFDD_DEBUG_TRACEBACKS=1`. See [LAN hardening](#).

# 121 Python package

## 122 Python package (open-fdd)

Install from PyPI:

```

pip install open-fdd
# Arrow runtime + playground (default 3.0.1+)
pip install "open-fdd[ml]"
# optional numpy/sklearn for offline graph ML experiments

```

### 122.1 Modules

| Module                              | Purpose                                                                                         |
|-------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>open_fdd.arrow_runtime</code> | <b>Default</b> — <code>apply_faults_arrow</code> on PyArrow Tables, cookbook masks, column maps |
| <code>open_fdd.playground</code>    | Rule Lab lint/compile helpers for Arrow rules                                                   |

Retired in 3.0.1+: `open_fdd.engine` (YAML/pandas RuleRunner) is **not** shipped on PyPI. Use Operator Bridge Rule Lab or import rules from `workspace/data/rules_py/`.

## 122.2 When to use package-only

| Scenario                       | Use                                                                       |
|--------------------------------|---------------------------------------------------------------------------|
| Lint/test Arrow rules offline  | <code>open_fdd.arrow_runtime</code><br>+ <code>open_fdd.playground</code> |
| Graph ML experiments (Layer B) | <code>open-fdd[ml]</code> in<br>experiments/ (see<br>issue #211)          |
| Full building operator stack   | <b>Docker</b> Operator<br>Bridge                                          |

## 122.3 Versioning

Publish: `git tag open-fdd-vX.Y.Z` → GitHub Actions **Publish open-fdd**.

```
import open_fdd
print(open_fdd.__version__)
```

## 122.4 Offline Arrow example

```
import pyarrow as pa
import pyarrow.compute as pc
from open_fdd.arrow_runtime import run_arrow_rule

code = '''
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["SAT"], float(cfg["high"]))
'''

table = pa.table({"SAT": [70.0, 90.0, 88.0]})
result = run_arrow_rule(code, table, {"high": 85})
print(result.flagged_count)
```

API surface: `open_fdd/arrow_runtime` docstrings and [Arrow recipes](#).



# 123 Configuration reference

## 124 Configuration reference

### 124.1 Bridge / auth

| Variable              | Default   | Notes                       |
|-----------------------|-----------|-----------------------------|
| OFDD_BRIDGE_HOST      | 127.0.0.1 | 0.0.0.0 for LAN             |
| OFDD_AUTH_SECRET      | —         | Required for LAN            |
| OFDD_AUTH_DISABLED    | 0         | Localhost dev only          |
| OFDD_INSECURE_LAN_DEV | 0         | Lab only with auth disabled |

### 124.2 BACnet

| Variable                  | Notes                                        |
|---------------------------|----------------------------------------------|
| BACNET_BIND               | ip/mask:47808 in<br>commission.env           |
| OFDD_BACNET_WRITE_ENABLED | Default off — site-specific<br>name may vary |

### 124.3 Docker edge

| Variable                      | Notes                       |
|-------------------------------|-----------------------------|
| OPENFDD_IMAGE_TAG             | GHCR tag for Ansible deploy |
| openfdd_docker_pull_from_ghcr | Ansible host_var            |

Rule Lab Python rules use `rules_store.json + workspace/data/rules_py/*.py`. Legacy YAML schema (`docs/config_schema.json`) is archived — not used by Operator Bridge 3.0.1+.

# 125 Glossary

# 126 Glossary

| Term                   | Definition                                                        |
|------------------------|-------------------------------------------------------------------|
| <b>Operator Bridge</b> | FastAPI service serving API + static dashboard                    |
| <b>Rule Lab</b>        | UI and APIs for Arrow<br>apply_faults_arrow() rules               |
| <b>Feather store</b>   | On-disk columnar historian under<br>workspace/data/feather_store/ |
| <b>fdd_input</b>       | Brick tag linking a point to a rule binding<br>role               |
| <b>Check-engine</b>    | Building-level GREEN/YELLOW/RED fault<br>summary                  |
| <b>Commission</b>      | BACnet discovery/read (and optional<br>write) agent container     |
| <b>Poll driver</b>     | Scheduled BACnet RPM reads into<br>historian                      |
| <b>GHCR</b>            | GitHub Container Registry — ghcr.io/<br>bbartling/openfdd-*       |

# 127 Workspace env files

# 128 Workspace env files

All live under workspace/. Copy from \*.example templates;  
committed \*.local files are gitignored.

| File           | Loaded by                                 | Purpose                                                                                                                       |
|----------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| auth.env.local | run_local.sh, Docker<br>compose, Ansible  | JWT secret<br>(OFDD_AUTH_SECRET),<br>integrator/operator/<br>agent passwords,<br>OFDD_AUTH_DISABLED for<br>localhost dev only |
| data.env.local | run_local.sh, bridge,<br>feather maintain | Feather historian caps<br>(OFDD_FEATHER_*), audit/<br>error log rotation                                                      |

| File               | Loaded by                   | Purpose                                                                                              |
|--------------------|-----------------------------|------------------------------------------------------------------------------------------------------|
|                    |                             | (OFDD_LOG_*), BACnet discovery mutation toggle                                                       |
| json_api.env.local | bridge, Ansible edge sync   | API keys for JSON API driver <code>\${ENV:VAR}</code> substitution (e.g. OPENWEATHER_API_KEY)        |
| ollama.env.local   | run_local.sh, bridge        | Ollama base URL, model name, RAM tier, GPU mode, timeouts — see <a href="#">Ollama and analytics</a> |
| mcp.env.local      | run_local.sh, MCP sidecar   | Enable MCP RAG REST (OFDD_MCP_*), index path                                                         |
| caddy.env.local    | run_local.sh, Ansible edge  | Reverse proxy mode (http / tls / off), TLS CN, port overrides                                        |
| pentest.env.local  | pentest_production_stack.sh | Production-like LAN scan stack: CORS, auth, passive BACnet defaults                                  |

## 128.1 Ansible / edge extras

| Path                                   | Purpose                                                                                |
|----------------------------------------|----------------------------------------------------------------------------------------|
| infra/ansible/secrets/<host>.env.local | SSH deploy credentials, GHCR pull tokens, site-specific BACnet bind — maintainers only |

## 128.2 Related docs

- [Secrets and auth](#) — required production variables
- [Logging and audit](#) — OFDD\_LOG\_\* detail
- [JSON API driver](#) — `${ENV:VAR}` pattern
- [Configuration reference](#) — bridge runtime flags not in env files

# 129 Appendix

# 130 Appendix

Reference material for integrators and library users.

| Page                                    | Content                         |
|-----------------------------------------|---------------------------------|
| <a href="#">API routes</a>              | Operator Bridge REST/WebSocket  |
| <a href="#">Python package</a>          | PyPI open-fdd                   |
| <a href="#">Configuration reference</a> | Key env variables               |
| <a href="#">Workspace env files</a>     | workspace/*.env.local inventory |
| <a href="#">Glossary</a>                | Terms                           |

## 131 GL36 lab site (Acme)

## 132 GL36 lab site (Acme)

**Lab note only.** This documents a maintainer **example edge host** (site acme, building vm-bbartling) used for Guideline 36 supervisory FDD development. Replace inventory host names, IPs, and BACnet ranges for your site.

Reference for **ASHRAE Guideline 36** supervisory monitoring on a live lab edge. Rule code lives in workspace/data/rules\_py/; commissioning uses **Arrow constants** (no config.json).

Trim & respond Niagara reference: [README\\_TRIM\\_RESPOND](#).

### 132.1 BACnet poll scope

Device filter (infra/ansible/scripts/acme\_trim\_devices.sh):

| Range       | Equipment       |
|-------------|-----------------|
| 1-100       | JCI VMA/VAV     |
| 1100        | RTU-01 (AHU)    |
| 1002        | Hot water plant |
| 11000-13000 | Trane UC210 VAV |

Poll interval: **60 s**. Commission from control machine:

```
cd infra/ansible
POLL_INTERVAL=60 ./scripts/acme_commission_gl36.sh
```

Points land in edge\_backup/local/acme/vm-bbartling/points.csv for Ansible sync.

## 132.2 Example FDD rules (Arrow)

### 132.2.1 Duct static pressure high (AHU)

```
"""AHU duct static high – GL36 trim advisory."""

import pyarrow.compute as pc

VALUE_COLUMN = "duct-static"
HIGH_INWC = 1.20
LOOKBACK_HOURS = 3

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin, tmax = pc.min(ts).as_py(), pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_INWC)
```

### 132.2.2 SAT too cold vs setpoint

```
"""AHU SAT too cold vs setpoint."""

import pyarrow.compute as pc

SAT_COLUMN = "sa-t"
SP_COLUMN = "sa-t-sp"
COLD_DELTA_F = 5.0

def apply_faults_arrow(table, cfg, context=None):
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

### 132.2.3 Site rule stubs in the repo

| Rule file                      | Use                     |
|--------------------------------|-------------------------|
| bench_stat_zn-t_flatline_1h.py | Zone temp<br>flatline — |

| Rule file                                           | Use                               |
|-----------------------------------------------------|-----------------------------------|
|                                                     | bind ZN-T per VAV                 |
| acme_duct_static_flatline_1h_gl36_duct_t_r_input.py | RTU duct static flatline          |
| oat_vs_web_spread_1h.py                             | Local OAT vs OpenWeather JSON API |

### 132.2.4 Local OA-T vs web weather (JSON API)

1. Control machine: workspace/json\_api.env.local with OPENWEATHER\_API\_KEY
2. JSON API tab → **OpenWeatherMap** preset → **Register** (default poll **30 min**)
3. Bind BACnet local OAT in the model (oa-t historian column)
4. Enable oat\_vs\_web\_spread\_1h.py in Rule Lab — flags when |oa-t - web-oat-t| > 8 °F

## 132.3 Deploy / upgrade (Ansible, this lab only)

```
cd infra/ansible
OPENFDD_IMAGE_TAG=latest ./deploy.sh docker --limit
<inventory_host> -e enable_bacnet_poll_driver=true
```

Full UI + image upgrade from control machine:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
infra/ansible/scripts/post_deploy_check.sh --limit
<inventory_host> --full
```

Do **not** rsync dev auth.env.local to the edge (deploy\_sync\_auth: false in private host\_vars). Regenerate credentials on the edge host if login fails after upgrade.

## 132.4 AI-assisted data modeling (lab workflow)

1. BACnet poll running (~340 GL36 points @ 60 s on this lab host).
2. **Model & assignments** → export commissioning JSON for LLM-assisted binding.
3. Import returned JSON via commissioning import.

4. Pin FDD rules on ZN-T, DA-T, duct-static, OAT, etc. — export shows readable names in fdd\_rules\_linked.

Model seed: workspace/data/acme\_gl36\_model.json.

## 133 Examples & lab notes

## 134 Examples & lab notes

Site-specific commissioning notes, GL36 examples, and maintainer lab workflows. These pages are **not** required for a generic edge deployment.

| Page                                 | Description                                                                 |
|--------------------------------------|-----------------------------------------------------------------------------|
| <a href="#">GL36 lab site (Acme)</a> | Example real-building poll scope, Arrow rule stubs, Ansible deploy commands |

For production IT operators, start with [Quick Start and Operations](#).

## 135 Arrow-native runtime

## 136 Arrow-native runtime

Open-FDD 3.0 executes Rule Lab rules on **PyArrow tables** end to end.

```
feather_store.read_site_table()
→ fdd_runner / playground.run_arrow_table()
→ apply_faults_arrow(table, cfg, context)
→ pyarrow.compute.masks
```

### 136.1 Authoring contract

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Cookbook helpers: `open_fdd.arrow_runtime.cookbook` (flatline, spread, OOB, after-hours fan).

Script-mode analytics rules receive `table` (PyArrow) and `cfg` in the sandbox — not pandas DataFrames.

## 136.2 Package layout

| Module                                           | Role                              |
|--------------------------------------------------|-----------------------------------|
| <code>open_fdd.arrow_runtime.backend</code>      | Execute rule code, batch chunks   |
| <code>open_fdd.arrow_runtime.cookbook</code>     | Shared fault masks                |
| <code>open_fdd.arrow_runtime.windows</code>      | Rolling min/max, consecutive-true |
| <code>open_fdd.playground.arrow_templates</code> | Rule Lab starter templates        |